

Технически университет – Варна  
Катедра “Компютърни науки и технологии”

---

Васил Смърков, Милена Карова, Ганка Ковачева,  
Виолета Божикова, Валентина Антонова

# **ПРОГРАМИРАНЕ И ИЗПОЛЗВАНЕ НА КОМПЮТРИ 1**

Ръководство за лабораторни упражнения  
(Borland C++/Microsoft Visual C++ 6.0)

---

Варна  
2008

## СЪДЪРЖАНИЕ

|                                                                                                                                                                                              |            |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| <b>Въведение</b>                                                                                                                                                                             | <b>3</b>   |
| <b>Тема 1. Структура на C/C++ програма. Основни типове данни - константи, променливи и функции. Операции и изрази. Програми с използване на стандартни библиотечни функции.</b>              | <b>5</b>   |
| <b>Тема 2. Оператор за присвояване. Входно - изходни операции. Съставяне на линейни програми.</b>                                                                                            | <b>13</b>  |
| <b>Тема 3. Вход и изход на данни. Форматни манипулатори и флагове. Функции за вход/изход на данни.</b>                                                                                       | <b>24</b>  |
| <b>Тема 4. Програмиране на задачи с разклонение (избор). Логически операции и операции за сравнение. Оператори if, switch, goto. Структуриране на програми с избор и използване на меню.</b> | <b>38</b>  |
| <b>Тема 5. Програмиране на задачи с цикли. Оператори за цикъл for, while, do...while. Оператори break и continue.</b>                                                                        | <b>50</b>  |
| <b>Тема 6. Масиви. Символни низове. Функции за работа със символни низове. Програми с масиви и символни низове.</b>                                                                          | <b>63</b>  |
| <b>Тема 7. Указатели. Операции с указатели. Масиви и указатели. Програми с използване на указатели.</b>                                                                                      | <b>74</b>  |
| <b>Тема 8. Функции. Предаване на параметри и връщане на резултати. Използване на указатели като параметри.</b>                                                                               | <b>86</b>  |
| <b>Тема 9. Рекурсия. Съставяне и анализ на програми с рекурсия. Модулна организация на C/C++ програми.</b>                                                                                   | <b>99</b>  |
| <b>Тема 10. Структури. Масиви от структури. Указатели към структури. Обединения. Програми със структури и обединения.</b>                                                                    | <b>109</b> |
| <b>Тема 11. Файлов вход/изход. Функции за работа с файлове. Програми за работа с текстови и двоични файлове.</b>                                                                             | <b>124</b> |
| <b>Тема 12. Програми и функции с побитови операции. Управление на паметта.</b>                                                                                                               | <b>140</b> |
| <b>Приложение А. Програмна среда Borland C++.</b>                                                                                                                                            | <b>151</b> |
| <b>Използвана и препоръчвана литература</b>                                                                                                                                                  | <b>159</b> |

## **Въведение**

Ръководството е предназначено основно за студентите от специалност "Компютърни системи и технологии", изучаващи дисциплината "Програмиране и използване на компютри 1".

В съответствие с учебната програма, основната цел на ръководството е да даде на студентите знания и умения по програмиране с използване на възможностите и приложенията на програмните езици от високо ниво и по-конкретно – на езика C/C++ (версия и програмна среда Borland C++/ Microsoft Visual C++ 6.0).

Във всяка от темите основно внимание е отделено на практическата подготовка на базата на основните принципи на програмирането, алгоритмичния подход при проектирането на програми, структурното и модулното програмиране. Всяка тема съдържа необходимата теоретична част, примерни фрагменти от програми и варианти на програмни реализации, анализ на тяхното изпълнение и очаквани резултати, както и въпроси и задачи за самостоятелна работа.

Особено внимание е отделено на логическата последователност, взаимната свързаност и преходите между отделните теми. В някои от темите умишлено са "повтарят" по-важните елементи, правила и специфични особености на езика за програмиране C/C++. За повишаване на интереса при самостоятелната подготовка на студентите, в част от темите са използвани "изпреварващи" примерни програмни реализации, свързани със следващи теми (съдържащи детайлно представяне на съответни теоретични/практически знания и умения). За по-доброто разбиране и прилагане на основните термини и понятия в програмирането, за значителна част от използваните в ръководството са представени и техните "оригинали" на английски.

За удобство при ползване на ръководството и за самостоятелна работа, в Приложение А е представен вариант на описание на програмната среда Borland C++ с подходящо подобрени примерни програми и режими на работа при тяхното тестване (след компилиране), изпълнение и анализ на резултатите.

Примерните задачи са съобразени с възможностите и интересите на студентите, като е акцентирано на многовариантността на програмната реализация,

включително и с използване на значителна част от вградените стандартни библиотечни функции. За по-добра разбираемост и четливост всяка от програмите и функциите е съпроводена с подробни коментари в описателната и в изпълнимата част. Примерните програми в отделните теми са взаимно свързани, с естествено нарастваща сложност и препратки към предходни и следващи теми, както и с използване на програмни фрагменти и функции, препоръчителни и удобни за самостоятелна практическа подготовка.

Всички примерни програми са тествани чрез съответен набор от реални контролни данни (включително и като файлове с данни) и са на разположение на студентите на flash памет/оптичен диск или в съответна папка на компютрите в учебните зали.

Тези характеристики на ръководството са достатъчно основание за ползването му и от студенти от други специалности, както и за всеки желаещ да разшири първоначалната си практическа подготовка в областта на съвременните езици за програмиране.

Полезно и препоръчително е използването на посочените в ръководството литературни източници, както и на друга допълнителна налична литература с подобно предназначение и съдържание.

Ръководството е под общата редакция на доц. д-р инж. Васил Смърков, като участието на авторите по теми е:

- Тема 1, Тема 9, Тема 12 и Приложение А - доц. д-р инж. В. Смърков;

- Тема 2, Тема 3 и Тема 11 - гл. ас. д-р инж. М. Карова;

- Тема 5 и Тема 6 - гл. ас. д-р инж. Г. Ковачева;

- Тема 4 и Тема 8 - гл. ас. д-р инж. В. Божикова;

- Тема 7 и Тема 10 - гл. ас. инж. В. Антонова.

В процеса на общата редакция в значителна степен са запазени идеите и стила на отделните автори.

Авторите ще приемат с благодарност всички мнения и бележки по съдържанието на лабораторните упражнения в ръководството, както и препоръки с цел подобряването им.

## Тема 1. Структура на C/C++ програма. Основни типове данни - константи, променливи и функции. Операции и изрази. Стандартни библиотечни функции.

**Цел:** Съставяне, тестване (след компилация) и анализ на резултати от изпълнението на C/C++ програми с дефиниране и обработка на основни типове данни и използване на стандартни функции.

**Забележка:** Препоръчително е самостоятелното запознаване с **Приложение А. Програмна среда Borland C++** (вижте на стр. 151) и реализацията на представените и детайлно коментирани **примерни програми**.

### Основни сведения за C/C++ програмите

1. Езикът за програмиране от високо ниво C/C++ е удобен и ефективен за проектиране, ползване и поддържане на различни по сложност **приложни** и **системни** програми. Създаден е от програмисти за програмисти, както начинаещи така и професионалисти [4].

Предоставя удобства и възможности за използване на множество **библиотеки от стандартни функции** (със съответно предназначение) чрез **включване (include)** в програмата на **предпроцесорни директиви със заглавни (headers) файлове**.

Елементите на програмния език C/C++ могат да бъдат представени като "йерархична система", базирана на множество от **допустими символи - букви, цифри и специални символи**, от които по определени **правила** се изграждат **думите на езика (ключови и потребителски)**. От **думите**, по определени **синтактични** правила, се изграждат **"изречения"**, наричани **оператори (statements)** в езика за програмиране (използва се и термина **команди**). Като **последователност от оператори**, в съответствие с определен (избран) **алгоритъм**, се съставят **програми**, съдържащи една (т. нар **main()** - главна) или повече **функции** с определено предназначение.

2. Общият вид на структурата и съдържанието на C++ програма е:

**Директиви на предпроцесора**

**Деклариране на потребителски функции (т. нар. прототипи)**

**Дефиниране на глобални променливи/константи**

**Дефиниция на функция main()**

**Дефиниции на потребителски функции**

Общият вид на дефиниция (програмно описание) на **функция** е:

**тип име([параметри])**

// Заглавие на функцията

{

// Начало на "тялото" на функцията

**Дефиниране на локални променливи/константи**

**Изпълними оператори**

}

// Край на "тялото" на функцията

Типът определя връщания (чрез **оператор** във вида: *return израз*;) **резултат**: **целочислен** (тип `int`), **символен** (тип `char`), **реален** (тип `float` - с "плаваща точка" или `double` - "двойна точност") или **неопределен** (`void` – за т. нар. **функции-процедури**). Чрез **параметрите** се извършва **обмен на данни** (от **указан тип**) **между функциите** в програмата. Възможно е да не се използват параметри (например при използване на **глобални** променливи, "видими" от всички/някои **функции** в програмата).

**Пример: Дефиниция на функция** за пресмятане лице на квадрат със страна `x` (**цяло** число), чиято стойност се предава като **параметър**:

```
int lice_kv(int x)    // Функция за лице на квадрат
{
    int rez;           // Дефиниране на rez - резултат
    rez = x*x;         // Пресмятане на rez
    return rez;        // Връщане стойност на резултата
}
```

### 3. Данните в C/C++ програма са **константи** и **променливи**.

**Променливите** могат да получават различни стойности по време на изпълнение на програмата и задължително се представят чрез **име** (състои се от **букви и цифри** и задължително започва с **буква**). Текущата стойност на променлива се съхранява в съответна **област от паметта**, определена от компилатора.

**Променливите** могат да **получават стойности** чрез: **оператор за присвояване** или чрез **въвеждане** (от **клавиатура** или от **външен файл**) **на данни** със съответни **стойности**. Задължително се дефинират в **програмата** (като **глобални**) или във **функция** (като **локални**, "видими" само във функцията) преди да бъдат използвани:

**тип име\_на\_променлива [= начална стойност];**

**Присвояването на начална стойност** не е задължително.

**Константите** имат **постоянна стойност** и не могат да се променят по време на изпълнение на програмата. Когато е необходимо могат да бъдат **именувани** като се дефинират във вида:

**тип const име\_на\_константа = стойност;**

За **логическо описание** (в "представите" на потребителя) на данните, подлежащи на програмна обработка, се използва понятието **тип на данните**. Типът на данните **определя: допустимите им стойности, допустимите операции с тях, разположението им в паметта и достъпа до тях** [4]. Коя да е от данните, използвана в програмата, може да бъде само от **един единствен тип**. Типовете могат да бъдат **стандартно дефинирани** ("вградени" в програмната система) и **дефинирани от потребителя** (програмиста).

Типовете на данните, използвани в C/C++ програмите са:

- **прости** - с **една** стойност: **числова** (тип `int` за **цели числа**, тип `float/double` за **реални числа**), **символна** (тип `char`) или **логическа** (тип `bool` - в C++ или приемани като аритметични в C програмите със стойност **0** – "неистина" (**false**) или **1** – "истина" (**true**));

- **съставни** (състоят се от **повече от един елементи**, обединени като **множество** от стойности);
- **указатели** (указват мястото на разположение – **адреса** на данните в паметта).

**4. Изрази.** Изразите, както ви е известно, са правила за изчисляване (определяне) на **стойност** – **числова, символна** или **логическа**.

**Израз**, като програмна конструкция, може да бъде **константа, променлива, функция** (която **върща стойност**) или съвкупност от тях, свързани със **знаци за операции** (наричани **оператори**).

Част от израз, заграден в скоби се нарича **подизраз**.

**5. Оператор за присвояване.** Предназначен е да дава (използва се термина “присвоява”) **стойност** на променлива от съответен тип.

**Общият вид** на оператора е: **Променлива = Израз**;

Преди присвояването, изчислената стойност от израза се преобразува (ако се различава) спрямо типа на **променливата**.

**Примерни варианти** на описаните по-горе елементи на езика **C/C++** (версия **Borland C++**) са представени, коментирани и анализирани в следващите програмни реализации. Обхванати са **алгоритми** с използване на **програмни конструкции** за реализация на структури: **последователност** (наричана “верига”), **избор** (разклонение) и **цикъл**.

## Примерни задачи и програми

**Задача 1.1.** Условието на задачата е дефинирано в **заглавния коментар** на представения по-долу вариант на програма. Използвани са **стандартни функции** за **форматирано** въвеждане от **клавиатура** и извеждане на **екрана**, дефинирани в заглавните файлове **stdio.h** (функции: **printf()** за **извеждане на екрана** и **scanf()** за **въвеждане от клавиатура**) и **conio.h** (функции: **clrscr()** за **изчистване на екрана** и **getch()** за **показване на екрана с резултати**). Обработват се **целочислени** (тип **int/long int**) - данни чрез операции **делене** (символ **/**) и **остатък от целочислено делене** (символ **%**). За определяне на **часа, минутите и секундите** се използва оператор за **присвояване**.

```
/* Лабораторно упражнение 1. Задача 1. */
/* C/C++ програма, която по въведено от клавиатурата време в
секунди, определя и извежда точното време, изтекло от 00:00:00
часа на текущото денонощие. */
#include <stdio.h>
#include <conio.h>
void main()
{long int tsek;          /* Дефиниране на времето в секунди - от тип
                           long int */
  int chas, min, sek; /* Целочислени променливи за час, минута,
                           секунда */
  clrscr();           // Стандартна функция за изчистване на екрана
  printf("\n Въведете време в секунди <= 86400: ");
```

```
scanf("%ld", &tsek); /* Въвеждане на tsek като 32 битово цяло
                     число */
chas = tsek/3600;    // Операция деление
min = tsek/60%60;
sek = tsek%60;      // Операция "остатък от целочислено делене"
printf("\n Часът е %d:%d:%d", chas, min, sek);
printf("\n Натиснете клавиш..."); // Подказващ текст
getch();           // Стандартна функция, очаква натискане на клавиш
}
```

Въведете, редактирайте, компилирайте и изпълнете програмата. Тъй като времето в секунди за едно денонощие може да бъде **86400**, променливата **tsek** е дефинирана от типа **long int**. При въвеждането чрез стандартната функция **scanf()** се използва **форматния спецификатор %ld** (**long decimal** – за “дълго” десетично число). За извеждането на часа, минутите и секундите (дефинирани от тип **int**) се използва форматния спецификатор **%d (decimal)**. След като отстранете допуснатите синтактични грешки, опитайте изпълнение на програмата с няколко различни стойности за времето в секунди (примерно **3600**, **3690**, **55555**, **86400** и други) и анализирайте получаваните резултати.

Възможни ли са други варианти за определяне на точното време?

Както забелязвате, в програмата се изпълнява **последователност от оператори** без проверка на някакви условия, т.е. тя е **линейна**.

Опитайте да “усложните” програмата с пресмятане и на изминалите денонощия.

**Задача 1.2.** Условието (представено по-долу като заглавен коментар) и структурата на примерната програма са подобни на тези в предходната задача (реализация на линеен алгоритъм с използване на структура последователност).

Обработват се реални числа (с **цяла** и **дробна** част) чрез използване на **стандартните математически функции** – квадратен корен **sqrt()** (**square root**) и повдигане на степен **pow()** (**power** – връща стойност от тип **double**), дефинирани в **заглавния** файл **math.h**. Тъй като **входните** и **изходните** данни са дефинирани от тип **float**, при въвеждането и извеждането (чрез стандартната функция **printf()**) се използва **форматния спецификатор %f (float)**.

```
/* Лабораторно упражнение 1. Задача 2. */
/* C/C++ програма за изчисляване и извеждане на екрана периметъра
Р и лицето S на равнобедрен триъгълник по въведени от клавиатурата
основа В и височина Н (реални числа, в сантиметри). */
#include <stdio.h>
#include <conio.h>
#include <math.h> /* Включване на математически функции,
                  деклариращи в math.h */

void main()
{float b, h;      // Дефиниране на В и Н, реални числа
 float p, s;     // Дефиниране на реални променливи за резултатите
 clrscr();
 printf("\n Въведете две числа за В и Н на триъгълник: ");
```



```

scanf("%f%f", &b, &h);
p = b + 2 * (sqrt(pow(b/2,2) + pow(h,2))); /* Изчисляване на
                                           периметъра */
s = b * h/2;
printf("\n Основата е %.2f см., а височината - %.2f см.", b, h);
printf("\n Периметърът е %.2f см.", p); /* Извеждане на
                                           периметъра с точност 2 знака след десетичната точка */
printf("\n Лицето на триъгълника е %.2f кв. см.", s);
printf("\n Натиснете клавиш...");
getch();
}

```

Въведете и редактирайте програмата. Компилирайте програмата и отстранете допуснатите **синтактични** грешки. Изпълнете програмата с **няколко варианта** на входните данни (примерно за триъгълник с основа 6 см. и височина 4 см.). Проверете верността на получаваните резултати.

**Задача 1.3.** C/C++ програма за определяне високосна ли е въведена от клавиатурата година.

Годината е **високосна**, ако се дели (**без остатък**) на **4**, но не се дели и на **100** или се дели целочислено на **400**. Очевидно е необходимо използването на **логически израз**, съдържащ операциите **логическо умножение** (операция **and**, представена чрез оператора **&&**) и **логическо събиране** (операция **or**, представена чрез оператора **||**).

В предложения по-долу вариант на програма се използва стандартната функция **копиране на символен низ (strcpy() – string copy)**, дефинирана в заглавния файл **string.h**. Програмата съдържа структура от типа **избор (разклонение)**, реализирана в конкретния случай чрез програмната конструкция – оператор **if...else (ако...иначе)** (вижте **Тема 4**). Годината е високосна (променлива **visgod**, дефинирана като **целочислена**), ако стойността на логическия израз в оператора за присвояване е със стойност **1** (различна от нула - **“ИСТИНА”**).

```

/* Лабораторно упражнение 1. Задача 3. */
/* C/C++ програма за извеждане на екрана "Годината .... е
високосна.", ако въведената от клавиатурата година е високосна или
"Годината .... не е високосна." - в обратния случай. */
#include <stdio.h>
#include <conio.h>
#include <string.h> /* Включване на заглавен файл със стандартни
                     функции за работа със символни низове */

void main()
{int god, visgod;
  char vis[15];          /* Дефиниране на символен низ - масив с
                           размерност 15 символа */

  clrscr();
  printf("\n Въведете цяло число за година: ");
  scanf("%d", &god);
  visgod = ((god%4 == 0) && (god%100 != 0)) || (god%400==0);
  if(visgod) strcpy(vis, "е високосна.");
  else strcpy(vis, "не е високосна.");
}

```

```
printf("\n Годината %d %s", god, vis);
printf("\n Натиснете клавиш...");
getch();
}
```

Въведете и редактирайте програмата. Компилирайте програмата и отстранете допуснатите синтактични грешки. Изпълнете програмата с няколко варианта на въвежданата година, примерно **2000, 2007, 2008, 2010, 3000, 1900, 1600, 1381**. Проверете верността на получаваните резултати.

Опитайте да съставите по-опростен вариант на програмата.

**Задача 1.4.** Условието (представено по-долу като заглавен коментар) и структурата на примерната програма са свързани с обработката на данни от тип **char** (**символен** тип – в случая **unsigned char**). Програмно се представят и **ASCII** кодовете (съответните **еднобайтови** десетични числови стойности) на въвеждани от клавиатурата (в **цикъл**) символи на **букви** (латиница/кирилица).

За организацията на програмната конструкция **цикъл** се използва **оператора за цикъл** с постусловие **do...while** (вижте **Тема 5**). Операторите в **обхвата** на цикъла (**блока** от оператори след **do**) се изпълняват многократно до въвеждане на главна/малка буква **N/n** (латиница/кирилица). Обърнете внимание на изразите за “пресмятане” на предходната/следващата и съответната главна буква и извеждането чрез съответните форматни **спецификатори** (**%c** и **%d**) в стандартната функция **printf()**.

```
/* Лабораторно упражнение 1. Задача 4. */
/* C/C++ програма за въвеждане на малка буква и извеждане на ASCII
   кода на предходната, следващата и съответната ѝ главна буква. */
/* Вариант с цикъл do...while и запитване за продължение */
#include <stdio.h>
#include <conio.h>
void main()
{unsigned char mbukva, gbukva; /* mbukva - малка , gbukva -
                               главна буква */
  char otg;                  // За отговор на запитване - Y/N
  clrscr();
  do{                        // Начало на цикъла със запитване за продължение
    printf("\n Въведете малка буква: ");
    scanf("%c", &mbukva);
    printf("\n ASCII кода на въведената буква %c е %d",
           mbukva, mbukva);
    printf("\n ASCII кода на предходната буква %c е %d",
           mbukva-1, mbukva-1);
    printf("\n ASCII кода на следващата буква %c е %d",
           mbukva+1, mbukva+1);
    gbukva = mbukva - ('a'-'A'); /* Разликата между малка и
                                голяма буква е 32 */
    printf("\n Съответната главна буква е %c", gbukva );
    printf("\n ASCII кода на буквата %c е %d", gbukva, gbukva);
    printf("\n \n Желаете ли да продължите? Y/N: ");
```

```

printf("\n За отказ въведете N/n, за продължение - Enter: ");
fflush(stdin);          /* Изчистване на буфера на stdin за
                           въвеждане на следващи данни */
scanf("%c", &otg);
}while(!(otg == 'N' || otg == 'n' || otg == 'H' || otg == 'h'));
}

```

Въведете и редактирайте програмата. Компилирайте програмата и отстранете допуснатите синтактични грешки. Изпълнете програмата с няколко варианта на входни данни – избрана от вас малка латинска/кирилска буква. Проверете верността на получаваните резултати. Модифицирайте и изпълнете вариант на програмата с разликата (променлива с име **razi**) на кои да е други малка и главна буква от латинската азбука. Проследете и анализирайте резултата.

**Забележете**, че с данните от тип **char**, могат да се извършват аритметични операции на цели числа (в случая събиране и изваждане).

**Задача 1.5.** Условието (представено по-долу като заглавен коментар) и структурата на примерната програма са свързани с обработката на данни от тип **double** (**реални** числа с “двойна” точност).

За пресмятане броя на чувалите със жито се използва програмната конструкция **цикъл**, реализирана в програмата чрез **два вложени** оператора (**външен** и **вътрешен**) за цикъл **for** (т. нар. **броячен** цикъл).

За извеждане на резултата е използван **потока cout** (дефиниран в заглавния файл **iostream.h**) за реализация на **оператор за извеждане** във вида: **cout << израз1 << израз2;**

```

/* Лабораторно упражнение 1. Задача 5. */
/* C++ програма за пресмятане броя на чувалите със жито, които
според легендата "хитрият" цар пожелал да подари на автора на
шахматната игра. Шеговито авторът на шахматната игра поискал от
царя толкова зърна жито, колкото се получава, ако на първото
квадратче се "постави" едно зърно, на второто - два пъти повече и
т.н., като на 64-тото квадратче се "поставят" 64 пъти повече
зърна, отколкото на 63-тото (т.е. 64!). */
/* Необходимо е да се пресметне сумата на факториелите от 1! до
64!. Ако се приеме, че в един килограм има 5000 житени зърна, то в
чувал от 50 кг. зърната в един чувал ще бъдат 250000. */
/* Вариант с два вложени цикъла for - външен е индекс (брояч) i и
вътрешен с индекс j. */
#include <iostream.h>
#include <conio.h>

void main()
{double ch_jito=0.0, zyrna; // Променливата ch_jito е за чувалите
clrscr();
for(int i=1; i<=64; i++) // Външен цикъл, i се изменя от 1 до 64
{zyrna = 1.0;           // Начална стойност - 1 зърно
for(int j=1; j<=i; j++) // Вътрешен цикъл за пресмятане на i!
zyrna = zyrna * j;      // Операция умножение
ch_jito = ch_jito + zyrna; /* Натрупване на сумата от
                           факториелите */
}

```

```
cout << "\n\t Чувалите със жито са " << ch_jito/250000;
cout << "\n\t Натиснете клавиш...";
getch();
}
```

Въведете, редактирайте, тествайте и изпълнете програмата. За изпълнението на програмата не е необходимо въвеждането на **входни данни**. Със сигурност резултатът ще ви изненада. Броят на чувалите със жито, които е трябвало да подари “хитрият” цар, е  **$5.156061 * 10^{83}$** !!!

Ако се съмнявате във верността на програмно получения резултат, променете броя на циклите във външния цикъл от **64** на **10**. Изпълнете модифицираната програма и сравнете резултата с получения, като използвате например програмата **Calculator** в среда **Windows** (последователно пресмятане и сумиране на факториелите от **1!** до **10!** и разделяне на сумата на **250000**).

Съставете и изпълнете вариант на програмата като замените двата вложени цикъла **for** със съответни цикли с предусловие **while**.

## Въпроси и задачи за самостоятелна работа

1. На кои основни елементи, като йерархична система, се базира езика за програмиране **C/C++**?
2. Какво е предназначението на заглавните (headers) файлове, включвани в **C/C++** програмите?
3. С коя предпроцесорна директива се включват заглавни (headers) файлове в **C/C++** програмите?
4. Какви са различията при дефинирането и използването на константите и променливите в **C/C++** програма?
5. Кои дефинирани променливи са локални?
6. Какво определя типа на данните в езика за програмиране **C/C++**?
7. Кои са типовете данни, които могат да бъдат дефинирани и използвани в езика за програмиране **C/C++**?
8. Какво е предназначението на оператора за присвояване и какво е характерно при изпълнението му?
9. Кои са основните стандартни функции за въвеждане/извеждане на данни в **C/C++** програма, декларирани в заглавните файлове **stdio.h** и **conio.h**?
10. Какво е предназначението на заглавният файл **math.h** и какви стандартни функции са декларирани в него?
11. Защо в Задача 1.5. променливата за чувалите със жито **ch\_jito** е дефинирана от тип **double**? Възможно ли е да бъде от тип **float**?
12. Съставете и изпълнете програма за намиране сумата **sumc** от цифрите на трицифрено число, въвеждано от клавиатурата.
13. Съставете програма за изчисляване периметъра **P** на правилен многоъгълник по въведени от клавиатурата брой страни - **br** и размер на страната - **str**.
14. Съставете програма за изчисляване сумата от **ASCII** кодовете на буквите в името на специалност КСТ.

## Тема 2. Оператор за присвояване. Входно - изходни операции. Съставяне на линейни програми.

**Цел:** Запознаване с оператора за присвояване и входно – изходни операции, използвани в езика **C/C++** при съставяне на линейни и други програми с различна структура и сложност.

### Оператор за присвояване

От предходната тема знаете, че **оператор за присвояване** (представя се със символа **=**) може да се използва за присвояване (задаване) на стойност на **променлива**.

**Променливата** се записва от лявата страна на **=**, а от дясната се записва **израз**:

**променлива = израз;**

**Пример:** `total = sum*1.2;`

Променливата **total** трябва предварително да е с дефиниран **тип**. Възможно е **дефинирането** да се осъществи в самия **оператор**:

**double total = sum\*1.2;**

**Запомнете:** Не трябва да се бърка **оператора за присвояване** с **оператора за сравнение равно ли е (==)**!

Възможно е **многократно присвояване** (ако в един израз се използват **два** или **повече** символа за присвояване **=**). **Многократните** присвоявания определят **ред на изпълнение** на действията от **дясно на ляво**.

**Пример:** `a = b = c = d = 10;`

Присвояването се извършва в следната **последователност**: числото **10** се присвоява на **d**; стойността на **d** се присвоява на **c**; стойността на **c** се присвоява на **b**; стойността на **b** се присвоява на **a**.

Възможно е използването на оператор за присвояване на който в лявата част **променливата** е от **един тип**, а в дясната - константа, променлива или израз от **друг тип**. В този случай определящ е **типа на променливата от лявата страна** на оператора за присвояване (понякога това води до **загуба на точност**).

**Пример:** `int a ; float b = 5.7833;`

`a=b;`

`cout << a; //` Полученият резултат, който се извежда на екрана е **5**

Ако се „смесят“ типовете **int** и **char**, компилаторът работи с **ASCII** кодовете на символите (числови стойности от **0** до **255**).

**Пример:** `int a ; char b = 'Z';`

`a=b;`

`cout << a; //` Извежда се ASCII кода на буквата Z – числото **90**

**Удобно и препоръчително** е използването на т. нар. **съставен оператор** за присвояване (когато **променливата** от лявата страна се използва в **израза** в дясно). **Съставните аритметични** присвоявания,

използвани в **C/C++** програмите, са **5** на брой (**+=**, **-=**, **\*=**, **/=**, **%=**) и по приоритет на действията заемат най-ниска позиция.

**Пример:**

|                                |                         |
|--------------------------------|-------------------------|
| <code>bonus = bonus + 2</code> | <code>bonus += 2</code> |
| <code>bonus = bonus - 2</code> | <code>bonus -= 2</code> |
| <code>bonus = bonus * 2</code> | <code>bonus *= 2</code> |
| <code>bonus = bonus/2</code>   | <code>bonus /= 2</code> |
| <code>bonus = bonus%2</code>   | <code>bonus %= 2</code> |

### 1. Явно преобразуване на типовете.

При **неявното** преобразуване на типовете, данните се преобразуват към типа с **най- висок приоритет**.

При **явното (typecasting)** се използват **два** формата:

**(тип на данните) израз** и

**тип на данните (израз),**

където тип на данните може да бъде всеки валиден в **C/C++** тип (**int**, **char**, **float** или **double**), а **изразът** може да е променлива, константа или комбинация от **променливи** и **константи**, свързани с **операции**. Явната промяна типа на данните е **временна**, само за участието им в **израза**.

**Пример:**

```
double chastno;
```

```
int a,b;
```

```
...
```

```
chastno=(double) a/b;
```

```
или chastno=double (a) /b;
```

```
cout<<a;
```

Възниква проблем при оперирането с данни от **различни типове**. Променливите **a** и **b** са от цял тип. При действие **деление** се получава резултат от **цял** тип (губи се дробната част). Операторът за явно преобразуване на типа **double**, преобразува **само временно** типа на **a** при участието ѝ в израза. Така се получава частно от тип **double**, което се присвоява на променливата **chastno**. Операторът **cout << a**, извежда цяла стойност на променливата **a** (дефинирана е от тип **int** - цял тип).

### Входно - изходни операции

Входно - изходната система на **C++** работи чрез **потоци** от информация [5]. **Потокът (stream)** е свързан към дадено физическо устройство (най – често **клавиатура** като **входно** и **монитор** като **изходно** устройство, наричани общо **конзола**). Той представлява последователност от символи или други данни. Ако потокът е насочен навън от програмата, той е **output stream**; ако е насочен към програмата – **input stream**. Операторът **cin >>** (нарича се **оператор за въвеждане**) се използва за въвеждане на данни **от клавиатурата**, а **cout <<** (нарича се **оператор за извеждане**) - за извеждане на данни

на **екрана на монитора** (“подават” се към екрана отляво надясно). Двете входно - изходни операции са дефинирани във файла **iostream.h**.

Забележете, че **cout <<** и **cin >>** извършват противоположни действия.

### **Примери:**

```
float salary;  
cin >> salary;
```

Операторът **cin >>** очаква потребителя да въведе стойност от клавиатурата за променливата **salary** (заплата).

```
cout << " Моята заплата е: " << salary;
```

Извежда се символен низ и реално число на екрана на монитора.

Често **cin >>** и **cout <<** се използват по двойки. Чрез **cout <<** се извежда **подканващо съобщение** към потребителя какво той трябва да въведе. След това чрез **cin >>** се очаква **въвеждане** на данните:

```
cout << "Каква е вашата заплата?";  
cin >> salary; // Очаква потребителя да въведе число
```

В това упражнение са използвани операторите за **вход/изход cin >>** и **cout <<** (с **формати на данните** “по подразбиране”). Възможно е използването и на **printf()** и **scanf()** стандартни функции (дефинирани в заглавния файл **stdio.h**) за **форматиран вход/изход**.

## **Примерни задачи и линейни програми**

**Задача 2.1.** Какъв ще бъде резултатът от изпълнението на следните операции в **C/C++** програмата:

```
6/2  
6%2  
29/4.0  
29.0/4  
29.0/4.0  
29%6
```

### **Примерен вариант на решение:**

```
#include <iostream.h>
```

```
void main()  
{ cout << 6/2;           // Целочислено деление  
  cout << "\n";  
  cout << 6%2;           // Остатък от целочислено деление  
  cout << "\n";  
  cout << 29/4.0;         // Обикновено деление (на реални числа)  
  cout << "\n";  
  cout << 29.0/4 << "\n"; // Обикновено деление  
  cout << 29/4 << "\n";   // Целочислено деление  
  cout << 29.0/4.0 << "\n"; // Обикновено деление  
  cout << 29%6 << "\n";   // Остатък от целочислено деление  
}
```

Получават се следните **резултати** (активирайте **Alt+F5**):

```
3           // Целочислено деление
0           // Остатък от целочислено деление
7.25        // Обикновено деление (на реални числа)
7.25        // Обикновено деление
7           // Целочислено деление
7.25        // Обикновено деление
5           // Остатък от целочислено деление
```

**Целочисленото деление** се характеризира с операнди **две цели числа**, при което и резултатът е **цяло число**. При **обикновеното деление** (деление на числа с **плаваща точка**) поне единият операнд е от тип **float**, или **double** и резултатът е от тип **float** или **double**.

Обърнете внимание в последните **4** реда на програмата – **управляващият символ** за преминаване на нов ред **"\n"** (new line) може да се постави в списъка на един и същ оператор **cout <<** заедно с извеждането на стойността на израза.

**Задача 2.2** Съставете програма на **C/C++** чрез която се въвежда вашето име, фамилия и тегло и същите се извеждат в подходящ вид и с пояснителен текст на екрана на монитора.

#### Примерен вариант на решение:

```
/* C/C++ програма за въвеждане на име, фамилия и тегло и
извеждането им в подходящ вид на екрана */
#include <iostream.h>

void main ()
{ char ime[20];
  char familia[20];
  int teglo;
  cout << "\n" << " Въведете своето име";
  cin.getline(ime,20);           // Въвежда се името
  cout << "\n" << " Въведете своята фамилия";
  cin.getline(familia,20);       // Въвежда се фамилията
  cout << "\n" << " Въведете своето тегло";
  cin >> teglo;                  // Въвежда се теглото
  cout << "\n" << ime << " " << familia << " има тегло" << " "
    << teglo << " кг." << "\n";
}
```

Чрез декларативният оператор **char ime[20]** се дефинира последователност от **20** символа, наречена **масив от символи**, като символите заемат **последователни байтове в паметта**. При дефиниране на масива се използват квадратни скоби **[размер\_на\_масива]** за запазване на съответно място в паметта за масива. Масивът е с дължина **20** символа и името му е **ime**. Той трябва да съдържа низ от символи, който **не може** да му се зададе с обикновен оператор за присвояване (освен при **дефиниране с инициализация**). Символните низове са представени по-детайлно в **Тема 6**. Въведете, компилирайте и изпълнете програмата.



Възможно е да използвате функцията **strcpy()** (от заглавния файл **string.h**) за копиране на символен низ.

Функцията **getline()** е за въвеждане на **цял ред** от символи (вижте **Тема 3**). Във вида **cin.getline(string, num)** **cin** се разглежда като **обект (клавиатура)** от който се въвеждат определен брой (**num**) символи наведнъж. В случая от един ред се въвеждат до **20** символа за името. По аналогичен начин се формира стойността на масива **familia[20]**.

**Задача 2.3.** Професор провежда изпит в университет. Изпитва 10 студента, по учебна дисциплина. Съставете програма на **C/C++**, която очаква да въвеждате оценките за 10 студента, намира и извежда средно-аритметичната оценка по учебната дисциплина.

#### Примерен вариант на решение:

```
#include <iostream.h>

void main ()
{ int oc1, oc2, oc3, oc4, oc5, oc6, oc7, oc8, oc9, oc10;
  float srusp=0.0; // Начална стойност 0.0 на променливата srusp
  cout << "\n Оценка1: ";
  cin >> oc1;
  srusp=srup+oc1;
  cout << "\n Оценка2: ";
  cin >> oc2;
  srusp+= oc2;
  cout << "\n Оценка3: ";
  cin >> oc3;
  srusp+= oc3;
  // За останалите 7 оценки напишете същите оператори.
  cout << "\n Средният успех по дисциплината е: "<< srusp/10;
}
```

Последният оператор **cout** може да запишете по друг начин като използвате изходния **манипулатор setprecision()** (вижте **Тема 3**). Той ограничава точността на изхода, зададена със **cout**. Вътре в скобите посочвате с **цяла константа** или с променлива колко **цифри след десетичната точка** искате да има в извежданото число. **Манипулаторът** изисква включването на заглавния файл **iomanip.h**. Операторът **cout <<** ще изглежда по следния начин при зададена точност **два** знака след десетичната точка (за среда **Visual C++ 6.0 setprecision()** включва и **позицията на десетичната точка!**):

```
cout << "\n Средният успех по дисциплината е: " <<
      setprecision(3) << srusp/10;
```

В следващата програма може да дефинирате променливата **srusp** от тип **int** и да използвате явно преобразуване на типовете (**typecasting**):

```
#include <iostream.h>

void main ()
{ int oc1, oc2, oc3, oc4, oc5, oc6, oc7, oc8, oc9, oc10, srusp=0;
  cout << "\n "Оценка1: ";
```

```

cin >> oc1;
srusp=srusp+oc1;
.....
cout << "\n Средният успех по дисциплината е: "
      << float(srusp)/10;
}

```

Целочислената променлива **srusp** се преобразува явно за да се получи резултат **реално число**. Същият резултат ще се получи ако напишем **srusp/10.0**. Ако напишете **srusp/10**, резултатът ще бъде **цяло число**, понеже участващите операнди са **цели числа**.

Възможна е и следната **грешка**:

```
float (srusp)=srusp/10;
```

Резултатът от делението е **цяло число**, дробната част се **отхвърля** и след това се присвоява на **реално** число, чиято дробна част е **0**.

Възможно е вместо използването на **10** променливи от цял тип **oc1**, **oc2**, **oc3** и т. н. да използвате една **единствена** променлива и име **oc**.

**Задача 2.4.** Трансформирайте следващите формули в еквивалентни оператори за присвояване в **C/C++** програма:

$$z = (a - b) \cdot (a - c) \cdot 2$$

$$f = \frac{a^2}{b^3}$$

$$d = \frac{3+3}{4+4} \cdot \frac{(8-x^2)(4+2-1)}{(x-9) \cdot x^3}$$

Записването на горните изрази в езика **C/C++** изглежда по следния начин:

```
z=(a-b)*(a-c)*2
```

```
f=pow(a,2)/pow(b,3)
```

```
d=(3+3)/(4+4)*(8-pow(x,2))*(4+2-1)/((x-9)*pow(x,3))
```

Знаете, че при използването на стандартни **математически** функции като **sqrt()** (**корен квадратен**) или **pow()** (**степенуване**) е необходимо включването в програмата на заглавния файл **math.h**.

**Примерен вариант на програмно решение:**

```
#include <iostream.h>
```

```
#include <math.h>
```

```
void main()
```

```
{ float a,b,c;
```

```
float x,z,f;
```

```
double d;
```

```
cout << "\n" << "Въведете стойности за a,b,c и x на един ред,
                разделени с празен интервал: ";
```

```
cin >> a >> b >> c >> x;
```

```
z=(a-b)*(a-c)*2;
```

```
f=a*a/pow(b,3);
```

```
d=((3+3)/(4+4))*(8-x*x)*(4+2-1)/((x-9)*pow(x,3));
```

```
cout << "\n z = " << z;
```

```
cout << "\n f = " << f;
```

```
cout << "\n d = " << d;
```

```
}
```

**Задача 2.5.** Какъв е изходът от следващата **C/C++** програма с вход и изход?

```
#include <iostream.h>

void main()
{   char title[12];
    cin.getline(title,6);
    cout << title << "\n";
}
```

Посредством функцията `getline(title,6)` в паметта на компютъра ще се въведат само първите **6** символа от реда, който ще наберете от клавиатурата. Тези символи се присвояват на **СИМВОЛНИЯ** низ (stringa - **string**) `title`. Изпълнете програмата с различни низове.

**Задача 2.6.** Каква стойност ще има променливата `c` след изпълнението на следната програма:

```
#include <iostream.h>

void main()
{   int c;
    c = 'A'+5;
    cout << c;
}
```

Числото **5** се прибавя към **ASCII** кода на символа **'A'** (десетичното число **65**). Стойността на `c` се получава **70**. В **C/C++** е възможно **смесването** на типа `char` с `int` в аритметични изрази.

**Задача 2.7.** Какъв е изходът от следната програма:

```
#include <iostream.h>
#define X1 a+b
#define X2 X1*X1

void main ()
{   int a=2;
    int b=3;
    cout <<a<< ", " << b << ", " << (X2) << "\n";
}
```

**Решение:** Използвайки `X2` (от директивата `define X2`) в `cout`, то тя се замества от `a+b*a+b`. Като резултат ще се получи оператора `cout` във следния вид: `cout <<a<<","<<b<<","<<(a+b*a+b)<<"\n"`. На екрана на монитора ще се появи резултат във вида: **2,3,11** (след **Alt+F5**).

**Задача 2.8.** Какъв е типът на резултата в изразите на следващата **C/C++** програма, където `m`, `n`, `k`, `l` са променливи от различен тип?

```
#include <iostream.h>

void main()
{   int k=4;           // Целочислен тип
    float l=4.5;       // Реално число с плаваща точка
    char m='A';        // Символен тип
    double n=4.888888;  // Реално число с двойна точност
}
```

```

    cout<<"\n"<<n+1;
    cout<<"\n"<<n+k;
    cout<<"\n"<<k+m;
    cout<<"\n"<<n*m;
    cout<<"\n"<<k+l+m+n;
}

```

Типът **double** (реални числа с "двойна" точност) има най-висок приоритет и затова ако в един израз участват стойности от тип **int** и **double**, то резултатът се получава от тип **double**.

**Резултати** (активирайте **Alt+F5** след изпълнение на програмата):

9.38889

8.88889

69 // Към цялото число k се прибавя ASCII кода на буквата 'A'

317.778

78.3889

**Забележка:** Дефиниране с инициализация е възможно и във вида (например в случая за променливата **k**): **k (4) ;**

**Задача 2.9.** Нека **n** е променлива от тип **int**, а **x** - променлива от тип **double**. Обяснете разликата между:

**n=x;** и

**n=int(x+0.5);**

За кои стойности на **x** двата оператора дават един и същ резултат? За кои стойности на **x** те се различават? Какво става, ако **x** е отрицателно? За целта съставете и изпълнете с различни стойности на променливата **x** вариант на **C/C++** програма.

**Примерен вариант на програмно решение:**

```
#include <iostream.h>
```

```

void main()
{int n;
 double x;
 cout<<"\n Въведете реално число: ";
 cin>>x>>"\n";
 n=x;
 cout<<"\n"<<n;
 n=int(x+0.5); //x+0.5 е от тип double, но int го превръща в цял
 cout<<"\n"<<n;
}

```

**Разлика** между резултатите от двата оператора за присвояване, при **x** **положително число** няма. Получават се **два еднакви** резултата от **цял тип**. В първия случай определящ е типа на променливата от лявата страна на оператора за присвояване, а във втория – се използва явно преобразуване на типа.

При всяка стойност на **x** от вида **XX...X.0** (дробната част е **0**), резултатите от двата оператора са еднакви.

При стойност на **x отрицателно число**, резултатите от двата оператора за присвояване са различни (за **x=-2.0 n=-2** и **n=-1**). Само когато дробната част на **x** е **0.5**, тогава се получават еднакви резултати.

Изпълнете програмата с различни стойности на въвежданото число **x** (положителни и отрицателни) и анализирайте резултатите.

**Задача 2.10.** Съставете **C/C++** програма, в която се определят **цифрите** на въведено от клавиатурата **трицифрено** число и те се извеждат на екрана.

**Примерен вариант на програмно решение:**

```
#include <iostream.h>

void main(void)
{int chislo,edinici,desetici,stotici,temp;
  cout << "\n Въведете цяло трицифрено число: ";
  cin >> chislo;
  edinici = chislo%10;          /* Остатък от делението на числото
                                представлява цифра на единиците */
  //temp = chislo/10; /* Този ред може да отпадне, ако операторът
                        за присвояване в следващия ред се използва
                        в израза за изчисляване на десетиците. */
  desetici = (temp = chislo/10)%10; /* Използване на оператор за
                                     присвояване в аритметичен израз */
  stotici = temp/10;
  cout << "\n" << "Цифра на единици: " << edinici;
  cout << "\n" << "Цифра на десетици: " << desetici;
  cout << "\n" << "Цифра на стотици: " << stotici;
}
```

Изпълнете програмата с различни стойности на въвежданото **трицифрено** число и анализирайте резултатите.

**Задача 2.11.** Работник работи в магазин за бяла техника 50 часа седмично. Първите 40 часа се плащат по тарифа 2.50 лв. за 1 час, следващите 5 часа след 40 се плащат един път и половина повече и двойно повече за часовете над 45. Ако приемем, че данъчният процент е 29, напишете програма, която извежда на екрана брутната заплата, данъците и нетната заплата.

**Примерен вариант на програмно решение:**

```
#include <iostream.h>

void main(void)
{
  float stavka = 2.50,zaplata,danaci,neto;
  zaplata = 40*stavka+5*1.5*stavka+5*2*stavka;
  cout << "\n" << "Заплатата е: " << zaplata << "лв.";
  danaci = zaplata*0.29;
  cout << "\n" << "Данъци: " << danaci << "лв.";
  neto = zaplata-danaci;
  cout << "\n" << "Чиста сума: " << neto << "лв.";
}
```

**Резултатите са следните:**

**Заплатата е: 143.75лв.**

**Данъци: 41.6875лв.**

**Чиста сума: 102.063лв.**

За да се поставят два знака след десетичната точка в резултатите за заплатата, данъците и чиста сума е необходимо вмъкването на манипулатора **setprecision(2)** (за среда **Borland C**) и **setprecision(3)** (за среда **Visual C++ 6.0**) и хедърния файл **iomani.h** (вижте **Тема 3**).

В този случай резултатите са следните:

**Заплатата е: 143.75лв.**

**Данъци: 41.69лв.**

**Чиста сума: 102.06лв.**

**Задача 2.12.** Напишете **C/C++** програма, която подканва потребителя да въведе число с плаваща точка **x** и след това извежда:

- $\sin(x)$ ;
- $\cos(x)$ ;
- $\tan(x)$ ;
- $\sinh(x)$ ;
- десетичен логаритъм от  $x$ ;
- абсолютната стойност от  $x$ ;
- най – близкото цяло число по - малко от  $x$ ;
- най – близкото цяло число по - голямо от  $x$ ;

**Примерен вариант на програмно решение:**

```
#include <iostream.h>
```

```
#include <math.h>
```

```
void main()
```

```
{float x;
```

```
cout << "\n Въведете реално число: ";
```

```
cin >> x;
```

```
cout << "\n Синус от " << x << " е " << sin(x);
```

```
cout << "\n Косинус от " << x << " е " << cos(x);
```

```
cout << "\n Тангенс от " << x << " е " << tan(x);
```

```
cout << "\n Хиперболичен синус от " << x << " е " << sinh(x);
```

```
cout << "\n Десетичен логаритъм от " << x << " е " << log10(x);
```

```
cout << "\n Абсолютна стойност от " << x << " е " << fabs(x);
```

```
cout << "\n Най – близкото цяло число по-малко от " << x <<  
" е " << ceil(x);
```

```
cout << "\n Най – близкото цяло число по-голямо от " << x <<  
" е " << floor(x);
```

```
}
```

Въведете и изпълнете програмата с различни стойности на въвежданото число **x** и анализирайте резултатите. Не забравяйте, че за тригонометричните функции **x** трябва да бъде в **радиани**!

## Въпроси и задачи за самостоятелна работа

1. Каква е разликата между унарна и бинарна операция в C/C++ програма?

2. Каква е разликата при въвеждане на низ от символи чрез операцията `cin >>` и чрез `cin.getline()`?

3. Какъв е изходът от следната програма?

```
#include <iostream.h>
#define ANT1 a+a+a
#define ANT2 ANT1-ANT1
void main()
{ int a=1;
  cout<<"Сумата е "<< ANT2<<"\n";
}
```

4. Еквивалентни ли са двата оператора за присвояване?

```
sales*=4-factor+bonus; И
sales=sales*4-factor+bonus;
```

5. Еквивалентни ли са двете декларации за типа на променливите?

```
double rate=0.07, time, balance=0.00; И
double rate(0.07), time, balance(0.00);
```

6. Коректна стойност ли ще получи променливата  $v=6+(m=9-a)$ , ако  $v$ ,  $m$  или  $a$  са променливи от цял тип?

7. Какъв е резултатът от изчислението на всеки от следните изрази?

$9\%2 + 1$   
 $(1 + (10 - (2 + 2)))$

8. Как можете да получите последната цифра на цяло число? А първата? С други думи, ако  $n$  е 56324, как се получава 4 и 5? Съставете вариант на C/C++ програма.

9. Трансформирайте следващата формула в еквивалентен оператор за присвояване в C/C++ програма (включете и вход и изход на данни):

$$x = \frac{y * 2 + 5}{14 - 6/3}$$

10. Напишете програма, която пита потребителя за дължините на две съседни страни на правоъгълник и след това извежда:

- лицето и периметъра на правоъгълника;
- дължината на диагонала на правоъгълника (използвайте Питагоровата теорема).

11. Напишете програма, която пресмята и извежда лицето на разностранен триъгълник със страни 8, 9, и 10. Използвайте Хероновата формула за намиране лице на триъгълник.

12. Напишете програма, която извежда всяка от първите 10 степени на 2 ( $2^1, 2^2, 2^3, \dots, 2^{10}$ ). Напишете коментари и включете името си в началото на програмата. Изведете константи низове, които описват всеки от изведените резултати. Първите два реда от вашата програма трябва да изглеждат така:

2 на първа степен е 2  
2 на втора степен е 4 и т. н.

### Тема 3. Вход и изход на данни. Форматни манипулатори и флагове. Функции за вход/изход на данни.

**Цел:** Запознаване с входно - изходната система на **C/C++** и с колекцията от функции от стандартната библиотека, реализиращи конзолен вход/изход на данните.

#### Входно/изходни потоци, манипулатори и флагове

Прехвърлянето на данни от едно място на друго (работа с **клавиатура**, **монитор**, **файлове**, прехвърляне между две области от **паметта** и други) в **C++** се решава посредством **обекти**, наречени **потоци (streams)**. **Източникът** на данни записва данните в потока, а **приемникът** ги “взима” от потока. Така се постига унифициране на операциите за обмен на данни, **независимо** от вида на **източника** и **приемника** на данните.

В **C/C++** програмите всеки **вход** и **изход** се третира като **поток от символи**. Това означава, че можете да използвате един и същи функции за въвеждане на **входни данни** от **клавиатура** или **модем**; едни и същи функции за **запис (извеждане)** в **дисков файл**, на **екран** или на **принтер**.

**Конзолата** е основният интерфейс на компютъра (**C/C++** програмата) и се състои от **клавиатура** и **екран**. Обикновено **клавиатурата** е стандартното **входно устройство** (източник на данни), а **екранът** – стандартното **изходно устройство** (приемник на данни).

За реализация на входно/изходни операции се използват **два** начина :

- чрез **стандартни** библиотечни **функции**;
- чрез операциите **>> (въвеждане)** и **<< (извеждане)** на данни.

Стандартно всяка програма автоматично поддържа **4** потока:

- **cin** - **входен** поток (аналог на файла **stdin** в **C**);
- **cout** - **изходен** поток (аналог на файла **stdout** в **C**);
- **cerr** - поток за **грешки** (аналог на файла **stderr** в **C**);
- **clog** - **буфериран cerr**.

Програмната среда **C/C++** осигурява поддръжката на входно - изходната система чрез **заглавния файл iostream.h (входно-изходен поток)**. В този файл са дефинирани множество стандартни функции, поддържащи входно/изходните операции.

В **C++** съществуват **два** начина за постигане на удобна функционалност на входно/изходните операции [6]:

- чрез промяна на **флаговете** за **състоянието на потока**;
- чрез използване на **манипулатори**.

Вторият начин е по- удобен за промяна на текущото състояние на потока. Управляват се директно **флаговете** на **състоянието на**



**потока.** За да могат да се използват манипулатори, в **C/C++** програмата трябва да се включи **заглавния файл iomanip.h**.

По-използвани манипулатори са:

- **dec** десетично (**decimal**) преобразуване
- **hex** шестнадесетично (**hexadecimal**) преобразуване
- **oct** осмично (**octal**) преобразуване
- **endl** нов ред (**end line**)
- **ends** добавяне на символа „край на **стринга**” (**'\0'**)
- **flush** изчистване на потока
- **setprecision(int)** задаване точност на числа с плаваща точка
- **setw(int)** задаване дължина на полето (брой позиции)
- **setfill(char)** задаване на символ за запълване на позиции
- **setf(ios::flagname)** установяване на флаговете за състояние
- **unsetf(ios::flagname)** нулиране на флаговете за състояние
- **fixed** извеждане с цифри след десетичната точка
- **showpoint** задължително извеждане с десетична точка
- **scientific** задължително извеждане в експоненциален вид

**Задача 3.1.** Разгледайте следната **C/C++** програма и анализирайте как се използват манипулаторите.

```
#include <iostream.h>
#include <iomanip.h>
void main()
{
    cout << 10 << 15 << endl;
    cout << 10<< setw(10) << 15 << endl;
    cout << 10 << hex << setw(10) << 15 << endl;
    cout << 10 << dec << setiosflags(ios::left) <<15 << endl;
    cout << 10 << setw(10) << hex << 15 << endl;

    cout << setfill('-') << endl; // Запълване със символ '-'
    cout << 10 << 15 << endl;
    cout << 10 << setw(10) <<15 << endl;
    cout << 10 << hex << setw(10) << 15 << endl;
    cout << 10 << dec << setiosflags(ios::left) <<15 <<endl;
    cout << 10 << setw(10) << hex << 15 <<endl;
}
```

Получават се **резултати** в следния вид:

```
1015 // Двете числа са залепени едно до друго
10      15 /* За числото 15 е оставено поле с 10 символа с
           дясно подравняване */
10      f  // Числото 15 се изписва в шестнадесетичен код (f)
a15     /* Числото 10 е изписано в шестнадесетичен код
           (a), а 15 е изписано с ляво подравняване */
10f     /* Продължава да важи лявото подравняване на
           числата и затова числото 15 (изписано в
           шестнадесетичен код - f) е долепено до 10 */
```

Във втората част на програмата се установява **манипулаторът** за запълване на позиции **setfill('-')** със символа **-** (тире). **Резултатите** се повтарят като вместо празен интервал се извежда **-**.

```
1015
10-----15
10-----f
a15
10f
```

**Задача 3.2.** Съставете **C++** програма, която **извежда** числото **4.567** по следните начини: **1)** без форматиране; **2)** с **1** знак след десетичната точка; **3)** с **2** знака след десетичната точка; **4)** с **4** и **5** знака след десетичната точка.

**Примерен вариант на решение:**

```
#include <iostream.h>
#include <iomanip.h>
void main()
{cout << 4.567 << endl;
 cout << setprecision(2) << 4.567 << endl << setprecision(3)
    << 4.567 << endl;
 cout << setprecision(5) << 4.567 << setprecision(6) << 4.567;
}
```

Получават се **резултати** в следния вид:

```
4.567
4.6
4.57
4.5674.567
```

**Манипулаторът setprecision (цяло число)** изисква като аргумент цяло число показващо **броя на цифрите след десетичната точка** и позицията на десетичната точка (за среда **Visual C++ 6.0**). По тази причина при второто извеждане със **setprecision(2)** се получава **4.6** с **една** цифра след десетичната точка (другата позиция е позицията на десетичната точка). **Ако числото съдържа повече цифри след десетичната точка, setprecision закръглява десетичната част.**

Аналогичен е резултатът при третото извеждане за **2** знака след десетичната точка със **setprecision(3)**.

При последното извеждане **setprecision(5)** и **setprecision(6)** не влияят, тъй като числото има само **3** цифри след десетичната точка.

Възможен е и друг вариант на решение при който манипулаторът **endl** не се използва, а се използва управляващият символ **\n**.

При използването на **endl** не е задължително включването на заглавния файл **iomanip.h**:

```
#include <iostream.h>
void main()
{cout << 4.567 << "\n";
 cout << setprecision(2) << 4.567 << "\n" << setprecision(3)
    << 4.567 << "\n";
}
```

```
cout << setprecision(5) << 4.567 << setprecision(6) << 4.567;
}
```

Във втория вариант на програмата манипулатора **endl** се заменя с низа **"\n"**. В този случай няма голямо значение кой от двата варианта ще се избере. Счита се **"\n"** изисква натискане на клавиша **Shift** заради кавичките и това би затруднило програмиста при въвеждане текста на програмата. Но ако извеждате **СИМВОЛЕН НИЗ**, употребата на **"\n"** е по-удачна, защото кавичките остават в стринга и се получава една обща символна последователност:

```
cout << " Това е моята първа програма на C \n";
cout << " Тя може да послужи за учебна цел \n";
```

Горният фрагмент, разбира се, може да се напише и с **endl**, без промяна в действието му:

```
cout << " Това е моята първа програма на C" << endl;
cout << " Тя може да послужи за учебна цел" << endl;
```

**Задача 3.3.** Съставете програма, която извежда на екрана стойността на променливата **price** от тип **double** в различни формати.

**Примерен вариант на решение:**

```
#include <iostream>
using namespace std;
void main()
{double price(78.5779008);
cout << endl << "Стойността на price е равна на " << price
<< endl;
cout.setf(ios::showpoint);
cout.precision(2);
cout << endl << "Стойността на price е равна на " << price
<< endl;
cout.precision(5);
cout << endl << "Стойността на price е равна на " << price
<< endl;

cout.unsetf(ios::showpoint);
cout.setf(ios::fixed);
cout.precision(5);
cout << endl << "Стойността на price е равна на " << price
<< endl;

cout.unsetf(ios::fixed);
cout.setf(ios::scientific);
cout << endl << "Експоненциален запис " << price;
cout.precision(2);
cout << endl << "Стойността на price е равна на "
<< price << endl;

cout.unsetf(ios::scientific);
cout.precision(0);
cout << endl << "Стойността на price е равна на " << price
<< endl;
//cerr << endl << "Стойността на price е равна на "
```

```

        << price << endl;
    }

```

Този вариант на решение се отличава от предходните в частта си за включване на заглавния файл **iostream.h**.

```

#include <iostream>
using namespace std;
и
#include <iostream.h>

```

са идентични.

**C++** поддържа голям брой стандартни библиотеки с **функции**, разположени в различни **именувани пространства (namespaces)**. Това позволява дефинирането на едни и същи по име функции в различни **именувани пространства**. По-голямата част от стандартните библиотеки разполагат техните функции в **именуваното пространство std**. Ето защо най-често се използва **namespace std**. Операторите **cin >>**, **cout <<**, манипулаторът **endl** са дефинирани в **std** и са част от заглавния файл **iostream.h**, дефиниран също в **std**.

При **извеждането** на данни, в горната програма се използват **флагове**. Функцията **setf(ios::флаг)** установява флаговете и управлява извеждането на цели (**int**) и реални (**float/double**) числа. Стойността на **флаг** може да бъде: **showpos** – пред всички неотрицателни числа се поставя знак **плюс**; **fixed** – за нормално извеждане на реалното число с **десетична точка**; **showpoint** – за поставяне на десетична **точка** независимо дали числото е цяло или реално; **scientific** - за извеждане на числото в **експоненциален вид**. Веднъж зададени **setf(ios::fixed)**, **setf(ios::showpoint)** и **setf(ios::scientific)** не се повтарят, независимо колко **форматни изхода** ще се изпълняват. Отменят се с функцията: **unsetf(ios::fixed)**, **unsetf(ios::showpoint)** и **unsetf(ios::scientific)**.

Използването на **функцията precision(цяло число)** зависи от установения **флаг**. Ако стойността на **ios** е **fixed (setf(ios::fixed))**, то **precision** задава броя цифри в дробната част на реалното число; ако стойността е **showpoint**, **precision()** задава броя цифри чрез които се извежда числото (в цялата и дробната част); ако е **scientific** - задава броя цифри в дробната част на експоненциалния запис. Параметърът **цяло число** може да бъде константа, променлива или израз от цял тип. Правете разлика между **setprecision(цяло число)** и **setf(ios::fixed)**, **cout.precision(цяло число)**. Манипулаторът **setprecision(цяло число)** включва и позицията на десетичната точка.

Извеждане чрез **cout.precision(0)**, възстановява **извеждането** по **подразбиране**.

В примерното решение **резултатите** се получават в следния вид:

Стойността на **price** е равна на **78.5779** /\* Извеждане по  
подразбиране \*/

Стойността на **price** е равна на **79.** /\* Числото се извежда с  
десетична точка и с общо 2 цифри \*/

Стойността на price е равна на 78.578 /\* Числото се извежда с десетична точка и с общо 5 цифри \*/

Стойността на price е равна на 78.57790 /\* Числото се извежда с фиксирана десетична точка и 5 цифри след десетичната точка \*/

Експоненциален запис 7.85779e+001 /\* Числото се извежда в експоненциален вид \*/

Стойността на price е равна на 7.86e+001 /\* Числото се извежда в експоненциален вид с 2 цифри след десетичната точка \*/

Стойността на price е равна на 78.5779 /\* Извеждане по подразбиране \*/

Ако към горното решение добавите един програмен ред с използване на оператора за изход към cerr:

```
cerr << endl << "Стойността на price е равна на" << price << endl;
```

ще се изведе : Стойността на price е равна на 78.5

Операторът cerr << функционира по същия начин както cout << . Но ако се появи грешка по време на извеждане чрез cerr, извеждането на данните ще е съпроводено със **съобщение за грешка**.

**Задача 3.4.** Съставете C++ програма, която извежда имената на футболни отбори и техните точки за 3 седмици, използвайки **манипулатор за ширина** (брой позиции в полето за данни).

**Примерен вариант на решение:**

```
#include <iostream.h>
```

```
#include <iomanip.h>
```

```
void main()
```

```
{cout << setw(10) << "Левски" << setw(10) << "ЦСКА" << setw(10) << "Славия" << setw(10) << "Спартак" << setw(10) << "Ботев" << endl;
```

```
cout << setw(10) << 2 << setw(10) << 5 << setw(10) << 3 << setw(10) << 0 << setw(10) << 4 << endl;
```

```
cout << setw(10) << 6 << setw(10) << 3 << setw(10) << 1 << setw(10) << 0 << setw(10) << 2 << endl;
```

```
cout << setw(10) << 1 << setw(10) << 1 << setw(10) << 2 << setw(10) << 2 << setw(10) << 1 << endl;
```

```
}
```

Получават се **резултати** в следния вид:

| Левски | ЦСКА | Славия | Спартак | Ботев |
|--------|------|--------|---------|-------|
| 2      | 5    | 3      | 0       | 4     |
| 6      | 3    | 1      | 0       | 2     |
| 1      | 1    | 2      | 2       | 1     |

Получените резултати са във вид на **таблица**, чрез използване на **манипулатор setw(цяло число)**. Цялото число определя ширината на

полето, в което ще се разположи изходната данна. Когато зададете голяма ширина и извеждате данните в нея, те са **подравнени в дясно**. Манипулаторът **setw()** е единственият, който **не запазва действието** си от една операция към следваща. Ето защо, ако искате да подравните вдясно **три** числа в полета с ширина **10 позиции**, трябва да използвате **setw()** **три пъти**.

Възможно е в решението на тази задача **setw(10)** да бъде заменен със **cout.width(10)**, например:

```
cout.width(10);  
cout<<1;
```

Програмата ще се получи с повече редове, защото изисква използването на повече **cout**.

**Задача 3.5.** Какви стойности ще бъдат въведени за променливите **a**, **b** и **c** в следната **C/C++** програма:

```
#include <iostream.h>  
void main()  
{int a, b, c;  
    cout << "\n Въведете 3 стойности за променливите a,b,c";  
    cin >> a >> b >> c;  
    cout<<"\n Получените стойности са: " <<a<< " " <<b<< " " <<c;  
}
```

ако техните стойности се въведат от клавиатурата по следния начин:

**5,6,7 ?**

**C/C++** програмата не счита **запетаята** за празен символ. В този случай само променливата **a** ще получи стойност **5**, а всички останали **променливи ще имат произволни стойности**.

Потребителят може да въведе **трите** стойности, разделени с **интервали** или да натиска клавиша **Tab** при въвеждането им.

Входът от клавиатурата **е буфериран**. Всяка последователност от натиснати клавиши се разглежда като пакет, който се обработва след като се натисне клавишът **Enter**. Няма значение дали в примера **трите** числа ще се въведат като:

**5 6 7** или като: **5**

**6**

**7** (всяко на отделен ред).

Променете типа на променливите **a**, **b** и **c**. Примерно:

```
int a, float b,int c;
```

Ако входната последователност от клавиатурата е **10.5 23.1 15**, какви стойности ще получат променливите **a**, **b** и **c**.

За променливата **a** ще се приеме **стойност 10**, тъй като тя е от цял тип и десетичната точка в последователността се явява **разделител**. Променливата **b** е от **тип float** и приема стойност с десетична точка – **0.5**. Променливата **c** е от **цял тип** и получава следващата цяла стойност от буферната последователност: **23**.

## Функции за вход/изход на данни

### 1. Функции за вход и изход: `getline()`, `get()`, `gets()` и `put()`, `puts()`.

Често в програмите се налага въвеждането на **низ от символи** и това е възможно да с извърши както със `cin >>`, така и с функцията `getline()`. Докато `cin >>` спира да въвежда при достигане на **първия празен интервал**, то `getline()` спира да въвежда едва при натискане на клавиша **Enter**. Всичко въведено от потребителя до този момент, се съхранява в **масива от символи (стринга)**. Най-често формата на функцията изглежда така:

*`cin.getline(име_на_стринг, максимален брой въвеждани символи)`.*

Функцията `getline()` винаги спира да въвежда, щом бъде достигнат посоченият **максимален** брой символи в низа и осигурява място за завършващия **нулев символ '\0'**.

Ако потребителят въведе точно толкова символа, колкото е максимално допустимия брой, **C/C++** програмата заменя последния символ с **нулевия (терминиращия) символ**, осигурявайки по този начин превръщането на въведения ред от символи в низ.

**Задача 3.6.** Какво ще изведе следващата програма, ако потребителят е въвел последователността **Варна10**, в отговор на запитването за град?

```
#include <iostream.h>
void main()
{ char city[6];
  cin.getline(city,6);
  cout << endl<< city << endl;
}
```

При въвеждане на стринга **Варна10** чрез `cin.getline(city,6)`, променливата `city` получава стойност **Варна**.

Ако промените `char city[6]` със `char city[12]` и въведете **Черноморска столица** (по същия начин), променливата `city` получава стойност **Черно** (т.е **5** символа за стринга и **1** за `\0` – край на низа).

Променете `char city[6]` със `char city[25]` и `cin.getline(city,25)`. Въведете от клавиатурата **Черноморска столица**. Променливата `city` получава стойност **Черноморска столица**, състояща се от **19** символа и един за край на низа.

**Задача 3.7.** Какви стойности ще получат променливите `c1`, `c2`, `c3` при изпълнение на следната програма:

```
#include <iostream.h>
void main()
{ char c1,c2,c3;
  cin.get(c1);
  cin.get(c2);
  cin.get(c3);
  cout << endl<< "Стойността на променливата c1 е "<< c1;
  cout << endl<< "Стойността на променливата c2 е "<< c2;
```

```
    cout << endl<< "Стойността на променливата c3 е "<< c3;
}
```

при въвеждане на следната последователност:

**AB**

**CD**

Получават се **резултати** в следния вид:

**Стойността на променливата c1 е A**

**Стойността на променливата c2 е B**

**Стойността на променливата c3 е**

За **c3** се появява празен интервал, който в случая е еквивалентен на **'\n'**. Променливата **c3** не получава стойност **'C'**.

Функцията **get()** е за **буферирано** въвеждане. Когато въвеждате символи, те не се предават незабавно към вашата програма, а се записват в буфер. **Буферът се пълни със символи**, докато се натисне **Enter**. Затова ако зададете въпрос:

**“Желаете ли да видите резултата отново (Y/N)?”** и използвате функцията **get()** за въвеждане, програмата не приема натискането на **Y**, докато не се натисне **Enter**.

Тази е причината променливата **c3** да получи стойност **'\n'**.

С използването на буфериран вход потребителят може да въведе низ от символи за обработка с **get()** в цикъл и да поправя входните данни с натискане на клавиша **Backspace** преди **Enter**.

**Задача 3.8.** Съставете **C/C++** програма, която дава възможност за въвеждане на **ред от символи** и повтарянето им като „ехо” докато се натисне **Enter**.

**Примерен вариант на решение:**

```
#include <iostream.h>
void main()
{char symbol;
  cout <<endl <<"Въведете 1 ред символи и ще ги повтора"<< endl;
  do
  { cin.get(symbol);
    cout << symbol;
  } while (symbol!='\n');
  cout << endl << "Това е демонстрацията!" << endl;
}
```

Получават се **примерни резултати** в следния вид:

**Въведете 1 ред символи и ще ги повтора**

Обучение по C

**Обучение по C**

Получава се „ехо” на буферираните символи, включително и празните интервали. Правете разлика между символа **'\n'** и стринга **"\n"**. Действието е едно и също, но типовете на данните са различни: **char** и **символен низ** (константа). Във варианта на програмата е използван оператор за цикъл **do...while** (вижте **Тема 5**).



Поставете програмен ред `cout.put(symbol)` вътре в цикъла. Функцията `put()` се използва за извеждане на символи. В този случай всяка буква се повтаря докато спре въвеждането.

**Задача 3.9.** Какво ще изведе следната програма при вход един ред във вида: 0 1 2 3 4 5 6 7 8 9 10 11? Заменете в редове **6** и **11** на програмата оператора `cin.get(next)` с оператор `cin >> next`. Как се променя **изхода** на програмата при тази замяна?

```
#include <iostream.h> // 1
void main()           // 2
{char next;           // 3
 int count=0;         // 4
 cout<<"Въведете на един ред:0 1 2 3 4 5 6 7 8 9 10 11"<<endl; // 5
 cin.get(next);       // 6
 while (next != '\n') // 7
 { if((count%2)==0)   // 8
   cout << next;      // 9
   count++;           // 10
   cin.get(next)      // 11;
 }                   // 12
}                   // 13
```

Получават се **резултати** в следния вид:

01234567891 1

Програмата извежда само **четни по ред символи** (`if ((count%2)==0)`) и така **празните** интервали между числата се **изключват**. Двете последни числа **10** и **11** се състоят от **2** цифри и затова **0** на **10** се игнорира (явява се **четен** символ), извежда се празния интервал между тях и **втората** единица на числото **11** (тя също е **четен** символ). Използва се оператор за цикъл **while** (вижте **Тема 5**).

При замяна на операторите в редове **6** и **11** (с оператор `cin >> next`) **резултатите** се получават в следния вид:

0246811

Цикълът **while** (изпълнява се **докато** израза в скобите е различен от **0** – логическа стойност “истина”) се получава **безкраен**. Логическият израз (`next != '\n'`) е винаги “истина” защото `cin >> next` прескача символа за нов ред (третира го като празен интервал). Извежда се всеки **четен по ред** символ (с прескачане на празните интервали). Последните две цифри **11** са първите цифри от числата **10** и **11**.

Ако замените оператора `cout << next` в ред **9** със `cout.put(next)` разлика в изведената последователност не се наблюдава.

**Задача 3.10.** Съставете програма за въвеждане на числа, с запитване за коректност на числото (`yes/no`) като използвате **2 потребителски дефинирани** функции: `newLine()` за игнориране на празни интервали и край на реда и `getNumber(int &number)` – за въвеждане на конкретното число с въпрос за неговата коректност.

**Примерен вариант на решение:**

```
#include <iostream>
using namespace std;
```

```

void newLine();
// Функция за премахване на символа за нов ред - деклариране
void getInt(int &number);
// Функция за въвеждане на цяло число и запитване за коректност

void main()
{ int n;
  getInt(n);
  cout << "Крайната стойност е равна на " << n << endl <<
    "Край на програмата";
}

void newLine()
{ char symbol;
  do
  { cin.get(symbol);
    } while (symbol != '\n');
}

void getInt(int &number)
{ char ans;
  do
  { cout << "Въведи число: ";
    cin >> number;
    cout << "Вие въведохте " << number << " Това коректно ли
      е? (yes/no): ";
    cin >> ans;
    newLine();
  } while ((ans == 'N') || (ans == 'n'));
}

```

### Примерни резултати:

```

Въведи число: 57
Вие въведохте 57 Това коректно ли е? (yes/no): No No No
Въведи число: 75
Вие въведохте 75 Това коректно ли е? (yes/no): yes
Крайната стойност е равна на 75
Край на програмата

```

Възможно е **функцията newLine()** да се модифицира и в нейното тяло да се постави **стандартната функция ignore()**.

```

void newLine()
{
  cin.ignore(10000, '\n');
}

```

Тази функция ще работи за редове с до **10000** символа (числото може да се промени) и ще игнорира символа за нов ред.

Функцията **gets()** се използва за въвеждане на **стринг** докато се стигне **край на реда**. Символът край на реда **'\n'** не се включва в стринга.

Функцията **put()** се използва за извеждане на **символ**. Най-често се използва с обекта **cin** във вида **cin.put(char)**.

## 2. Използване на функциите `getch()`, `putch()`, `getche()`, `putche()`.

Функцията `getch()` се различава от останалите функции за въвеждане на символи по това, че при получаване на входен символ, не го извежда на екрана (няма „ехо“). При останалите функции, по време на изпращането на символи към буфера, вие ги виждате на екрана. Ако искате да ги виждате на екрана веднага след `getch()` трябва да използвате функцията `putch()`:

```
symbol = getch();  
putch(symbol);
```

Възможно е горните два реда (въвеждане с „ехо“) да се заменят с използване на функцията `getche()`, която въвежда символ с „ехо“:

```
symbol = getche();
```

Използването на `putch()` отпада.

Някои програмисти предпочитат да накарат потребителя да натисне **Enter** след отговор на запитване или избор от меню. Считат, че допълнителното време предоставено за буфериран вход дава на потребителя време за обмисляне на отговора и възможност за корекция с клавиша **Backspace**. Други предпочитат да получат отговора на потребителя с един единствен символ и да го обработят незабавно.

При използване на `getch()`, `putch()`, `getche()` и `putche()` е необходимо включването на заглавния файл `conio.h`.

**Задача 3.11.** Съставете C/C++ програма за: въвеждане на 10 символа без „ехо“, записването им в масива `bukwi`, извеждане на масива на екрана и извеждане на символите на принтер (при извеждане на данни на принтер се включва заглавния файл `fstream.h`). За поелементна обработка на масива използвайте оператор за цикъл `for` (вижте **Тема 5** и **Тема 6**).

### Примерен вариант на решение:

```
#include <iostream.h>  
#include <fstream.h>  
#include <conio.h>  
void main()  
{int counter;          // Брояч на символите в масива – от 0 до 10  
  char bukwi[10];      // Съхранява 10 букви, за низ – bukwi[11]  
  cout << "Моля въведете 10 букви:" << endl;  
  for(counter = 0; counter < 10; counter++)  
    bukwi[counter] = getch();    // Въвежда символ в масива  
  for (counter = 0; counter < 10; counter++)  
    putch(bukwi[counter]);      // Извежда символите на екрана  
  ofstream prn("PRN");          // Принтерът – като изходен файл  
  for (counter = 0; counter < 10; counter++)  
    prn.put(bukwi[counter]);    // Извежда символите на принтера  
  cout << endl;  
}
```

При изпълнение на тази програма, **не трябва** да въвеждате **Enter** след буквите. Цикълът завърша автоматично след въвеждането на десетата буква поради **небуферирания** вход и цикъла **for**.

Всяко **външно устройство** се разглежда от **C/C++** програмата като **изходен файл**. Използва се обекта **ofstream**, който се представя по-подробно в **Тема 11**.

Ако въвеждате **десетте** букви като **символен низ**, можете да използвате **функцията get()** и въвеждането да се осъществи посредством:

```
cin.get(bukwi,11)
```

Декларирайте масива **bukwi[11]** като символен низ. Не забравяйте че **get()** е **буферирана функция** и изисква натискането на **Enter**.

Примерен вариант на променената част на програмата е:

```
int counter;  
char bukwi[11];  
cout << "Моля въведете 10 букви: " << endl;  
cin.get(bukwi,11);
```

**Резултатите** се получават в следния вид:

Моля въведете 10 букви:

abcdefghijkl <Enter>

abcdefghijkl

### 3. Въвеждане и извеждане чрез функции **getchar()** и **putchar()**.

Тези две **функции** са дефинирани в заглавния файл **stdio.h**. Функцията **getchar()** връща следващия символ, въведен от клавиатурата и използва **линейно** буфериране (изисква натискане на **Enter**). Функцията **putchar()** извежда един символ. Ако се появи грешка при извеждане, се връща **EOF** (End Of File - край на файла).

**Задача 3.12.** Съставете **C/C++** програма, която за всеки въведен **символ**, извежда **точка**.

**Примерен вариант на решение:**

```
#include <iostream.h>  
#include <stdio.h>  
void main()  
{ char symbol;  
  printf("\n Въведете последователност от символи:");  
  do  
  { symbol = getchar();  
    putchar('.');  
  } while (symbol != '\n');  
}
```

**Примерен резултат:**

При вход: **abcde**

Изход:.....

Точките са **6**, защото **'\n'** е също символ (за край на реда).

## Въпроси и задачи за самостоятелна работа

1. Какво е грешно в следния програмен фрагмент?

```
char symbol[80];  
.....  
putchar(symbol)
```

2. Каква е разликата между функциите `getchar()`, `getche()` и `getch()`?  
3. Каква е разликата между функциите `getch()` и `get()`?  
4. Как можете да въведете от клавиатурата низ, съдържащ интервали?  
5. Възможно ли е функцията `puts()` да извежда символ? Може ли да се замени с `printf()` и какво е предимството на `puts()`?  
6. Кой изходен манипулатор ограничава броя на десетичните цифри при извеждане на реални числа?  
7. Кой заглавен файл трябва да включите, когато използвате входно/изходни манипулатори?  
8. Какво е предимството на `cin.getline()` пред `cin >>` при въвеждане на низове?  
9. Разгледайте следния фрагмент от C/C++ програма:

```
string s;  
cout << "Въведете низ от символи: " << endl <;  
getline(cin,s);  
cout << s << "Край на програмата";
```

Работеща ли е програма, използваща такъв фрагмент? Какъв е резултатът на изхода?

10. Напишете програма, извеждаща ASCII кодовете на всеки въведен символ (като десетично и шестнадесетично число).  
11. Съставете програма, представляваща “машинописна машинка”, с използването на функциите `get()` и `put()`.  
12. Напишете програма, която пита потребителя за въвеждане на 10 букви и ги извежда в обратен ред.  
13. Съставете програма за проверка дали една дума е палиндром (дума изписваща се еднакво в прав и обратен ред).  
14. Напишете програма, която променя флаговете на `cout.setf`, така че числата с положителни стойности да се извеждат със знак +.  
15. Съставете C/C++ програма, която използва форматиращи функции и извежда следните последователности:  
Hello  
%%%%%%%%Hello  
Hello%%%%%%%%  
123.234567  
123.235%%  
16. Съставете C/C++ програма, която извежда натуралния и десетичен логаритъм на числата от 2 до 100. Форматът на таблицата трябва да е такъв, че числата да са дясно подравнени в полета с ширина 10 позици и използваната точност да е 5 десетични цифри.

## Тема 4. Програмиране на задачи с разклонение (избор). Логически операции и операции за сравнение. Оператори if, switch, goto. Структуриране на програми с избор и използване на меню.

**Цел:** Съставяне, тестване, изпълнение и анализ на разклонени C/C++ програми, включващи оператори за управление if, switch и goto.

### Логически изрази

Логическите изрази се конструират от изрази за отношение/аритметични изрази, свързани помежду си чрез логически операции:

- ! - (логическо отрицание - **not**),
- && - (логическо умножение - **and**);
- || - (логическо събиране - **or**).

Резултатът от **логически израз** е истина (**true**) или лъжа (**false**). Трябва да се отбележи, че резултат равен на **0** в C/C++ се счита за **false**, а различен от **0** – за **true**.

**Израз за отношение** се конструира от **аритметични изрази**, свързани чрез **операции за сравнение**, а резултатът е истина (**true**) или лъжа (**false**). Използваните в C/C++ **операции за сравнение** са:

- < - по-малко от; <= - по-малко или равно на;
- > - по-голямо от; >= - по-голямо или равно на;
- == - равно на; != - различно от.

**Забележка:** Оператор за **присвояване** = не бива да бъде бъркан с операция за сравнение ==.

**Примери:** Нека **a**, **b** и **c** са три променливи със следните стойности:

**a** = 5; **b** = 15; **c** = 80;

Всеки от изразите по-долу е с резултат **true**:

**a** < **b**, **c** > **b**, **b** < **c**, **b** <= **c**, **a** != **c**, **c** >= **b**,  
**a** + **b** < 12 && **b/a** == 3 || **c** == 80.

Всеки от изразите по-долу е с резултат **false**:

**a** < **b**, **c** > **b**, **b** == **c**, **b** >= **c**, **a** != **a**, **c** <= **b**,  
**a** + **b** < 20 && **b/a** == 3 || **c** != 80.

Поради различния приоритет на операциите, последният израз е еквивалентен на:

((**a** + **b**) < 20) && ((**a/b**) == 3) || (**c** != 80).

**Приоритетът** на използваните операции в **логическите изрази** е както следва (от най-висок, към най-нисък):

- ! (с най-висок приоритет)
- <, <=, >, >= (с еднакъв приоритет)
- ==, !=
- &&
- || (с най-нисък приоритет)

## Оператори за управление if (if...else), switch, goto

### 1. Оператор if.

Общият вид на оператор **if** (известен като “оператор за вземане на решение” или “оператор за разклонение”) е:

```
if (условие)
```

```
    оператор
```

където:

- **условие** е всеки валиден **логически/аритметичен** израз или променлива от тип **bool** (валиден в **C++**).
- **оператор** може да бъде единствен **C/C++** оператор от всеки вид, завършващ със символа **;** или **съставен** оператор. Изпълнява се, ако резултатът от **условие** е **true** (различен от нула).

**Забележка:** **Съставен** оператор (наричан и **блок**) е един или повече оператора, заградени с фигурни скоби **{ }**.

**Пример 1.** Представената по-долу програма изисква от потребителя да въведе две **цели** числа **a**, **b**. Ако **a > b**, програмата разменя стойностите на променливите.

```
#include <iostream.h>
```

```
void main()
```

```
{int a,b,temp;          // Променлива temp – за размяна
```

```
    cout<<"\n Програмата въвежда стойности за a и b";
```

```
    cout<<"\n Ако a>b – следва размяна на стойностите им.";
```

```
    cout<<"\n Въведете стойност за a=: ";
```

```
    cin>>a;
```

```
    cout<<"\n Въведете стойност за b=: ";
```

```
    cin>>b;
```

```
    if(a > b)              // Размяна на стойностите
```

```
    {cout<<"\n Размяна на a и b!!!"; // Съставен оператор
```

```
        temp = a;
```

```
        a = b;
```

```
        b = temp;
```

```
    }                      // Край на размяната
```

```
    cout<<"\n a е "<<a<<"\n";
```

```
    cout<<"\n b е "<<b<<"\n";
```

```
}                          // Край на main()
```

**Примерно изпълнение:**

Програмата въвежда стойности за **a** и **b**

Ако **a>b** – следва размяна на стойностите им.

Въведете стойност за **a=: 6**

Въведете стойност за **b=: 2**

Размяна на **a** и **b!!!**;

**a** е 2

**b** е 6

## 2. Оператор *if...else*.

Операторът *if* може да се използва в комбинация с оператор *else* (*else* никога не може да се използва самостоятелно):

```
if (условие)
    оператор1
else
    оператор2
```

където:

- **условие** е всеки валиден **логически/аритметичен** израз или променлива от тип **bool**.
- Ако **условие** е **true** - изпълнява се **оператор1**, в противен случай (ако условие е **false**) - изпълнява се **оператор2**.
- **оператор1** и **оператор2** могат да бъдат единствен **C/C++** оператор от всеки вид, завършващ с ; или **съставен** оператор.

**Пример 2.** Представената по-долу примерна програма реализира избор от множество алтернативи чрез използване на *if...else* оператор.

```
/* Потребителят въвежда номер на месец: от 1 до 12.
Извежда се съответния годишен сезон, използвайки
оператор if...else.*/
#include <iostream.h>
void main()
{int month;
cout<<"\n Въведете номер на месец: ";
cin>>month;
if(month==1 || month==2 || month==12)
    cout<<" Зима!!!\n";
else if (month==3 || month==4 || month==5)
    // Или (month>=3 && month <=5)
    cout<<" Пролет!!!\n";
else if(month==6 || month==7 || month==8)
    cout<<" Лято!!!\n";
else if(month==9 || month==10 || month==11)
    cout<<" Есен!!!\n";
else cout<<month<<" не е валиден месец!\n";
} // Край на main()
```

**Примерно изпълнение:**

Въведете номер на месец: 3  
Пролет!!!

Както виждате в тази примерна програма, в условието на оператор *if* се използва логическата операция **||** (логическо събиране - **or**).  
**Резултатът** е **Пролет**, ако месецът е с номер 3 или 4 или 5.

**Пример 3.** Студент получава оценка в точки на всеки изпит. Нека тези точки съответстват на следните оценки:

≥ 90 → 'A'



80÷89 → 'B'  
70÷79 → 'C'  
65÷69 → 'D'  
<65 → 'F'

Да се състави програма, която изисква от студента да въведе своите точки. Програмата да извежда съответния еквивалент – буква.

```
/* Потребителят въвежда оценка (grade): от 0 до 100
точки. Програмата прави проверка за коректност на
въведените точки и извежда стойността на една символна
променлива symbol, съхраняваща съответната символна
оценка: F, D, C, B, A */
#include <iostream.h>
void main()
{int grade; char symbol;
  cout<<"\n Въведете оценка в точки: ";
  cin>>grade;
  if(grade < 0 || grade > 100)
    cout<< grade <<" не е валидна оценка.\n";
  else
  {if(grade < 65) symbol = 'F';
    else if (grade < 70) symbol = 'D';
    else if (grade < 80) symbol = 'C';
    else if (grade < 90) symbol = 'B';
    else symbol = 'A';
    cout<<grade<<" е "<< symbol <<"\n";
  }
} // Край на main()
```

#### Примерно изпълнение:

Въведете оценка в точки: 60  
66 е D

### 3. Оператор *switch*

Оператор **switch** е алтернатива на оператор **if...else** при реализация на **множествен избор**.

Общият вид на **switch** оператор е:

```
switch (израз)
{case константа_1: оператор_1; break;
 case константа_2: оператор_2; break;
 .....
 case константа_N: оператор_N; break;
 default: оператор_def; // default е незадължителен
      break; // Ако default не е последен в switch
}
```

Където **израз** е от **цял** или от **символен** тип. Стойността на **израз** се сравнява с всяка от константите (**константа\_1**, **константа\_2**,... **константа\_N**), които трябва да бъдат с **различни** стойности, но от същия тип като **израз**. Ако има съвпадение между стойността на **израз** и някоя от **константите**, следва изпълнение на съответния **оператор**, до достигане на оператор **break** (след което се “излиза” от обхвата на **switch** и се изпълнява първия следващ по ред оператор). Ако не се намери такова съвпадение – изпълнява се **оператор\_def**, асоцииран с **default** алтернативата.

**Пример 4.** Тази реализация е алтернатива на **Пример 2**, с използване на оператор **switch**.

```
/* Потребителят въвежда номер на месец: от 1 до 12.
Извежда се съответния годишен сезон използвайки
оператор за мулти-разклонение switch.*/
#include <iostream.h>
void main()
{int month;
 cout<<"\n Въведете номер на месец: ";
 cin>>month;
 switch (month)
 {case 1: case 2: case 12: cout<<"Зима!!!\n"; break;
  case 3: case 4: case 5: cout<<"Пролет!!!\n"; break;
  case 6: case 7: case 8: cout<<"Лято!!!\n"; break;
  case 9: case 10: case 11: cout<<"Есен!!!\n"; break;
  default: cout<<month<<" не е валиден месец!!!\n";
 } // Край на switch
} // Край на main()
```

**Примерно изпълнение:**

Въведете номер на месец: 8  
Лято!!!

#### 4. Оператор **goto**

При изпълнение на оператор **goto** се извършва **преход** (във **функция**) към оператор с конкретен **етикет**, вместо да се изпълни следващия по ред оператор.

**Общият** вид на този оператор е:

**goto етикет\_на\_оператор;**

Ще демонстрираме **goto** оператор със следната програма, намираща корените на квадратно уравнение:

**Пример 5.**

```
/* Пример за използване на goto оператор: намиране на
корените на квадратно уравнение */
#include <iostream.h>
#include <math.h>
```

```

void main()
{int a,b,c;
 cout<<"\n Въведете коефициентите на квадр. уравнение";
 cout<<"\n Въведете стойност за a: ";
 LOOP: cin>>a;
 if(a==0)
 {cout<<"\n Некоректна стойност за a!";
  cout<<"\n Въведете стойност за a отново!!!";
  goto LOOP; // Цикъл, осигуряващ правилна стойност на a
 }
 cout<<"\n Въведете стойност за b: "; cin>>b;
 cout<<"\n Въведете стойност за c: "; cin>>c;
 double d,x1,x2;
 d=pow(b,2)-4*a*c;
 if (d<0)
 cout<<"\n Уравнението няма реални корени!";
 else if (d==0)
 {x1=x2=-b/(2*a); cout<<"\n x1="<<x1<<"\t x2="<<x2;}
 else
 {x1=(-b+sqrt(d))/(2*a); x2=(-b-sqrt(d))/(2*a);
  cout<<"\n x1="<<x1<<"\t x2="<<x2;
 }
 cout<<endl;
 }
}

```

#### Примерно изпълнение:

```

Въведете коефициентите на квадр. уравнение
Въведете стойност за a: 2
Въведете стойност за b: 4
Въведете стойност за c: 2
x1=-1    x2=-1

```

В този пример **LOOP** е етикет пред оператор *if*. Когато компилаторът срещне конструкцията **goto LOOP**, той ще потърси етикет с име **LOOP** и ще продължи изпълнението на програмата от оператора след този етикет.

Препоръчително е оператор **goto** да бъде избягван винаги, когато това е възможно, тъй като неговата употреба води до получаване на не добре структуриран програмен код [4].

### Примерни задачи и програми с разклонение и меню техника

**Задача 4.1.** Представената по-долу програма реализира известната задача “Мишка в лабиринт”. Лабиринтът има 2 врати ‘X’ и ‘Y’, чието състояние (1 или 0) се контролира от потребителя (1 – отворена врата, 0 – затворена врата). Мишката може да излезе от лабиринта, ако има поне една отворена врата. Целта на програмата е да анализира

състоянието на вратите и да изведе съобщение, дали мишката може да излезе от лабиринта или не [9 -12].

```
/* Лабораторно упражнение 4. Задача 1. */
// Определя се дали мишката може да избяга от лабиринта
#include <iostream.h>

void main()
{char answer;
 int X,Y;
 cout<<"\n Отворена ли е врата X? Да (Y|y) или не (N|n):";
 cin>>answer;
 if(answer == 'Y' || answer == 'y')
  X = 1; // X е отворена
 else
  X = 0; // X е затворена
 cout<<" Отворена ли е врата Y? Да (Y|y) или не (N|n):";
 cin>>answer;
 if (answer == 'Y' || answer == 'y')
  Y = 1; // Y е отворена
 else
  Y = 0; // Y е затворена
 if (X || Y) // Ако поне една врата е отворена
  cout<<" Мишката може да избяга.\n";
 else
  cout<<" Мишката не може да избяга.\n";
}
```

**Задача 4.2.** Две фирми за превоз на товари Saturn и Jupiter са в непрекъсната “битка” за нови клиенти. Техните най-нови условия са представени в таблицата по-долу. Стратегически клиент на двете фирми поръчва разработката на програма, която да му даде отговор за това, коя фирма да избере за пренос на товарите си, след указване на теглото (weight) на дадена пратка. Програмата трябва да изчисли цената за превоз на пратката за двете фирми (saturn\_cost, jupiter\_cost) и да предложи на клиента подходящ превозвач. Програмата трябва да е лека за модифициране, съобразно честите промени на тарифите на всяка една от фирмите [9 – 12].

| Saturn                                                                                                                                                                                                    | Jupiter                                                                                                                                                                                                            |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Условия:<br>10 лв. – за всяка пратка до 30 килограма.<br>20 лв. – за всяка пратка от 31 до 70 килограма.<br>Над 70 килограма: по 0.50 за килограм.<br>Пратки над 70 килограма не се превозват през нощта. | Условия:<br>Цена на превоз:<br>Такса 5 лв., плюс 10 стотинки за всеки килограм.<br>Фирмата превозва пратки и през нощта, независимо от техния размер. Но през нощта таксата е 7 лв., плюс 20 стотинки на килограм. |

```

/* Лабораторно упражнение 4. Задача 2. */
/* Въвеждат се теглото на пратката weight и се указва дали
превозът ще се извърши през нощта – answer (Y или N). Извеждат се
цените за превоз на всяка една фирма (saturn_cost, jupiter_cost).*/
#include <iostream.h>
#include <iomanip.h>

void main()
{int weight; // Тегло на пратката
double saturn_cost, jupiter_cost; // Цени за превоз
char answer; // Дали превозът ще се извърши през нощта
cout<<"\n Ще се превозва ли през нощта? Y или N: ";
cin>>answer;
cout<<" Въведете тегло на пратката в kg: ";
cin>>weight;
// Изчисляване, за answer == 'N' || answer == 'n'
jupiter_cost = 5.00 + 0.10 * weight;
if(weight <= 30) saturn_cost = 10.00;
else if(weight <= 70) saturn_cost = 20.00;
else saturn_cost = 0.50 * weight;
// Поправка, за answer == 'Y' || answer == 'y'
if(answer == 'Y' || answer == 'y')
{if(weight > 70) saturn_cost = 0.00;
jupiter_cost = 7.00 + 0.20 * weight;
}
cout<<setprecision(2);
if (!saturn_cost && jupiter_cost) // Ако saturn_cost е 0
cout<<" Изберете Jupiter за "<<jupiter_cost<<" лв.";
else if (saturn_cost == jupiter_cost) // Ако цените са еднакви
cout<<" Изберете който и да е – за "<<jupiter_cost<<" лв.";
else if (saturn_cost < jupiter_cost) // Ако Saturn е по-изгоден
cout<<" Изберете Saturn за "<<saturn_cost <<" лв.";
else // Ако Jupiter е по-ниска цена
cout<<" Изберете Jupiter за "<< jupiter_cost<<" лв.";
}

```

### Примерно изпълнение:

1. Ще се превозва ли през нощта? Y или N: n  
Въведете тегло на пратката в kg: 50  
Изберете Jupiter за 10 лв.
2. Ще се превозва ли през нощта? Y или N: y  
Въведете тегло на пратката в kg: 29  
Изберете Saturn за 10 лв.

Модифицирайте **Програма 4.2**, като добавите нов превозвач на товари Venus, който предоставя следните условия за превоз:

Експресна доставка:

- 20 лв. – за всяка пратка до 50 кг.
- При пратки над 50 кг.: по 0.50 лв. за всеки килограм над 50-тия.

Обикновена доставка:

- 10 лв. – за всяка пратка до 50 кг.
- При пратки над 50 кг.: по 0.25 лв. за всеки килограм над 50-тия.

**Задача 4.3.** Една фирма плаща на работниците си по различни схеми. Всички работници получават годишна заплата (salary). Съставете програма за изчисляване на базовата заплата на всеки работник, по въведена годишна заплата и схема на плащане. Въведете 0, ако работниците получават заплата всяка седмица (52 пъти). Въведете 1, ако на работниците се плаща 2 пъти месечно (24 пъти). Въведете 2, ако работниците получават заплата веднъж в месеца (12 пъти).

Забележка: За да се изчисли базовата заплата, потребителят въвежда годишна заплата (salary) и схемата на плащане (payType): 0, 1, или 2 (всичко останало се счита за невалидно).

```
/* Лабораторно упражнение 4. Задача 3. */
/*Програмата изчислява базовата заплата на работник, по въведена
годишна заплата и схема на плащане */
#include <iostream.h>

void main()
{int payType; /*0 - седмично; 1 - два пъти в месеца; 2 - един път
                в месеца */
double salary; // Годишна заплата
cout<<"\n Въведете начин на плащане (0, 1 или 2): ";
cin>>payType;
if(payType <0 || payType >2)
    cout<<" Начинът на плащане може да е само 0, 1, или 2\n";
else
{cout<<" Въведете годишна заплата в лева: ";
cin>>salary;
switch (payType)
{case 0: // Седмично плащане
    cout<<" Базова заплата = "<<salary/52.0<<" лв.\n"; break;
case 1: // Плащане два пъти в месеца
    cout<<" Базова заплата = "<<salary/24.0<<" лв.\n"; break;
case 2: // Плащане един път в месеца
    cout<<" Базова заплата = "<<salary/12.0<<" лв.\n"; break;
} // Край на switch
} // Край на else
} // Край на main
```

#### Примерно изпълнение:

Въведете начин на плащане (0, 1 или 2): 0  
Въведете годишна заплата в лева: 30000  
Базова заплата = 576.923 лв.

Въведете начин на плащане (0, 1 или 2): 1  
Въведете годишна заплата в лева: 30000  
Базова заплата = 1250 лв.

Въведете начин на плащане (0, 1 или 2): 2  
Въведете годишна заплата в лева: 30000  
Базова заплата = 2500 лв.

Преработете програмата, използвайки меню техника за избор на изчисление, например:

```
cout << "\n-----";
cout << "\n|                Начин на плащане:                |";
cout << "\n|                0 - седмично                            |";
cout << "\n|                1 - два пъти в месеца                    |";
cout << "\n|                2 - един път в месеца                    |";
cout << "\n-----";
cout << "\n \n Изберете - 1, 2 или 3: ";
cin>>payType;
```

**Задача 4.4.** Да се състави C/C++ програма за елементарен калкулатор: потребителят въвежда израз, например 12\*5 или 28/6, след което на екрана се извежда получения резултат. Обърнете внимание на евентуални грешки, свързани с деление на 0.

```
/* Лабораторно упражнение 4. Задача 4. */
/* Елементарен калкулатор (операции +,-,*,/,%). Въвежда се израз.
Изчислява се и се извежда резултата */
#include <iostream.h>
```

```
void main()
{int x, y;           // Операнди
 char op;           // Операция
 cout<<"\n Въведете израз, (например 12+3): ";
 cin>>x>>op>>y;    // Въвеждат се операндите и операцията
 switch (op)
 {case '+': cout<<x+y; break;
 case '-': cout<<x-y; break;
 case '*': cout<<x*y; break;
 case '/':      // Възможно е деление на 0!!!
   if(y==0) cout<<" Делението на 0 не е позволено.\n";
   else cout<<(double)x/y; // Превръщане в double
   break;
 case '%': cout<<x%y; break;
 default: cout<<op<<" не е валиден оператор.\n";
 }
           // Край на switch
}
           // Край на main
```

**Примерно изпълнение:**

```
Въведете израз, (например 12+3): 123*3
369
```

**Задача 4.5.** Следващата програма демонстрира използването на switch оператор. От потребителя се изисква да въведе дата във формат месец, ден, година, например 5 11 1999. За всяка въведена дата, програмата определя деня от седмицата weekday като цяло число от 0÷6, получено в резултат на целочисленото деление на calc\_day (вижте сложната формула за изчисляване на calc\_day по-долу) на числото 7. Сложната формула за изчисляване на calc\_day изисква месеците януари и февруари да бъдат считани като 13-ти и 14-ти месец на

предходната година. Също така – въвежданата година трябва да е по-голяма от 1752 [9 – 12].

$$\begin{aligned} calc\_day &= day + 2month + (3(month + 1)/5) \\ &+ year + year/4 - year/100 + year/400 + 1. \end{aligned}$$

$$week\_day = \frac{calc\_day}{7}.$$

```
/* Лабораторно упражнение 4. Задача 5. */
/* C/C++ програма за определяне на денят от седмицата по въведена
от потребителя дата. Използват се съответна формула за определяне
на денят от седмицата и опертор switch */
#include <fstream.h>
#include <iomanip.h>

void main()
{int month, day, year, calc_day, weekday;
// calc_day - изчислява се по формула, представена по-горе
// weekday - цяло число от 0÷6
  cout<<"\n Въведете дата (мм дд гггг): ";
  cin>>month>>day>>year;
/* Смяна на месеца - за месеци Януари или февруари и корекция на
годината - с предишна, тоест намаляваме с 1. */
  if(month < 3)
    {month = month + 12; -- year; }
  calc_day = day + 2*month + (3*(month+1)/5) + year + year/4 -
    year/100 + year/400 + 1;
  weekday = calc_day % 7;
/* Възстановяване на сменения месец (за месеци Януари или
февруари) и възстановяване на годината. */
  if(month > 12)
    {month = month - 12; ++ year; }
  cout<<" Въведената дата " << setw(2)<<month<<"/"<<
    setw(2)<<day<<"/"<< setw(4)<<year << " се пада в ";
  switch(weekday)
    {case 0: cout << " Неделя."<<endl; break;
     case 1: cout << " Понеделник."<<endl; break;
     case 2: cout << " Вторник."<<endl; break;
     case 3: cout << " Сряда."<<endl; break;
     case 4: cout << " Четвъртък."<<endl; break;
     case 5: cout << " Петък."<<endl; break;
     case 6: cout << " Събота."<<endl;
    }
}
```

### Примерно изпълнение:

Въведете дата (мм дд гггг): 3 3 1985

Въведената дата 3/ 3/1985 се пада в Неделя



## Въпроси и задачи за самостоятелна работа

1. Какъв може да бъде резултатът от изпълнението на логически израз?
2. Възможно ли е логически израз да съдържа аритметичен израз?
3. Кои са операторите за разклонение (избор) в C/C++ програмите и какви са основните разлики между тях?
4. Възможно ли е условието в оператор if да бъде константа или променлива?
5. Защо изразът в оператор switch може да бъде само от целочислен или от символен тип?
6. Защо константите след case в оператор switch трябва да бъдат с различни стойности?
7. Група от X на брой туристи ще посети туристически обект. Напишете C/C++ програма, която изчислява груповата такса за посещение на този обект, имайки предвид, че таксата за вход е Y лева, но за посетители под 18 години и за студенти таксата се редуцира с 40%.
8. Търговски пътник получава премия от X лева към своята месечна заплата от Y лева, ако в рамките на месеца има продажби за най-малко 1500 лева или е установил контакти с най-малко 10 нови клиента. Търговският пътник получава с 25% по-ниска премия, ако в рамките на месеца има продажби за най-малко 500 лева, но повече от 25 нови клиента. Напишете C/C++ програма за изчисляване и извеждане на получената премия и новополучената заплата.
9. Напишете C/C++ програма, която въвежда получените за текущия семестър оценки от изпити на студент. Да се изведе средния успех на студента и да се изчисли и изведе стипендията му за следващия семестър както следва: ако успехът на студента е отличен, неговата стипендия е MAX лева, ако успехът му е много добър – стипендията е 75% от MAX, за добър успех стипендията на студента е 60% от MAX.
10. Нека глобата, налагана на водачи на моторни превозни средства, превишаващи допустимата скорост зависи от минималната работна заплата MIN и се определя по следния начин:  
Ако превишаването на скоростта е:  
до 20 км./ч. – глобата е 15% от MIN,  
от 21÷40 км./ч. - глобата е 25% от MIN,  
от 41÷50 км./ч. – глобата е 40% от MIN  
от 50÷69 км./ч. – глобата е 70% от MIN.  
Над 70 км./ч. – санкцията е „отнемане на правото за управление!”.  
Напишете C/C++ програма, която изисква от потребителя да въведе скоростта на моторното превозно средство и допустимата скорост. Програмата да “анализира” получените резултати и да информира за наложените санкции.

## Тема 5. Програмиране на задачи с цикли. Оператори за цикъл `for`, `while`, `do...while`. Оператори `break` и `continue`.

**Цел:** Реализиране и анализиране на циклични C/C++ програми, за запознаване с операторите за организиране на цикъл и начините за тяхното прилагане.

### Програмиране на задачи с цикли. Оператори за цикъл.

Когато е необходимо да се наруши естествения ред на изпълнение на операторите (във функция) и да се организира **многократно изпълнение** на оператор/блок от оператори се прилагат типови **циклични** програмни конструкции, известни като **оператори за цикъл**.

В C/C++ програмите е възможно използването на **три** типа оператори за цикъл: `for`, `while` и `do...while`.

#### 1. Оператор за цикъл `for`.

Общият вид на този оператор (известен и като **броячен** цикъл) е:

```
for (израз1; израз2; израз3)  
    оператор;
```

където:

**израз1** служи за **инициализация** (начално задаване) на **индекса** на цикъла (или и на други променливи) и се изпълнява **веднъж** – при **влизване** в цикъла.

**израз2** е проверка на **условието за излизане от цикъла** и се пресмята преди **всяко преминаване през тялото** на цикъла.

**израз3** е **актуализация** на параметрите на цикъла и се пресмята **след всяко изпълнение на оператора в тялото** на цикъла.

**Тялото** на цикъла е означено с **оператор**, който може да е единствен C/C++ оператор или **съставен** оператор.

Цикълът продължава докато **израз2** стане **0** (*false*), при което управлението се предава на оператора, следващ непосредствено тялото на `for`. Възможно е цикълът да не се изпълни **ниतो веднъж**.

Някои от изразите в отделните секции на оператор `for` могат да отсъстват като стойността им се приема за **различна от 0** (*true*).

**Примери:** Оператори `for` за сумиране на числата от 1 до 100.

```
for(int i=1,sum=0; i<=100; i++) sum+=i; // "Прав" брояч  
for(int i=100,sum=0; i>0; i--) sum+=i; // "Обратен" брояч
```

Цикълът `for` се прилага и при обработване на **масиви от данни**, когато предварително е известно в какви граници се изменят индексите.

#### 2. Оператор за цикъл с предусловие – `while`.

Общият вид на този оператор е:

```
while (израз)  
    оператор;
```

Операторът се изпълнява многократно докато **израз** има стойност **различна от 0** (*true*). Ако **израз** има стойност **0** (*false*), изпълнението на

цикъла се преустановява. Стойността на израза се изчислява преди изпълнението на оператора и е възможно **тялото на цикъла** да не бъде изпълнено **никога**.

**Пример:** Оператор **while** за сумиране на числата от 1 до 100.

```
int i=1, sum=0;
while (i<=100)
    {sum+=i; i++;}
```

Операторът за цикъл **while** е удобен за прилагане в задачи с **итерации**, където всяко ново решение се доближава до желания резултат с определена **точност** или друго условие за край на цикъла.

### 3. Оператор за цикъл с постусловие - **do...while**.

Общият вид на този оператор е:

```
do {
    оператор;
} while (израз);
```

Изпълнява се **операторът** (операторите) в тялото на цикъла и след това се проверява стойността на **израза**, като при стойност **различна от 0 (true)** операторът се изпълнява отново. Ако изразът има стойност **0 (false)**, цикълът се преустановява. Следователно, **първото изпълнение** на **оператора** не зависи от това, изпълнено ли е условието или не и **се изпълнява поне един път**.

**Пример:** Оператор **do...while** за сумиране на числата от 1 до 100.

```
int i=1; sum=0;
do{sum+=i; i++;
   }while (i<=100);
```

Операторът за цикъл **do...while** е удобен за проверка на **коректността** на входни данни, опериране с **менюта** и изчакване на определени **събития**.

## Оператори **break** и **continue**

### 1. Оператор **break**.

**Прекъсва** изпълнението на **цикъла** и препраща управлението на **първия оператор** след него. Ако има **вложени цикли** се прекъсва само **вътрешния** цикъл, където е включен оператора **break**.

**Пример:** Оператор **break** в тялото на “безкраен” цикъл с оператор **for** за сумиране на числата от 1 до 100.

```
for(int i=1, sum=0; ; i++) // Условието за край е true
    {sum+=i; if(i==100) break;}
```

Прилага се при възникване на ситуации, налагащи промяна в обработката на останалите елементи, например от масиви или входни данни. Когато **цикълът** се намира във **функция**, може да се използва оператор **return** за прекъсване на цикъла и връщане на управлението в извикващата функция.

## 2. Оператор *continue*.

Операторът се използва само с операторите за цикъл **for**, **while**, **do...while**. Препраща управлението в края на текущата **итерация** на цикъла и цикличният процес **продължава** със следващата итерация.

**Пример:** Оператор **continue** в тялото на оператор **for** за сумиране на числата от 1 до 10, с изключение на 7 и 8.

```
for(int i=1, sum=0; i<=10; i++)
{if(i==7 || i==8) continue;
 sum+=i;} // Този оператор не се изпълнява за i=7 и за i=8
```

Осигурява пропускането на част от операторите в тялото на цикъла, обикновено като резултат от проверка на условие.

## 3. Функция *exit()*.

Прекъсва **изпълнението на програмата** като връща код – **0** (към операционната система) за **нормално** завършване или **ненулева** стойност за индикация на **грешка**. Функцията е дефинирана в заглавния файл **stdlib.h**, където е и функцията **abort()** – за **незабавно излизане от програмата**.

Използват се за **безусловно** прекъсване на програмата.

## Вложени цикли

**Тялото на цикъла** се състои от изпълними оператори, в това число и други **оператори за цикъл** (т. нар. “**вложени**”). Телата на **вложените** цикли **не трябва да се пресичат**, т.е. вътрешният цикъл се съдържа изцяло в тялото на съответния му външен цикъл. **Броят** на вложените цикли **не е ограничен** [4]. Всеки **цикъл** използва различен **индекс**, има само **един вход** и изпълнението му може да започне само **от входа**.

Прилагат се при работа с многомерни масиви и сложни функции.

В операторите за цикъл в **C/C++** програмите индексът на цикъла и неговата гранична стойност могат да променят стойността си в тялото на цикъла, а при излизане от цикъла променливата **бройч** запазва своята стойност.

## Примерни задачи и програмни реализации с цикли

**Задача 5.1.** Да се състави **C/C++** програма, за въвеждане на последователност от цели числа до въвеждането на числото **0** и намиране на произведението само от положителните числа. Да се реализира в няколко варианта.

**Примерни решения:**

**Първият** вариант е реализиран чрез цикъл **while** и предварителна проверка на условието за край на цикъла. Затова **първото число** от поредицата се въвежда **преди цикъла** и участва в **условието за край**, а всяко следващо число се въвежда вътре в цикъла.

При въвеждане на числови данни се използва буфер, а разделителят е празна позиция, табулатор или нов ред. За да няма

грешки в резултата, след всяко въведено число натиснете например **Enter**, а накрая въведете **0** - за край на цикъла.

Променливата **pro**, в която ще се съхранява произведението (като междинен и краен резултат) е дефинирана като тип **unsigned long int** с начална стойност **1**, за да може в нея да се натрупва произведение от "произволен" брой положителни числа без да се получи **препълване** (максимална стойност до **4 294 967 295**).

```
/* Лабораторно упражнение 5. Задача 1. Вариант 1. */
```

```
#include <iostream.h>
```

```
void main()
{
    int num;
    unsigned long int pro=1;
    cout<<" Въведете цели числа или 0 за изход:\n";
    cin>>num;          // Първото въведено число
    while(num!=0)
    {
        if(num>0)
            pro*=num;
        cin>>num;      // Всяко следващо въведено число
    }
    cout<<" Произведението от положителните числата е = "<<pro;
}
```

Във **втория** вариант се използва **безкраен** цикъл и оператор **break**, който прекъсва цикъла при настъпване на събитието за край на цикъла.

```
/* Лабораторно упражнение 5. Задача 1. Вариант 2 */
```

```
#include <iostream.h>
```

```
void main()
{
    int num;
    unsigned long int pro=1;
    cout<<" Въведете цели числа или 0 за изход:\n";
    while(1)
    {
        cin>>num;
        if(num == 0)
            break;
        else if (num > 0)
            pro*=num;
    }
    cout<<" Произведението от положителните числата е = "<<pro;
}
```

Използването на оператор **,** (запетая), при който изразите, разделени със **запетая** се изпълняват в зададения **ред отляво надясно** и стойността на целия израз е равна на тази от **последния** изпълнен оператор, води до по-компактен запис на оператора за цикъл.

В **третия** вариант чрез израза в **while** се въвежда число и се проверява условието за край – дали числото е различно от **0**. Ако последното е истина се изпълняват операторите в тялото на цикъла.

```
/* Лабораторно упражнение 5. Задача 1. Вариант 3 */
```

```
#include <iostream.h>
```

```

void main()
{
    int num;
    unsigned long int pro=1;
    cout<<" Въведете цели числа или 0 за изход:\n";
    while(cin>>num, num!=0)
        {if(num > 0)
            pro*=num;
        }
    cout<<" Произведението от положителните числата е = "<<pro;
}

```

**Задача 5.2.** Съставете програма, в която се въвежда последователност от цели числа до въвеждане на символа за край на файл (**EOF**) и се намира сумата от числата, кратни на 3.

**Примерно решение:**

```

/* Лабораторно упражнение 5. Задача 2. */
#include <iostream.h>
#include <stdio.h>

void main()
{
    int num,sum=0;
    cout<<" Въведете последователност от цели числа (EOF-край) : \n";
    while (scanf ("%d", &num) !=EOF)
        {if (num%3==0)
            sum+=num;
        }
    cout<<" Сумата от кратните на 3 числа е = "<<sum;
}

```

Функцията **scanf()** връща **броя** на успешно въведените аргументи (в случая 1) или константа **EOF**, която е равна на -1 при въвеждане на **край на файл** (чрез клавишна комбинация **Ctrl+Z**) или при **грешка** [3].

**Задача 5.3.** Да се състави **C/C++** програма, която изчислява стойностите на функцията  $f(x)=x^4+2x^3-13x^2+8x$  за стойности на аргумента  $x$ , изменящи се в диапазона  $0,5 \leq x \leq 2,5$  със стъпка 0,1 [2].

**Примерно решение:**

Използват се **манипулатори** за **извеждане** на резултатите в **колони** за  $x$  и  $f$ . Дефинициите на тези манипулатори се намират в заглавния файл **iomanip.h**, което налага да бъде включен в програмата.

```

/* Лабораторно упражнение 5. Задача 3. */
#include <iostream.h>
#include <math.h>
#include <iomanip.h>

void main()
{float x,f;
    cout << setiosflags(ios::fixed | ios::showpoint);
    /* fixed - представя числата във формат с фиксирана точка, а
    showpoint - вмъква десетична точка в изходния поток */
}

```

```

for (x = 0.5; x <= 2.5; x+=0.1 )
{f = pow(x,4)+2*pow(x,3)-13*pow(x,2)+8*x;
cout << "x =" << setprecision(1) << setw(4) << x;
cout << "\t f(x)=" << setprecision(6) << setw(10) << f << endl;
}
}

```

**Задача 5.4.** Да се състави **C/C++** програма, чрез която по зададен интервал от кодове на **ASCII** символи да се изведат техните изображения в колона на екрана.

**Примерно решение:**

Знаете, че в паметта на компютъра всички данни се съхраняват в цифров вид. За съхраняване изображенията на символи, на всеки символ съответства цифров код, т.нар. **ASCII** код, който “присвоява” число от **0** до **255** на всяка главна или малка буква, цифра, препинателен знак или друг символ. Така, че когато записвате например ‘a’ в променлива от тип **char**, то в действителност записвате числото **97**, което е нейния **ASCII** код.

/\* Лабораторно упражнение 5. Задача 4. \*/

#include <stdio.h>

#include <conio.h>

void main()

{unsigned char x;

for(x=200; x<240; x++)

printf("\t ASCII кода %d е символ %c \n", x,x);

getch();

}

Въведете и изпълнете програмата, след което променете диапазона за стойности на **x**, за да видите изображенията и на **други символи**. За по широк диапазон (например от **0** до **255**) трябва да добавите в програмата оператори за задържане “**превъртането**” на екрана.

**Задача 5.5.** Съставете **C/C++** програма за намиране големината на ъглите на триъгълник и грешката при тяхното пресмятане, ако са зададени от клавиатура размери на страните – **a**, **b** и **c**. Включете в програмата проверка за валидност на въвежданите стойности.

**Примерно решение:**

Цикълът **do...while** се използва само за проверка коректността на входните данни (стойностите на **a**, **b** и **c** могат ли да бъдат страни на триъгълник). Останалите изчисления са от геометрията и включват **полупериметър** и **лице** (променливи **p** и **s**) на триъгълник, определяне на **ъгли** (променливи **alpha**, **beta** и **gamma**) чрез лице на триъгълник и определяне на **грешката** (променлива **E**), чрез допълнението на сумата от вътрешните ъгли до **180** градуса. Константата  $\pi=3,14159$  е дефинирана с директива **define**.

```

/* Лабораторно упражнение 5. Задача 5. */
#include <iostream.h>
#include <math.h>
#include <iomanip.h>
#define PI 3.14159

void main()
{
    float a,b,c,alpha,beta,gama,E,p,s;
    do{cout<<" Въведете страни на триъгълника:";
        cout<<"\n a= "; cin>>a;
        cout<<"\n b= "; cin>>b;
        cout<<"\n c= "; cin>>c;
        }while(a<=0||b<=0||c<=0||a>=b+c||b>=a+c||c>=a+b);
    p=(a+b+c)/2;
    s=sqrt(p*(p-a)*(p-b)*(p-c));
    alpha=asin(2*s/(b*c))*(180/PI);
    beta=asin(2*s/(a*c))*(180/PI);
    gama=asin(2*s/(a*b))*(180/PI);
    E=abs(alpha+beta+gama-180);
    cout<<"\n alpha = "<< setprecision(2)<<alpha;
    cout<<"\n beta = "<< setprecision(2)<<beta;
    cout<<"\n gama = "<< setprecision(2)<<gama;
    cout<<"\n Грешката е = "<< setprecision(2)<<E;
}

```

**Задача 5.6.** Да се състави C/C++ програма с меню за избор на математическа функция: логаритъм, експонента, квадратен корен или тригонометрична функция: **sin()**, **cos()**, **tg()**, **cotg()**, както и за пресмятане и извеждане на стойността ѝ по въведен от клавиатурата аргумент **a**.

**Примерно решение:**

Освен за определяне на коректността на входните данни, цикълът **do...while** е подходящ и за многократно извеждане на **меню** за избор на една от няколко възможности, докато се избере изход от менюто. В примерната програма се извежда **каскадно меню** за избор на **математически** функции.

```

/* Лабораторно упражнение 5. Задача 6. */
#include <iostream.h>
#include <math.h>
# include <conio.h>

void main()
{int choice,ch;    // За избор от меню
  float a;        // Аргумент на математическа функция

  do
  {clrscr();      // функция за изчистване на екрана
    cout<<"\n Въведете стойност за a: ";
    cin>>a;
    cout<<"\n    Математически функции\n";
    cout<<"1. log\n";

```



```

cout<<"2. exp\n";
cout<<"3. sqrt\n";
cout<<"4. Тригонометрични функции\n";
cout<<"5. Изход от програмата\n";
do
{
    cout<<"\n Изберете от 1 до 5: "; cin>>choice;
}while(choice<1||choice>5); //Проверка за коректност на избора
switch(choice)
{
    case 1: cout<<" log("<a<<" = "<<log(a); break;
    case 2: cout<<" exp("<a<<" = "<<exp(a); break;
    case 3: cout<<" sqrt("<a<<" = "<<sqrt(a); break;
    case 4: do{
        cout<<" \n Тригонометрични функции \n\n";
        cout<<"1. sin\n";
        cout<<"2. cos\n";
        cout<<"3. tg\n";
        cout<<"4. cotg\n";
        cout<<"5. Връщане в менюто\n";
        do
        {
            cout<<"\n Изберете от 1 до 5: "; cin>>ch;
        }while(ch<1 || ch>5);
        switch(ch)
        {
            case 1: cout<<"sin("<a<<" = "<<sin(a); break;
            case 2: cout<<"cos("<a<<" = "<<cos(a); break;
            case 3: cout<<"tg("<a<<" = "<<tan(a); break;
            case 4: cout<<"cotg("<a<<" = "<<1/tan(a);
                    break;
            case 5: cout<<" Връщане в менюто";
        } cout << endl; getch();
        } while(ch!=5); break;
    case 5: cout<<"\n Край на програмата! \n";
    } getch();
}while(choice!=5);
}

```

**Задача 5.7.** Да се състави C/C++ програма за пресмятане и извеждане на произведението **R** за зададен брой множители – **n**:

$$R=(x^2/2+1)(x^3/4+1)(x^4/6+1) \dots (x^{n+1}/2n+1)$$

**Примерно решение:**

Програмата натрупва определен **брой** множители **n**, зададен от клавиатурата. Променливата **R** е дефинирана от тип **double** за максимален интервал на стойността на резултата.

```

/* Лабораторно упражнение 5. Задача 7. */
#include <iostream.h>
#include <math.h>

void main()
{
    int n,i;
    float x;
    double R=1;
    cout<<"\n Въведете x= "; cin>>x;

```

```

    cout<<"\n Въведете n= "; cin>>n;
    for(i=1; i<n+1; i++)
        R*=(pow(x, (i+1)) / (2*i)+1);
    cout<<"\n Резултат R= "<<R;
}

```

**Задача 5.8.** Да се състави C/C++ програма за пресмятане на степенния ред:

$$V=a + x/a + (x/a)^3 + (x/a)^5 + \dots + (x/a)^{2n-1}$$

с точност до  $10^{-7}$  и извеждане на резултата на екрана.

**Примерно решение:**

Към натрупаната до момента **сума** се добавя ново събираемо, само ако не е удовлетворено все още **условието за точност**, а именно **разликата** (по абсолютна стойност) между **две последователни суми** да е под определената в условието за точност стойност. Променливата **v\_old** съхранява старата сума, а **v\_new** е новата. Променливата **n** има начална стойност **1** и се изменя през **2**, за да бъдат степените нечетни.

```

/* Лабораторно упражнение 5. Задача 8. */
#include <iostream.h>
#include <math.h>

void main()
{
    int n=1;
    float v_old,v_new,x,a;

    cout<<"x= "; cin>>x;
    cout<<"a= "; cin>>a;
    v_new=a; // Първото събираемо
    do{
        v_old=v_new;
        v_new+=pow(x/a,n); // Новата сума
        n+=2;
    }while(fabs(v_new-v_old)>pow(10,-7)); //Условие за точност
    cout<<"\n V= "<<v_new; // Извеждане на резултата
}

```

**Задача 5.9.** Моделирайте програмно играта на „топло – студено“ чрез генериране на случайно число в диапазона между **1** и **M**. Задачата е да познаете генерираното число, като всеки път когато въведете число, програмата ви насочва дали вашето число е по-голямо от него – „топло“ или по-малко от генерираното число – „студено“ [5].

**Примерно решение:**

Функцията **srand()**, която е дефинирана в заглавния файл **stdlib.h** се използва веднъж в началото на **main()**, което гарантира, че генерираната последователност от числа ще бъде различна при всяко стартиране на програмата. Функцията **time(NULL)** от заглавния файл **time.h** се използва като аргумент на **srand()**, с цел да се гарантира нова стартова стойност за инициализация на генератора, винаги когато програмата се стартира.

```

/* Лабораторно упражнение 5. Задача 9. */
#include <iostream.h>
#include <stdlib.h>
#include <time.h>
#define M 100                // Най-голямото число за познаване

void main()
{int br=0;                  // Брояч на опитите за познаване
 long rando,guess;         // Генерирана и входна стойности
 srand(time(NULL));        // Инициализация на генератора
 rando = rand()%M + 1;     // Генериране на стойност между 1 и 100
 cout << "\n Въведете стойност между 1 и " << M <<endl;;
 do{cin >> guess;          // Въвежда се число
  br++;
  if(rando > guess)
   cout << " ТОПЛО ";
  else if( rando < guess)
   cout << " СТУДЕНО ";
  else
   {cout << " Познахте! \a Генерираното число бе " << rando;
    cout << "\n Бяха ви нужни " << br <<" опита.";
   }
 }while(rando != guess);
}

```

Функцията **rand()** връща случайно цяло число, в интервала от **0** до **RAND\_MAX**, където **RAND\_MAX** е цяла константа (**RAND\_MAX=32767**), дефинирана във файла **stdlib.h**. Изразът **rand()%M**, при **M=100** ще даде остатъка от делението на **rand()** на **100**, тоест цяло число в интервала **0** до **99**. Следва увеличаване на получения резултат с **1**, затова резултатът от израза:

```
rando = rand() %M+1;
```

е случайно число, от **1** до **100**.

При познато число се извежда съобщение **“Познахте!”** и се генерира **звуков** сигнал, зададен чрез управляващия символ **‘\a’**.

**Задача 5.10.** Съставете **C/C++** програма за табулиране на функцията **V(x)**, която представлява корен **3-ти** от **x**, чрез итерационната формула  $V_i = V_{i-1} + 1/3 \cdot (x/V_{i-1}^2 - V_{i-1})$ , за стойности на **x** от **-10** до **+10**, с точност **10<sup>-5</sup>**. Начално приближение за **V(x) = x/3**, ако **x >= 1** и **V(x) = x**, ако **x < 1**.

**Примерно** решение:

Цикълът **for** променя стойностите на параметъра **x** от **-10** до **+10**, а цикълът **do...while**, който се съдържа в него, изчислява итерационната формула за **V(x)**. Това са **вложени цикли**. Началната стойност на **V(x)** се пресмята с **if** оператор преди да започне цикълът за нейното поточно изчисление. Условието за край е постигане на определената в условието точност, която представлява разликата (по абсолютна стойност) между **двете последни итерации** (приближения).

```

/* Лабораторно упражнение 5. Задача 10. */
#include <stdio.h>
#include <math.h>

void main()
{float v0,v1,x; //Стара (v0) и нова (v1) стойност на приближенията
printf("Корен 3-ти от: ");
for(x=-10; x<=10; x++)
{if(x>=1) v1=x/3;
else v1=x;
do{
v0=v1;
if(v0==0) v1=0;
else v1=v0+1./3.*(x/(v0*v0)-v0);
}while (fabs(v1-v0)>.00001);
printf("%8.3f е равен на: %8.3f\n",x,v1);
}
}

```

**Задача 5.11.** Съставете C/C++ програма за кодиране на текст, като замествате в него символа 'A' с 'B', символа 'B' с 'C' и т.н., 'Z' с 'A'.

**Примерно решение:**

В цикъл **do...while** се четат символи и в зависимост от това дали са малки букви от **a** до **z** или главни букви от **A** до **Z** се преобразуват в следваща буква. Особен случай е буквата **z/Z**, която преминава в буква **a/A**. Всички **останали символи** остават **без промяна**. Флагът **i=0** се използва за еднократно извеждане на пояснителен текст за резултата. От цикълът се излиза с натискане на клавиша **Enter**.

```

/* Лабораторно упражнение 5. Задача 11. */
#include <stdio.h>

void main()
{ char a,b;
int i=0;
printf("\n\t Въведете поредица от символи ");
printf("- за край натиснете <ENTER>:\n\n\t=>");
do{a=getchar();
if(a>='a' && a<'z')
b=a-'a'+'b'; // За малки букви
else if (a>='A' && a<'Z')
b=a-'A'+'B'; // За главни букви
else if (a=='z')
b=a-'z'+'a'; // За малка буква z
else if (a=='Z')
b=a-'Z'+'A'; // За голяма буква Z
else b=a; // За всички останали символи
if(i==0)printf("\n\t Преобразуван текст:\n\n\t=>");
printf("%c",b); i--;
}while (a != '\n');
}

```

**Задача 5.12.** Да се състави **C/C++** програма за пресмятане факториелите на числата от **5** до **15** по формулата:  $N! = 1 * 2 * 3 * \dots * (N-1) * N$ , като се прескочат **10!** и **13!**.

**Примерно решение:**

Прилага се отново **вложен** цикъл, където **външният** цикъл определя аргументите от **5** до **15** (променлива *i*), на които се търси **факториел** във **вътрешния** цикъл. Началната стойност на всеки от факториелите е **1**, тъй като са последователни произведения на числата от **1** до текущия аргумент. Прескачането на **10** и **13** се осъществява чрез оператор **continue** – преминава на **следваща итерация**.

```
/* Лабораторно упражнение 5. Задача 12. */
#include <stdio.h>

void main()
{long double s;                // За пресмятане на факториел
 printf("\n");
 for(int i=5; i<16; i++)
 {if(i==10 || i==13) continue;
  s=1;                        // Начална стойност на факториела
  for(int a=i; a>1; a--)
   s=s*a;
  printf(" %d! = %.0Lf\n",i,s);
 }
}
```

Въведете и изпълнете програмата, след което променете диапазона за стойности на *i* за пресмятане на съответните факториели. Модифицирайте програмата за вложения **for** – а със стойности от 1 до *i*.

**Задача 5.13.** Съставете програма, която чете реално число от клавиатурата и ако то е положително, извежда неговия квадратен корен, а ако е отрицателно, извежда неговия квадрат. Цикълът продължава докато се въведе “край на файл” **EOF** (клавишна комбинация **Ctrl+Z**) или символ, различен от цифра [3].

**Примерно решение:**

В програмата се използва цикъл **for**, в чийто заглавен оператор са включени оператори за въвеждане на данни със съответни подканващи съобщения. В секцията за **инициализация** се въвежда **първото** число, а в секцията за **актуализация** се въвежда всяко **следващо** число. Условието за **край на цикъла** е **логически израз**, в който се проверява едновременно настъпване на **две** събития: да не е въведена символната константа **EOF** и да не е въведена стойност, различна от число (например символ). Функцията **scanf()** връща **int n = брой** на успешно въведените числа. Ако възникне грешка при въвеждането или прекъсване на входния поток чрез **Ctrl+Z**, връща константа **EOF(-1)**.

```
/* Лабораторно упражнение 5. Задача 13. */
#include <stdio.h>
#include <math.h>
```

```

void main()
{double num,square,sq_root;
 int n;
 for(printf(" Въведете число или Ctrl+Z за край->" ),
      n=scanf("%lf",&num); (n!=EOF && n!=0);
      printf("\n Въведете число или Ctrl+Z за край->" ),
      n=scanf("%lf",&num))
{if(num < 0)
 printf("Квадрата на числото %.2lf е %.2lf \n",num, pow(num,2));
else if (num > 0)
 printf("Квадратен корен от числото %.2lf е %.2lf \n",num,
        sqrt(num));
else printf("Числото е 0 \n");
}
}

```

## Въпроси и задачи за самостоятелна работа

1. По какво се различават операторите за цикъл for, while и do...while?
2. Колко пъти се изпълнява тялото на цикъла в оператор do...while и while при false условие за край на цикъл?
3. За какво се използват оператори break и continue в тялото на цикъл?
4. Колко нива на влягане на цикли са възможни?
5. Възможна ли е инициализация на повече от една променливи в заглавния оператор на цикъла for?
6. Реализирайте C/C++ програма, която въвежда числа в интервала от 0 до 99. Да се изведат сумата и произведението на въведените числа, броят на четните и броят на нечетните числа, както и средно-аритметичната им стойност, като за прекратяване на въвеждането на нови числа се приложи символа \*.
7. Напишете C/C++ програма за въвеждане на множество от изпитни оценки, изразени в точки (от 0 до 100). Входа на данни се прекратява с въвеждане на отрицателно число. При въвеждането, всяка оценка трябва да се валидира, тоест да се провери дали е в интервала от 0 до 100. Програмата трябва да изведе броя на въведените оценки, максималната и минималната, както и броя на оценките в указан от клавиатурата интервал.
8. Да се състави програма за изчисляване и извеждане стойността на функцията  $y(x)$ , за всяко  $x$  в интервала от -100 до 100, със стъпка 10.

$$y(x) = \begin{cases} \sqrt{x^2 + 1}, & x < 0 \\ \frac{x + 10}{x - 20}, & x \geq 0 \end{cases}$$

9. Съставете C/C++ програма, която отделя и сумира цифрите на цяло десетично число (тип unsigned long int).
10. Съставете C/C++ програма, която броеи въвежданите от клавиатура символи до въвеждане на край на файл (EOF).

## Тема 6. Масиви. Символни низове. Функции за работа със символни низове. Програми за работа с масиви и символни низове.

**Цел:** Съставяне, тестване и анализ на циклични C/C++ програми, съдържащи дефиниране, инициализация и поелементна обработка на масиви и символни низове.

### Масиви. Дефиниране, инициализация и обработка

В програмния език C/C++ могат да бъдат дефинирани и обработвани **едномерни** и **многомерни** масиви. **Масивът** е структура, която се асоциира с едно **име**, един **начален адрес**, един **тип** и **множество от стойности**.

**Едномерният масив** е списък от **еднотипни променливи**, които се съхраняват в **последователни клетки** от паметта. Всяка отделна променлива от масива се нарича **елемент на масива** и **достъпът** до тях е чрез **името на масива** и **индекс** (или **адрес**). **Индексите** на масива в C/C++ програмите започват от **нула**.

**Обща форма** за дефиниране на масив:

***тип име\_на\_масив [размер1] [размер2] [...];***

Използват се всички типове на C/C++ за дефиниране на **типа на елементите**, включително и масив от структури или от масиви. Името се съставя по правилата за идентификатор, т.е. започва с **буква** или знак '\_', следвани от **букви** и/или **цифри**. **Размерът** е израз от **цял тип**, който може да има само **положителни** стойности. **Не може** да бъде **променлива**, но може да бъде предварително **дефинирана константа** и определя **максималния брой** елементи при дефиниране на масива.

**Пример:** `float masiv1[100], alpha[2*15+56];`

**Името** на масива играе роля на **указател към началния адрес** на масива или казано иначе, към **нулевия** елемент на масива.

Един масив може да се **инициализира** още при дефинирането му.

**Пример:** `int m[6] = {3,5,9,1,0,12};` // `m[0]=3, m[1]=5,.....,m[5]=12`

Ако явно се инициализират **по-малко елементи** на масива, то **останалите** се инициализират **неявно с 0**.

**Общата форма** за инициализация на едномерен масив е:

***тип име\_на\_масив [размер] = {списък от стойности};***

Списъкът от стойности представлява **последователност** от **константи** разделени със запетаи.

**Масивите се обработват поелементно**, най-често чрез **оператор за цикъл**. Най-удобен **оператор за цикъл** при работа с масиви е операторът **for**. Освен въвеждането и извеждането на **елементи на масив**, едни от най-типичните обработки са за **търсене**, **пренареждане** и **сортиране** на масиви.

**Пример:** Програма за дефиниране и инициализиране на **масив с оценки** на студент от изпитна сесия. Преброяват се отличните оценки.

```
#include <iostream.h>
void main()
{int oc[6]={4,5,6,6,3,5};      // Масив с 6 оценки
  int br6=0;                    // Брояч на шестици
  for(int i=0; i<6; i++)
    if(oc[i]==6) br6++;
  cout << " Броят на шестиците е " << br6 << endl;
}
```

## Символни низове. Инициализиране. Масиви от низове.

**Символният низ** (стринг - string) е **едномерен** масив от тип **char**, който завършва с нулев символ **'\0'**. Той указва на компилатора къде е **края** на низа от символи [4]. Когато се дефинира с **константа**, **размерът** на масива трябва да е **поне с едно повече** от **броя** на символите в низа – за константата **'\0'**. **А когато размерът не е зададен явно**, компилаторът **преброява** символите и поставя накрая **'\0'** и това е размера на масива.

### Примери:

```
char s[5]={'g','a','m','a','\0'}; или s[5]= "gama";
char st[]=" Това е масив от символи";
```

За разлика от други езици за програмиране, в **C** не се поддържа стандартно дефиниран тип за символен низ.

**Символните низове** могат да се **обработват по символи** или чрез използване на **стандартни функции**. Стандартните библиотечни функции за работа със символни низове изискват включване в съответната програма на заглавния файл **string.h**.

Някои по-често използвани **функции** за работа с **низове** са [7]:

- **strcpy(низ1, низ2)** - копиране на низ2 в низ1, като не изменя низ2 и завършва с **'\0'**. Връща указател към низ1;
- **strcmp(низ1, низ2)** - сравняване на низ1 и низ2 по символи и връща цяло число: 0 – при равенство, <0 – ако низ1<низ2 и >0 – ако низ1>низ2;
- **strlen(низ)** – определя дължината на низа (в брой символи), като не отчита **'\0'**;
- **strncpy(низ1, низ2, n)** - копира първите *n* символа от низ2 в низ1, като не изменя низ2 и завършва с **'\0'**. Връща указател към низ1;
- **strncmp(низ1, низ2, n)** - сравнява първите *n* символа на низ1 и низ2 и връща цяло число: 0 при равенство, <0 - при низ1<низ2 и >0 - при низ1>низ2;
- **stricmp(низ1, низ2)** - сравнява низ1 и низ2 по символи, независимо от дължината на низовете и вида на буквите – големи или малки (от латинската азбука);
- **strupr(низ)** – преобразува малките букви в низа (от латинската азбука) в големи;



- **strlwr(низ)** – преобразува големите букви в низа (от латинската азбука) в малки;
- **strcat(низ1, низ2)** – добавя копие на низ2 до низ1 и връща указател към низ1;
- **strncat(низ1, низ2, n)** - добавя не повече от n символа от низ2 до низ1 и завършва с '\0'. Връща указател към низ1;
- **strchr(низ1, ch)** – връща указател към първия символ от низ1, който съвпада със символа *ch*. Ако няма съвпадение връща нулев указател;
- **strpbrk(низ1, низ2)** – връща указател към първия символ в низ1, който съвпада с който и да е символ от низ2. Ако няма съвпадение се връща нулев указател;
- **strstr(низ1, низ2)** - връща указател към първото срещнато съвпадение на стринга от низ2 в стринга низ1. Ако няма съвпадение се връща нулев указател;

В тези функции **низ1, низ2** са низови променливи или константи. Низовата константа се задава като последователност от символи, заградена с двойни кавички и се явява като операция, която връща указател към мястото в паметта, където е записана.

**Масив от символни низове** е **двумерен** масив от тип **char**. При дефиниране на **масиви от символни низове** **левият** размер определя **броя на редовете (низове)**, а **десният** - броя на символите в реда, т.е. **максималната дължина на низовете**.

**Пример:** Масива с имена на градове: `char grad_imena[100][10];` може да побере **100** низа с имена до **10** символа дължина всеки.

Необходимата памет за **двумерен масив** се определя като **произведение от двата размера** на масива. Елементите на **двумерния** масив се записват в паметта по реда на нарастване на десния индекс, т.е. **по редове**.

**Инициализацията** на двумерния масив се извършва чрез задаване на стойности за елементите по редове, заградени във **фигурни скоби** и разделени със **запетаи**.

**Примери:** `int x[3][4]={ {2,5,9,6}, {8,9,6,7}, {1,4,0,3} };  
char stih[4][]={ "Настане вечер, ",  
                  "месец изгрее, ",  
                  "звезди обсипят ",  
                  "свода небесен, " };`

**Въвеждането** на стрингове по време на изпълнение на програмата се извършва с функциите **gets()** и **scanf()**, дефинирани в заглавния файл **stdio.h**, а **извеждането** на стрингове се извършва с използване на функциите **puts()** – за “стандартно” извеждане и **printf()** – за форматирано извеждане със спецификатор **%s** за стринг (също от заглавния файл **stdio.h**). Освен тези функции могат да се използват и операциите за **потоци**: **cin>>** и **cout<<** от заглавния файл **iostream.h**.

## Примерни задачи и програмни реализации с масиви и символни низове

**Задача 6.1.** Зададен е едномерен масив **A**, чиито елементи са цели положителни числа. Да се състави **C/C++** програма за намиране броя на четните и броя на нечетните елементи на масива и извеждане на: масива **A**, масива **B**, формиран от четните елементи на **A** и масива **C**, формиран от нечетните елементи на **A**.

**Примерно решение:**

В програмата са дефинирани **три** масива с еднакъв размер, като въвеждане на входни данни се извършва само в първия, а втория и третия съхраняват само **четните** (съответно **нечетните**) стойности на входния масив. Техните индекси **m** и **k** са нулирани преди цикъла **for** и се **инкрементират** само при запис на **нов елемент** в съответния масив.

```
/* Лабораторно упражнение 6. Задача 1. */
#include <iostream.h>
#include <conio.h>

void main()
{unsigned int a[10],b[10],c[10];
 int m=0,k=0,temp;
 clrscr();
 for(int i=0;i<10;i++)      // Основен цикъл за попълване на масив а
 {cout<<" Въведете положително число в A["<<i+1<<"]= ";
  cin>>temp;                // Чете в работна променлива
  while(temp <= 0)
  {cout<<" Само положителни числа моля - опитайте пак";
   cin>>temp;
  }
  a[i]=temp;                // Записва в масива положително число
  if(a[i]%2==0)              // Ако е четно
  {b[k]=a[i]; k++;}         // копира го в масива в
  else                      // Ако е нечетно
  {c[m]=a[i]; m++;}         // копира го в масива с
 }                          // Край на цикъла за въвеждане
 cout<<" Четните числа са:\n";
 if(k!=0)                   // Ако има четни
 for(i=0; i<k; i++)
  cout<<"B["<<i+1<<"]="<<b[i]<<"\n";
 else cout<<" Няма четни числа в масива";
 cout<<" Нечетните числа са:\n";
 if(m!=0)                   // Ако има нечетни
 for(i=0; i<m; i++)
  cout<<"C["<<i+1<<"]="<<c[i]<<"\n";
 else cout<<" Няма нечетни числа в масива";
 getch();
 }
```

Новите масиви **b[ ]** и **c[ ]** се попълват с въвеждането на входните данни. Това прави масива **a[10]** “излишен” - не се извежда на екрана.

Програмата демонстрира работа с **едномерни числови масиви**, за които най-подходящ оператор в обработката е **for**.

**Задача 6.2.** Реализирайте програма на **C/C++**, която имитира речник за превод на думи от един език на друг. Преводът се състои в заместване на всяка дума от езика **E1**, с нейното съответствие от езика **E2**, съгласно дадена таблица.

**Примерно решение:**

Основната дейност при тази задача е създаването на **речник от думи на двата езика**, съхранени в **двумерен символен масив** с име **dictionary[ ][ ]**. Първият размер е **брой** на думите, а вторият – **дължината** на стринга за всяка дума. От **меню** се избира посоката на превода и се влиза в **switch**, който прави две еднотипни претърсвания на речника за зададена от клавиатурата дума, чрез функцията **strcmp()**. Тази функция връща **0** при съвпадение на думата с някоя от речника, затова пред нея е поставено отрицание - **!**, за да обърне условието в **1 (true)**. Тогава се извежда следващата дума от речника, която представлява превода на зададената. След запитване се продължава със следваща дума за превеждане.

```
/* Лабораторно упражнение 6. Задача 2. */
#include <iostream.h>
#include <stdio.h>
#include <conio.h>
#include <string.h>
#define N 26                      // Брой думи в речника

void main()
{char dictionary[N][20]={
    "I",                          "аз",
    "am",                          "съм",
    "student",                     "студент",
    "at",                          "в",
    "Technical",                   "Технически",
    "University",                  "Университет",
    "of",                          "на",
    "Varna",                       "Варна",
    "Department",                  "катедра",
    "Computer science",            "Компютърни науки",
    "study",                       "уча",
    "programming",                 "програмиране",
    "C++",                         "C++"};

int i,flag;
char word[20],ch; // Символен масив за дума и символ за запитване
clrscr();
do{cout<<" Изберете:\n 1. English to Bulgarian";
  cout<<"\n 2. Български към Английски";
  cout<<"\n Вашият избор, моля: ";
  cin>>flag;
}while(flag<1 || flag>2);
switch(flag)
{case 1: do{cout<<"\n English word: ";
            fflush(stdin);          // Изчиства входния буфер
            gets(word);             // Чете дума
```

```

        for(i=0; i<N; i++)
            if(!strcmp(word,dictionary[i])) // Търси в речника
                cout<<"\n"<<word<<"-"<<dictionary[i+1];
                // Извежда превод
        cout<<"\n More words:[y/n]";
        ch=getchar(); // Чете отговор за продължаване
    }while(ch=='y' || ch=='Y'); break;
case 2: do{cout<<"\n Българска дума: ";
        fflush(stdin);
        gets(word);
        for(i=0; i<N; i++)
            if(!strcmp(word,dictionary[i]))
                cout<<"\n"<<word<<"-"<<dictionary[i+1];
        cout<<"\n Друга дума:[y/n]";
        ch=getchar();
    }while(ch=='y' || ch=='Y');
}
}

```

В програмата се дефинира и инициализира масив от символни низове с **явно указани размери**. Масивът от символни низове е двумерен от тип **char**, като първият размер определя броя на елементите в масива, а вторият определя максималната дължина на стринговете. Ако вторият размер е указан неявно (**dictionary[N][ ]**), то се приема дължината на въведения стринг и понякога се пести памет.

**Задача 6.3.** Зададена е матрица **A(3,3)**. Да се състави **C/C++** програма за преобразуване в нова матрица **B(3,3)** по правилото: ако поне един от елементите в последния ред на матрица **A** е отрицателен, то на елементите по главния диагонал на матрица **B** да се присвои стойност **1**, в противен случай матрицата **B** да не се преобразува. Да се изведат матриците **A** и **B** в подходяща таблична форма и с пояснителен текст.

**Примерно решение:**

Интересен момент в програмата е копирането на въведената от клавиатура матрица **A** в **B**, за да може в случай на намерен **отрицателен елемент** да се заменят само **диагоналните елементи с 1**, а останалите да **не се променят**. Наличието на **отрицателен елемент** се регистрира с вдигане на **flag** в **1** – тогава се променя само **главния диагонал** на **B**, в противен случай двете матрици са **еднакви**.

```

/* Лабораторно упражнение 6. Задача 3. */
#include <iostream.h>
#include <iomanip.h>
#include <conio.h>

void main()
{int i,j,flag=0;
 float A[3][3],B[3][3];
 clrscr();
 cout<<"\n Въведете матрицата A(3,3):";

```

```

for(i=0;i<3;i++)
    for(j=0;j<3;j++)
        {cout<<"\n A["<<i+1<<"] ["<<j+1<<"]=";
          cin>>A[i][j];
        }
for(i=0;i<3;i++)          // Презаписва матрица A в B
    for(j=0;j<3;j++)
        B[i][j]=A[i][j];
// Търси отрицателен елемент по последния ред на матрица A
for(i=0;i<3;i++)
    if(A[2][i]<0) flag=1;    // Ако има поне един отрицателен елемент
if(flag==1)
    for(i=0;i<3;i++)        // Записва единици по диагонала на B
        B[i][i]=1;
// Извеждане на двете матрици
clrscr();
cout<<"\n Матрица A(3,3):\n";
for(i=0;i<3;i++)
    {for(j=0;j<3;j++)
        cout<<setprecision(2)<<A[i][j]<<"\t";
      cout<<"\n";
    }
cout<<"\n Матрица B(3,3):\n";
for(i=0;i<3;i++)
    {for(j=0;j<3;j++)
        cout<<setprecision(2)<<B[i][j]<<"\t";
      cout<<"\n";
    }
getch();
}

```

Програмата демонстрира начини за обработка на **двумерни** числови **масиви** чрез **вложени цикли for**. Първият цикъл **for** управлява **редовете** на **матрицата**, а вторият изрежда **елементите** в **реда**.

**Задача 6.4.** Съставете **C/C++** програма за създаване и извеждане таблицата на Питагор. Тя е квадратна матрица с **10** реда и **10** стълба, като всеки елемент в нея е дефиниран като произведение от номера на реда и номера на стълба.

**Примерно решение:**

Задачата е “лека” и показва, че в **C/C++** програмите **броенето** в **масивите** започва от **0** и затова елементите на Питагор са  **$(i+1)*(j+1)$** .

```

/* Лабораторно упражнение 6. Задача 4. */
#include <iostream.h>
#include <conio.h>

void main()
{int P[10][10],i,j;
  for(i=0; i<10; i++)
    for(j=0; j<10; j++)
      P[i][j]=(i+1)*(j+1);
}

```

```

for(i=0; i<10; i++)
{for(j=0; j<10; j++)
    cout<<P[i][j]<<"\t";
    cout<<"\n";
}
getch();
}

```

**Задача 6.5.** Съставете C/C++ програма за намиране максималната и минималната дължина на въведени от клавиатурата редове от символи.

**Примерно решение:**

Дефинирани са **два масива с размер**, зададен като **символна константа** – **N**. Единият е **символен (text)** и представлява **N** на брой **реда** с по **80** символа в ред, а вторият е **числов (len)** и представлява **N** на брой **дължини** на редове. За определяне на дължините на редове се прилага функцията **strlen()**, която връща **броя символи** без терминиращия **'\0'** и е дефинирана в заглавния файл **string.h**.

```

/* Лабораторно упражнение 6. Задача 5. */
#include <stdio.h>
#include <conio.h>
#include <string.h>
#define N 10

void main()
{char text[N][80];          // Редове текст
  int len[N];
  int i,min,max,x=0,n=0;    // n - номер на ред с мин. брой символи
  clrscr();                // x - номер на ред с макс. брой символи
  printf(" Въведете текст до %d реда \n",N);
  for(i=0;i<N;i++)
    gets(text[i]);         // Реда завършва с Enter
  for(i=0; i<N; i++)
    len[i]=strlen(text[i]); // Вектор len[] с дължини на редове
  min=80;max=0;            // Инициализация на max и min
  for(i=0; i<N; i++)
  {if(len[i]<min)
    {min=len[i]; n=i+1; }  // Запазва в min по-малката дължина
    if(len[i]>max)
    { max=len[i]; x=i+1; } // Запазва в max по-голямата дължина
  }
  clrscr();
  for(i=0; i<N; i++)
    puts(text[i]);         // Извежда текста
  printf("\n Максимална дължина на %d ред: %d",x,max);
  printf("\n Минимална дължина на %d ред: %d",n,min);
  getch();
}

```

**Задача 6.6.** Съставете програма, която въвежда **цяло десетично** число, преобразува го в **двоично** и го извежда на екрана [3].

**Примерно решение:**

Преминаването от една бройна система в друга се извършва чрез **многократно деление** на зададеното число (в случая **десетичното** число **dm**) на **основата** на бройната система, в която ще се превръща (в случая **2**) и съхраняване на **остатъците** от делението като значещи цифри от **новата бройна система** (в случая **двоичната** - **bin**). Тези значещи цифри, **изведени в обратен ред** на получаването, представят стойността на входното число в **новата бройна система**.

```
/* Лабораторно упражнение 6. Задача 6. */
#include <stdio.h>
#include <conio.h>

void main()
{int bin[16],dm,i,k; // Масив за цифрите в двоичното число-bin[16]
 printf("\n Въведете десетично число: ");
 scanf("%d",&dm);
 printf(" Двоичното число е: ");
 for(i=0; dm>0; i++) // Докато частното е >0
 {bin[i]=dm%2; // Новия остатък
 dm=dm/2; // Новото частно
 }
 for(k=i-1; k>=0;k--) // Извеждане в обратен ред
 printf("%d", bin[k]); // на остатъците от делението
 getch();
}
```

Изпълнете програмата, след което съставете вариант със запитване за продължение за въвеждане на ново десетично число.

**Задача 6.7.** Даден е двумерен масив **A[4][5]**. Да се изчислят сумите от елементите по редове и да се намери минималната сума сред тях. Резултатът да се изведе във вида:

|                       |                       |                       |                       |                       |                      |
|-----------------------|-----------------------|-----------------------|-----------------------|-----------------------|----------------------|
| <b>a<sub>11</sub></b> | <b>a<sub>12</sub></b> | <b>a<sub>13</sub></b> | <b>a<sub>14</sub></b> | <b>a<sub>15</sub></b> | <b>S<sub>1</sub></b> |
| <b>a<sub>21</sub></b> | <b>a<sub>22</sub></b> | <b>a<sub>23</sub></b> | <b>a<sub>24</sub></b> | <b>a<sub>25</sub></b> | <b>S<sub>2</sub></b> |
| <b>a<sub>31</sub></b> | <b>a<sub>32</sub></b> | <b>a<sub>33</sub></b> | <b>a<sub>34</sub></b> | <b>a<sub>35</sub></b> | <b>S<sub>3</sub></b> |
| <b>a<sub>41</sub></b> | <b>a<sub>42</sub></b> | <b>a<sub>43</sub></b> | <b>a<sub>44</sub></b> | <b>a<sub>45</sub></b> | <b>S<sub>4</sub></b> |

**S<sub>min</sub>**

**Примерно решение:**

Въвежда се от клавиатура матрицата **A** с **двоен** цикъл. **Нулира се** векторната **сума** за всеки ред от матрицата (вътре в цикъла за съответния ред) и след това във вложения цикъл се натрупва самата сума за реда, заедно с извеждане на съответния елемент от матрица **A**. Това се повтаря за всеки елемент от реда и накрая се извежда получената **сума за реда**. След изчисляване на всички суми се взема **първата** за **минимална** и се сравнява с всички останали. Накрая в **S<sub>min</sub>** остава минималната, която се извежда на екрана.

```
/* Лабораторно упражнение 6. Задача 7. */
#include <iostream.h>
```

```

#include <conio.h>

void main()
{int A[4][5],S[4],i,j,Smin; // Матрица A и масив за сумите S
  for(i=0; i<4; i++)
    for(j=0; j<5; j++)
      {cout<<"\n A["<<i+1<<"]["<<j+1<<"]=" ";
        cin>>A[i][j]; // Въвеждане на A
      }
  for(i=0; i<4; i++) // Номер на ред
    {S[i]=0; // Нулиране на сумата за реда
      for(j=0; j<5; j++) // Номер на стълб
        {cout<<A[i][j]<<"\t"; // Извеждане на елемента от матрицата
          S[i] += A[i][j]; // Натрупване на сумата за реда
        }
      cout<<"\t"; // Разстояние между елементите в реда
      cout<<S[i]<<"\n"; // Извеждане на сумата за съотв. ред
    }
  cout<<"\n\n\n";
  Smin=S[0]; // Вземане първата сума за минимална
  for(i=0; i<4; i++)
    if(Smin>S[i]) Smin=S[i]; // Сравняване с останалите суми
    cout<<"Smin= "<<Smin<<"\n"; // Извеждане на минималната сума
  getch();
}

```

**Задача 6.8.** Моделирайте програмно игра за разпознаване на дума, чрез въвеждане на букви от клавиатурата.

**Примерно решение:**

В програмата има **инициализирана дума**, която трябва да познаем по букви. Видима е само **дължината** на думата. След прочитане на поредната **буква** с функцията **getchar()**, се прави **побуквено сравнение** на думата с въведената буква и при съвпадение се замества символа '\_' с **познатата** буква. Процесът продължава докато думата съвпадне с познатата поредица от букви. Тогава се извежда съобщение с броя на опитите за разпознаване на думата.

```

/* Лабораторно упражнение 6. Задача 8. */
#include <iostream.h>
#include <string.h>
#include <stdio.h>
#include <conio.h>

void main()
{char дума[] = "programa"; // Дума за познаване
  char word[] = "_____"; // Празни позиции за букви
  long ch; // Нова буква
  int i,count=0; // Брой опити
  do{cout<<" Познайте зададената дума";
    cout<<"\n"<<word<<"\n";
    cout<<"\n Въведете буква: \n";
    ch = getchar(); // Чете буква

```



```

cout<<"\n";
for (i=0; i<strlen(duma); i++) // Изрежда буквите в думата
    if (ch==duma[i])           // Сравнява ги с въведената буква
        word[i]=ch;           // Поставя познатата буква на мястото
count++;                       // Увеличава брояча на опити
}while(strcmp(word,duma));      // докато съвпадна думите
cout<<"\n"<<word;              // Извежда думата и броя опити
cout<<" Познахте думата от "<<count<<" път - браво!";
getch();
}

```

## Въпроси и задачи за самостоятелна работа

1. Защо елементите на масивите трябва да бъдат от един и същ тип?
2. Възможно ли е дефиниране на едномерен масив без фиксиран размер?
3. По какво се различава индексацията на елементите в масива при програмиране на C/C++ от други езици за програмиране?
4. Какви начини за инициализация на двумерен масив знаете? А как се инициализира двумерен символен масив?
5. Колко начина за въвеждане и извеждане на символни низове са ви известни?
6. Въведете стринг и определете колко пъти се среща в него конкретен символ, въведен от клавиатурата.
7. Определете колко пъти се среща дадена дума в зададен чрез инициализация текст.
8. Да се състави C/C++ програма за подреждане на елементите на едномерен масив A[20] по големина в низходящ ред. Приложете толкова варианти на сортировка, колкото са ви известни.
9. Даден е едномерен масив с "произволен" размер, който се въвежда от клавиатура. Съставете C/C++ програма, която намира максималния елемент и поредния му номер.
10. Съставете C/C++ програма за моделиране на игра ТОТО 6 от 49 чрез генериране на изтеглените в тираж числа и числата на участник в играта (съхранявани и сортирани в съответни масиви). Извежда се "печалбата" в тиража (за улучена тройка/четворка/петица/шестица) и съответен текст. Организирайте запитване за продължение.
11. Съставете C/C++ програма за въвеждане на двумерен масив от цели числа и определяне сумите от елементите само от нечетните редове на масива.
12. Съставете C/C++ програма за намиране произведението на положителните елементи по главния диагонал на матрица с размер 7 на 7.
13. Съставете C/C++ програма за намиране на максималният положителен елемент в горната триъгълна матрица (над главния диагонал) на квадратната матрица A[8][8].
14. Съставете C/C++ програма, която умножава две матрици с реални елементи.

## Тема 7. Указатели. Операции с указатели. Връзка между масиви и указатели. Програми с използване на указатели.

**Цел:** Запознаване с указателите и използваните операции с указатели в **C/C++** програмите. Съставяне, тестване и анализ на програми, представящи използването на указатели.

### Указатели – дефиниране и инициализация

**Указателят** е обект в езика за програмиране **C/C++**, чрез който става достъпен **адреса** на високо ниво.

Най-общо казано, указателят е символично представяне на адрес. В **C/C++** програмите, чрез операцията **&** може да се узнаят и използват **адресите на променливи**.

Например, при въвеждане чрез функцията **scanf("%d", &var)** се използва **&var** – адрес на клетка от паметта, където ще се запамети въведената стойност за променливата **var**. Фактическият адрес на променливата е цяло положително число, а символичното му представяне **&var** е константен указател тъй като адресът на клетката от паметта, отделена за променливата **var**, по време на изпълнение на програмата остава непроменен.

В езика **C/C++** се използват също и **променливи от тип указател**. **Стойността** на такава променлива е **адрес** на друга променлива.

Ако указателят е дефиниран с име **pvar**, то може да се напише:

```
pvar=&var; //На променливата pvar се присвоява адреса на var
```

Казва се, че **pvar** “сочи към” променлива с име **var**.

За да стане адреса на друга променлива с име **variant** стойност на променливата-указател **pvar**, трябва да се напише

```
pvar=&variant; // pvar сочи variant, а не var
```

- **Дефиниране на указател**

**Примери:**

```
int *pi;    // Указател към променлива от цял тип
float *pf;  // Указател към променлива от реален тип
char *pc;   // Указател към променлива от символен тип
```

Спецификацията за **променлива от тип указател** задава **типа** на променливата, към която **сочи** указателят, а символът звезда **\*** определя самата променлива като **указател**.

Указването на типа на **променливите**, към които сочат указателите е необходимо, тъй като различните типове заемат **различен брой байтове**, а за някои операции с указатели трябва да се знае **обема** на отделената памет [5]

- **Инициализация**

Указателят може да се инициализира с **адрес** (цяло положително число) или със специален адрес **0** (символната константа **NULL**, която е дефинирана като **0** в **stdio.h**).

Запомнете, че **указателят трябва да бъде инициализиран** преди използване за да не унищожи полезна информация от паметта.

## Основни операции с указатели

В езика **C/C++** се поддържат **две унарни операции**, които имат приоритет и асоциативност **отдясно наляво** (както другите унарни операции):

- Операцията **адресиране &** се прилага само към обекти от паметта. Резултатът от операцията е **адреса** на указаната **променлива** след оператора **&**.
- Операцията **извличане на стойност \*** оперира със **стойността**, която е запомнена на **адреса**, към който **сочи** променливата от тип **указател**.

Използването на оператор **\*** в **лявата част на оператор за присвояване** позволява на променливата, сочена от указателя, да се зададе стойност.

### Задача 7.1:

```
/* Програма с използване на операциите & и * с указатели */
#include <stdio.h>

void main()
{ int i, j, *pi; // Дефиниране на променливи и указател от цял тип
  i=7;
  pi=&i;          // Указателят pi сочи към i
  j=*pi;
  printf("j=%d \n", j);
}
```

Указателят **pi** се инициализира с адреса на **i**. Операцията **\*pi** извлича стойността, към която сочи указателят **pi** и тя се присвоява на променливата **j** (т.е. **j** приема стойност **7**).

### Задача 7.2:

```
/* C/C++ програма с използване на операции с указатели към
променливи от различен тип */
#include <stdio.h>

void main()
{ /* Дефиниране на променлива i от тип int
   и указател pi от тип "указател към int" */
  int i, *pi;
  /* Дефиниране на променлива f от тип float
   и указател pf от тип "указател към float" */
  float f, *pf;
  /* Дефиниране на променлива c от тип char
   и указател pc от тип "указател към char" */
  char c, *pc;
  pi=&i;    /* pi се инициализира с адреса на i */
  *pi=10;   /* i приема стойност 10 */
  pf=&f;    /* pf се инициализира с адреса на f */
}
```

```

*pf=3.14; /* f приема стойност 3.14 */
pc=&c; /* pc се инициализира с адреса на c */
*pc='A'; /* c приема стойност 'A' */
printf("\t\tПроменливи\t\t\t\t\tУказатели\n");
printf("        Тип\tДължина\t\tСтойност\tДължина\t\tСтойност\n");
printf("        int\t%d\t\t\t%d\t\t\t%d\t\t\t%u\n",
        sizeof(i), *pi, sizeof(pi), pi);
printf("        float\t%d\t\t\t%4.2f\t\t\t%d\t\t\t%u\n",
        sizeof(f), *pf, sizeof(pf), pf);
printf("        char\t%d\t\t\t%c\t\t\t%d\t\t\t%u\n",
        sizeof(c), *pc, sizeof(pc), pc);
pi=NULL; /* Указателят се инициализира със символната константа
        NULL */
printf("\n Сега pi не сочи към i, а към невалиден адрес- %u!\n",
        pi);
}

```

### Примерен резултат:

| Променливи |         |          | Указатели |          |
|------------|---------|----------|-----------|----------|
| Тип        | Дължина | Стойност | Дължина   | Стойност |
| int        | 2       | 10       | 4         | 4094     |
| float      | 4       | 3.14     | 4         | 4086     |
| char       | 1       | A        | 4         | 4081     |

Сега pi не сочи към i, а към невалиден адрес - 0!

Горната програма илюстрира, че в паметта указателят се съхранява по начин, който не зависи от типа на обекта, към който сочи. Операторът **sizeof**, следван от име на обект, дава дължината на обекта в байтове.

Програмата извежда във вид на таблица дължината и стойността на променлива от даден тип както и дължината и стойността на указателя към този тип (съответно за **int**, **float** и **char**). Стойностите на указателите са **адреси – цели числа без знак** и затова при извеждането им се използва форматен спецификатор **%u**. Виждате от примерния резултат, че указателят **pi** има стойност **4094**. Това е адрес на клетка от паметта за променливата **i** със стойност **10**.

### Връзка между масиви и указатели

Тясната връзка между масиви и указатели позволява ефективно да се организира работата с масиви.

**Името** на масива е **указател** и може да се счита за **константен**, тъй като неговата стойност е един определен (начален) **адрес на масива**, който не се променя по време на изпълнение на програмата. Този начален (базов) адрес е адресът на клетката от паметта, отделена за **първия елемент** на масива или **елемент** с индекс **0**.

Например, ако **arri** е име на **масив**, то както **arri** така и **&arri[0]** определят адреса на първия елемент на масива **arri** и е възможно от друга страна този адрес да присвоите като стойност на указател.

### Задача 7.3:

```
/* C/C++ програма за демонстриране на операции с указатели и
връзката между масиви и указатели */
#include <stdio.h>

/* Извежда стойността на указателя(адрес) */
#define PP(P) printf("\n#P" = %u ",P)
/* Извежда стойността, намираща се на този адрес */
#define PV(P) printf("#P" = %d ",*P)
/* Извежда адреса на самия указател */
#define PA(P) printf("&#P" = %u ",&P)

void main()
{ int arri[4]={10,20,30,40}, *pi1, *pi2, index;
  float arrf[4], *pf;
  pi1=arri; // На указателя се присвоява началния адрес на масива
  pf=arrf;

  printf("\n Увеличение на указатели\n\n");
  for(index=0; index<4; index++)
    {printf("Указатели + %d: %u\t",index,pi1+index);
     printf("%u\n",pf+index); }

  printf("\n Операции с указатели\n");
  printf("\n arri + 2 = %u ",arri+2);
  printf("&arri[2] = %u\n",&arri[2]);
  printf("\n *(arri+2) = %d  arri[2] = %d  pi1[2] = %d\n",
         *(arri+2),arri[2],pi1[2]);
  printf("\n *arri+2 = %d\n",*arri+2);

  /* Извежда стойността на указателя(адрес), стойността,
     намираща се на този адрес и адреса на самия указател */
  PP(pi1); PV(pi1); PA(pi1);
  pi1++;
  PP(pi1); PV(pi1); PA(pi1);
  pi2=&arri[3];
  PP(pi2); PV(pi2); PA(pi2);

  /* Разлика между указатели */
  printf("\n\n Броят на елементите между pi2 и pi1 е %d.\n",
         pi2-pi1+1);

  /* Сравняване на два указателя */
  if(pi1<pi2) printf("\n%d се намира преди %d.\n",*pi1,*pi2);
  ++pi2;
  PP(pi2); PV(pi2); PA(pi2);
}
```

### Примерен резултат:

Увеличение на указатели

|                |      |      |
|----------------|------|------|
| Указатели + 0: | 4088 | 4060 |
| Указатели + 1: | 4090 | 4064 |
| Указатели + 2: | 4092 | 4068 |
| Указатели + 3: | 4094 | 4072 |

#### Операции с указатели

```
arri + 2 = 4092 &arri[2] = 4092
*(arri+2) = 30 arri[2] = 30 pi1[2] = 30
*arri+2 = 12
```

```
pi1 = 4088 *pi1 = 10 &pi1 = 4084
pi1 = 4090 *pi1 = 20 &pi1 = 4084
pi2 = 4094 *pi2 = 40 &pi2 = 4080
```

Броят на елементите между pi2 и pi1 включително е 3.  
20 се намира преди 40.  
pi2 = 4096 \*pi2 = 0 &pi2 = 4080

Първият изведен ред показва **началните адреси** на **двата масива**, а следващият – резултати от увеличаването им с **1** и т.н. Знаете, че единицата за адресация е **байт**. В **16 битова** програмна среда тип **int** използва **два байта**, а тип **float** – **четири**. Когато “се добавя единица към указател”, компилаторът добавя **единица памет**. Увеличаването на указател от тип **int** с единица увеличава адреса с **две**, а при указател от тип **float** с **четири**. При **масивите** това означава да се премине към адреса на **следващия елемент**, а не към следващия **байт**. Затова **arri + 2** и **&arri[2]** е един и същ адрес, а **\*(arri+2)** и **arri[2]** е една и съща **стойност** – стойността на третия елемент на масива, докато **\*arri+2 = 12** (към стойността на първия елемент се добавя **2**). Операцията **\*** има по-висок приоритет от оператията **+**. За общия случай **arri + i** сочи към елемента на **i\*sizeof(int)** байта от началото на масива.

**Указателят може да бъде индексирен**, все едно че е дефиниран като **масив**. Както са еквивалентни изразите **\*(arri+2)** и **arri[2]**, така са еквивалентни и изразите **\*(pi1+2)** и **pi1[2]** (първоначално на указателя **pi1** е присвоен началния адрес на масива **arri**). Затова и с **pi1[2]** ще е достъпен **третия** елемент на масива със стойност **30**.

Горната програма демонстрира **основните операции**, които могат да се изпълняват с **променливи** от тип **указател**. За да се покажат резултатите от всяка операция се извеждат стойността на указателя (адрес, към който сочи указателят), стойността, намираща се на този адрес и адреса на самия указател. За тази цел с директивата **#define** на предпроцесора са дефинирани три макроса с аргументи. Когато името на даден параметър се предшества от **#**, комбинацията ще бъде разтеглена до низ в кавички, като параметърът ще бъде заместен с действителния аргумент. При извикване на **PP(pi1)**, макросът се разширява до вида **printf("\n""pi1"" = %u ",pi1)** и низовете се съединяват, като добиват следния вид **printf("\npi1 = %u ",pi1)**.

- **Присвояване**. На указателя се присвоява адрес като се използва име на масив или операцията адресиране (**&** - получаване на адрес). На **pi1** се присвоява адреса на началото на масива **arri** - **4088**; този адрес е стойността, намираща се в клетка от паметта с адрес **4084**.

Променливата **pi2** получава адреса на четвъртия и последен елемент на масива – **arri[3]**.

- **Извличане на стойност.** Операцията **\*** извлича стойността, съхраняваща се на адреса, сочен от указателя. В началото резултатът от **\*pi1** е числото **10**, намиращо се в клетка с адрес **4088**.
- **Адресиране.** Подобно на всяка друга променлива и променливата от тип указател има стойност и адрес. Операцията **&pi1** дава адреса на указателя **pi1** – **4084**. В клетка от паметта с този адрес се съхранява числото **4088**, което е началния адрес на масива **arri**.
- **Увеличаване на указател.** Това действие може да се изпълни с обикновено събиране или с операцията увеличаване. Операцията **pi1++** увеличава числовата стойност на променливата **pi1** с **2** (2 байта за всеки елемент на масив от цели числа), след което **pi1** вече сочи **arri[1]**. Операцията **\*pi1** извлича числото **20**, което е стойността на елемента **arri[1]**. Адресът на указателя **pi1** остава непроменен – **4084**, изменила се е само неговата стойност. По аналогичен начин може да се изпълни и **намаляване на указателя**. Тези операции трябва да се използват внимателно, тъй като не се следи дали указателят още сочи или вече не към елемент на масива. Операцията **++pi2** премества указателя **pi2** на следващите два байта и сега той сочи към някаква информация, която не е елемент на масива.

Операциите увеличаване или намаляване може да се използват с **променливи от тип указател**, но не и с **константи** от такъв тип. Не можем да променяме адреса на **arri** и изрази като

**arri +=1      arri++      ++arri      arri= pi1**

са неправилни.

- **Разлика.** Може да се намери разликата между два указателя, сочещи към елементи на един и същ масив, за да се определи броя на елементите на масива, които се намират между елементите, сочени от указателите (включително) – **pi2-pi1+1**. Резултатът е от типа, които има променливата, указваща размера на масива.
- **Сравняване на два указателя.** Ако указателите сочат към елементи от един и същ масив, тогава релациите като **==, !=, <, <=, >, >=** работят правилно.

#### Задача 7.4:

Да се състави **C/C++** програма, която намира средния успех от изпитите на студент за учебна година. Да се намери максималната оценка и на коя по ред дисциплина е получена като се отчита, че е възможно максималната оценка да не е само по една дисциплина.

```
/* C/C++ програма за намиране на средния успех (avg) от изпитите (n, където 0<n<=10) на студент за една учебна година и на максималната оценка в масива marks, с използване на указатели. */
```

```
#include <stdio.h>
#define NUMBER 10
```

```

void main()
{ int n, i;
  float marks[NUMBER], avg, *pm, *pend, *pmax;
  do
  { printf("\n Въведете броя на оценките:");
    scanf("%d",&n);
  } while (n<1 || n>NUMBER);
  for (i=0; i<n; ++i)
    {printf("\n Въведете оценка за %d-та дисциплина: ",i+1);
     scanf("%f", &marks[i]); }
  pm=marks; // На указателя се присвоява началния адрес на масива
  pend=marks+n; /* Указателят сочи към края на масива */
  for (avg=0; pm<pend; ++pm)
    avg+=*pm;
  avg/=n;
  printf("\n Средният успех е %4.2f.\n",avg);
  /* Намиране на максималната оценка */
  pm=marks;
  pmax=pm;
  for (++pm; pm<pend; ++pm)
    if (*pmax < *pm) pmax=pm;
  /* Намиране и извеждане на всички оценки, равни на максималната */
  pm=pmax;
  for (pm; pm<pend; ++pm)
    if (*pmax==*pm)
      { printf("\n Максималната оценка е =%4.2f",*pm);
        printf(" по %d -та дисциплина.\n",pm-marks+1); }
}

```

Указателят **pmax** се използва за съхраняване адреса на **максималния елемент** от масива. Проверката за намиране на всички оценки, равни на максималната, започва от елемента с максимална стойност. Индексът на максималния елемент се получава от разликата между указателя, сочещ към максималния елемент и началния адрес на масива **pm-marks** (след конвертиране на типа).

## Указатели от тип char

Низовата константа, записана като **"Това е низ"** представлява **масив от символи**. Тъй като във вътрешното представяне края на масива се означава с нулевия символ **'\0'**, то дължината на мястото, заделено в паметта, е с единица по-голяма от броя на символите между кавичките. **Низовете** могат да се съхраняват в **символни масиви** или към тях да се сочи чрез **указатели към символи**.

### Примери:

```

char astr[]="Това е низ"; /* Масив, съдържащ низ */
char *str="Това е низ";   /* Указател към низа */

```

Отделните символи в **масива** могат да се променят, но размера на **astr** ще се запази. При реализацията на втория оператор се заделя достатъчно свободна памет за съхраняването на низа и **адресът на първия символ от низа** се присвоява на **указателя**. Указателят **str**,



инициализиран да сочи към низова константа, впоследствие може да бъде пренасочен **другаде**. Обаче, при опит да се промени съдържанието на низа, резултатът ще бъде неопределен.

### Задача 7.5:

Да се състави програма за копиране съдържанието в обратен ред на символен масив в низ.

```
/* C/C++ програма за копиране в обратен ред на символен масив
str2[] в низ, чрез използване на указатели от тип char и динамично
заделяне на памет (функции от хедърен файл alloc.h). */

#include <stdio.h>
#include <string.h>
#include <alloc.h>

void main()
{ char str2[]="Това е низ", *str1, *s2, *s1;
  int len;
  len=strlen(str2);
  str1=(char*)malloc(len+1);/* Заделяне на необходимата памет */
  s1=str1;                  /* Соци началото на str1 */
  s2=str2+len;
  s2--;                     /* Соци последния символ на str2 преди '\0' */
  while (s2>=str2) // Цикъл за копиране на символите в обратен ред
    *s1++=*s2--;
  *s1='\0';
  printf("\n str1 = %s",str1);
  printf("\n str2 = %s",str2);
}
```

Стойността на **\*s2--** е символа, към който сочи **s2** преди намаляването (постфиксният оператор **--** не променя **s2**, докато символа не бъде взет). По същия начин символът се съхранява в старата позиция на **s1**, преди **s1** да бъде увеличен. Цикълът се изпълнява докато адресът, съхраняван в **s2** е по-голям или равен на началния адрес на **str2**.

### Масиви от указатели

След като **указателите** сами по себе си са променливи, те могат да се съхраняват в **масиви**, както всички други променливи. Например:

```
char *days[7];
```

дефинира **7** указателя, като не ги инициализира; инициализацията може да бъде направена явно – или статично, или посредством кода. Всеки елемент на **days** може да сочи към **символен масив** с определен **брой елементи**. Основното преимущество на **масива от указатели** е, че „редовете на масива” могат да бъдат с **различна дължина**. Иначе казано, всеки елемент на масива може да сочи към различен брой елементи или въобще да не сочи към елементи. **Масивите от указатели** най-често се използват за съхранение на **символни низове с различна дължина** (двумерен масив с различна

дължина на редовете – „назъбен” масив), но това не изключва създаването на масиви от указатели към други типове [5]

### Задача 7.6:

Програма за извеждане дните от седмицата, към които сочат елементите на масив от указатели към низове, в обратен ред.

```
/* C/C++ програма за извеждане дните от седмицата в обратен ред
чрез използване на масив от указатели към низове days[] */
#include <stdio.h>
void main()
{ char *days[]={ "Понеделник", "Вторник", "Сряда",
                  "Четвъртък", "Петък", "Събота", "Неделя" };
  int n=7;
  printf("\n");
  while (--n>=0) /* Цикъл за извеждане на дните от седмицата,
                  в обратен ред */
    printf("%s%s",*(days+n), (n>0) ? ", ":".");
  printf("\n");
}
```

Елементите на масива **days** са **указатели към низове**. Низовете се съхраняват някъде в паметта и действителното им местоположение не е от голямо значение, тъй като всеки указател сочи към **началния** символ на съответния низ. **Всеки низ** заема точно толкова памет, колкото е необходима за него и терминиращата му нула. Когато се извършва дереференция на даден елемент – указател чрез оператора **\***, се осъществява достъп до един от низовете. Броят на елементите се намалява докато стане **0**, като със всяко намаляване се осъществява преместване от последния към първия елемент на масива. Използва се **операция за условие ?** за да се извежда **разделител** между дните – **запетая** и интервал (**n>0**). В противен случай се извежда **точка**.

### Задача 7.7:

Да се състави програма, която чете текстови редове и извежда тези редове, в които се среща даден шаблон. Да се определи колко пъти се среща шаблона.

```
/* C/C++ програма за четене на текстови редове и извеждане само на
редовете, в които се среща даден шаблон. Определя се и се извежда
броя на съвпаденията. Използва се масив от указатели към низове и
динамично заделяне на памет за всеки низ в зависимост от неговата
дължина. */
#include <stdio.h>
#include <string.h>
#include <alloc.h> /* Включва функции за заделяне на памет */
#define MAXLINES 10 /* Максимален брой редове */
#define MAXLEN 60 /* Максимална дължина на входен ред */

void main()
{ /* Масив от указатели към текстови редове */
  char *lineptr[MAXLINES];
```

```

char flag, line[MAXLEN], *ptr, *p, *text;
int len, nlines, c, found, i;
nlines=0; /* Брой прочетени редове от входа */
/* Изграждане на масива от указатели */
printf("\n Максималният брой на въвежданите редове е 10!\n");
printf("Въведете ред от символи (до 60) или ctrl+Z за край\n");
do
{ /* Формиране на текстов ред в line с дължина len */
  for (len=0;len<MAXLEN-1&&(c=getchar())!=EOF && c!='\n';++len)
    line[len]=c;
  if (c=='\n')
    {line[len]=c;
     ++len; }
  line[len]='\0';
  /* Изграждане на елемент на масива от указатели */
  if (len>0)
    { p=(char*)malloc(len); /* Динамично заделяне на памет */
      line[len-1]='\0'; /* Подменя символа за нов ред с '\0' */
      strcpy(p,line); /* Копира line в p */
      /* Стойност за текущия елемент на масива */
      lineptr[nlines++]=p; }
} while ((nlines<MAXLINES) && (c!=EOF));
printf("\n Въведете низ за търсене:\n");
gets(text); /* Въвеждане на шаблон за търсене */
found=0; /* Брой на шаблона в текста */
i=0;
flag='N'; /* флаг за намерен шаблон в текстов ред */
printf("\n Редове, включващи шаблона:\n");

/* Търсене на шаблона и извеждане на редовете, които го съдържат*/
while (i<nlines)
{ ptr=lineptr[i];
  do
  { ptr=strstr(ptr,text);
    if (ptr!=NULL)
      {found++;
       flag='Y';
       ptr++; }
    } while (ptr!=NULL);
  if (flag=='Y') puts(lineptr[i]);
  i++;
  flag='N';
}
if (found==0)
  printf(" Няма\n");
else
  printf("\n Низът се среща %d път(и)!\n", found);
}

```

При формирането на текстовия ред се събират и пазят символите от всеки входен ред като се следи: да не се превишава определената **дължина**, дали е достигнат края на входния **текстови поток** и дали има нов ред. Всеки текстов ред съдържа поне **един символ**; дори ред, съдържащ само един символ за нов ред, има дължина **1**. В края на

създадения масив **line** се поставя **'\0'**, за да се отбележи края на символния низ. Броят се и входните редове, тъй като тази информация ще е нужна при търсенето на шаблона.

При дължина на реда по-голяма от **0**, се заделя **динамично** количество памет, равно на **len** (за динамично **заделяне на памет**, вижте **Тема 12**). Копира се съдържанието на **line** в заделената памет като стойността на указателя, който сочи към този низ става стойност на текущия елемент на **масива от указатели**.

Следва увеличаване на броя на прочетените редове. Формирането на масива от указатели продължава докато се изчерпи определения брой за текстовите редове или се достигне край на входния файл (**EOF**).`

Всеки елемент **lineptr[i]** е указател към първия символ от **i**-тия текстов ред (низ). Функцията **strstr** връща указател към първото срещнато **съвпадение** на шаблона в съответния низ или нулев указател при несъвпадение. Указателят, който сочи съвпадението се увеличава с **1** и търсенето започва отново от този адрес (символа след открития първи символ от предишното търсене). Това продължава до изчерпване на съвпаденията за текущия низ. В случай на **откриване** на поне едно **съвпадение** следва **извеждане** на съответния низ. **Броячът** на претърсените редове се увеличава с **1** и процедурата се повтаря за следващия текстови ред до изчерпването на редовете.

При намерени **съвпадения** се извежда техния **брой** или съобщение, че **няма** такива.

## Въпроси и задачи за самостоятелна работа

1. Каква е основната разлика между указател и обикновена променлива в C/C++ програмите?
2. С какви стойности може да се инициализира указател?
3. Как се извеждат адреса на указател и съхраняваната на този адрес стойност?
4. Какво е предназначението на двата оператора **\*** и **&**? Представете примери.
5. Кои аритметични операции с указатели са възможни? Представете примери.
6. За какво най-често се използват масивите от указатели?
7. Каква грешка е допусната в следния фрагмент от C/C++ програма:  

```
int a, *pa;  
pa=5.2;
```
8. Какви са стойностите **\*ptr** и **\*(ptr+2)** в следния фрагмент от програма:  

```
int *ptr, arr[4]={100, 10};  
ptr=arr;
```
9. Да се състави C/C++ програма, която намира минималната оценка на студент за една учебна година (брой изпити **n**, където  $0 < n \leq 12$ ) и на коя по ред дисциплина е получена като се отчита, че може

минималната оценка да не е само по една дисциплина. Да се изведе броя на минималните оценки.

10. Да се състави C/C++ програма, която сортира оценките на студент за една година в нарастващ/намаляващ ред като се използват указатели.
11. Да се състави C/C++ програма, която да прехвърля положителните и отрицателните елементи на въведена редица от реални числа в други две редици както и да изведе съдържанието им и броя на нулевите елементи от входната редица. Да се състави меню с възможности за избор: въвеждане на данни, обработка, извеждане на резултати и изход от програмата.
12. Да се състави C/C++ програма, която намира средно-аритметично на стойностите на елементите от въведена редица от реални числа, които са в зададен диапазон. Да се изведат намерените стойности като се използва масив от указатели към тях.
13. Да се състави C/C++ програма, която въвежда низ и намира броя на: буквите, цифрите, „интервал” и други символи.
14. Да се състави C/C++ програма, която въвежда низ и създава нов, в който са премахнати последните празни интервали, ако има такива във входа. Да се определи дължината на новия низ без да се използва функцията `strlen()`.
15. Да се състави C/C++ програма, която търси появяване на даден символ в даден низ. Указател трябва да сочи последното появяване на този символ при успешно търсене или стойността му да е `NULL`, ако символът не е намерен. Да се изведе броя на появяванията на символа.
16. Да се състави C/C++ програма, която формира нов низ, съдържащ въведени от клавиатурата име и фамилия. Да не се използват функции за работа със символни низове.
17. Да се състави C/C++ програма, която създава масив от указатели към имената на месеците в годината. Да се извежда името на месеца по въведен номер. Да се организира проверка за валидността на номера на месеца.

## Тема 8. Функции. Предаване на параметри и връщане на резултати. Използване на указатели като параметри.

**Цел:** Съставяне на собствени C/C++ функции. Съставяне, тестване и анализ на програми, включващи потребителски дефинирани функции.

### Функции - дефиниране и извикване. Прототипи на функции.

**Функциите** са “градивните елементи” на езика C/C++. Този стил прави програмите, написани на C/C++ по-лесни за разбиране, тестване, разширяване и поддръжка. Много често, когато една програма трябва да изпълнява задача, състояща се от много **подзадачи**, използването на **функция** за реализация на всяка една от **подзадачите** се явява естествен подход за конструиране на програмата [8].

**Прототипът** на функцията **декларира** функцията преди тя да се използва и преди нейната **дефиниция**. Компиляторът има нужда от прототипа за да може да се извърши правилно обръщение към функцията. Чрез прототипите функциите са “видими” в цялата програма.

**Общият вид** на прототип (деклариране) на функция е:

**[тип\_на\_резултата] име ([списък\_с\_параметри]) ;**

Единствената функция, която не се нуждае от прототип е функцията **main()**. Тя винаги се изпълнява **първа** и обикновено, но не задължително - се описва също **първа**.

**Дефиницията на функция** се състои от **две части**: **заглавна част** и **тяло** на функцията. От елементите в **заглавната част** на функцията само **името (следвано от скоби)** е **задължително**. **Тялото** на всяка функция е заградено във фигурни скоби **{}**.

**“Тип\_на\_резултата”** определя типа на данните, **връщани** от функцията. Ако функцията **не връща стойност**, то типът на резултата е **void** и в този случай може **да не се използва** оператора **return** за връщане на стойност към извикващата функция.

За **връщане на стойност** от извиканата функция, обратно - в извикващата функция, се използва оператора **return** във вида:

**return [израз] ;**

В тялото на една функция може да има няколко оператора **return**, за завършване по различен начин (да връща **различни стойности**) в зависимост от **определени условия** по време на нейното изпълнение.

**Предаването на параметри** (данни) от **извикващата функция** към **извиканата** (извиква се **по име**) е чрез:

- **стойност** (предаване чрез копиране);
- **адрес** (чрез **указател** или чрез **референция** (псевдоним)).

Предаването на **параметри по стойност** се характеризира с това, че действителните стойности (**фактическите параметри**), които се предават към функцията при нейното извикване, **се копират в стека**

(памет с последователен достъп) и **функцията работи със стойностите в стека, а не с оригиналните параметри.**

**Винаги**, когато във функцията се променят **фактическите параметри**, се използва методът с предаване на **параметри (променливи) по адрес (чрез указатели или чрез референция).**

Понятието **обхват на променливите** е много важно при съставянето на функции и е свързано с **“видимостта”** им от функциите. Ако една функция **не трябва** да използва дадена променлива, то тази променлива трябва да бъде **невидима** за функцията и обратно.

Променливите биват **глобални** (дефинирани **вън** от функцията и преди функцията), **локални** (дефинират се **вътре** във функция), автоматични - **auto** (локални променливи, които губят своята стойност след излизане от функцията) и статични – **static** (**запазват** своята стойност след излизане от функцията).

**Всяка функция в C/C++ програма може да извиква която и да е друга функция** (включително и **сама себе си** – т. нар. **рекурсия**).

В следващата примерна програма се демонстрира предаването на параметри към функциите, съответно – **по стойност**, **по референция** (чрез псевдоним) и чрез **указатели**.

**Пример:** Да се състави **C/C++** програма с **три** функции **PassByValue**, **Reference** и **Pointers** (с предаване на параметри съответно **по стойност**, **по референция** (адрес) и чрез **указател**). Всяка една от тези функции получава от **main()** **два** параметъра: брой ябълки (**int apples**) и брой круши (**int pears**). Всяка от функциите променя стойностите на формалните параметри, изчислява сумата от всички плодове и връща изчислената стойност в **main()**. **Сумата (int sum)** се извежда в **main()**. Наблюдавайте и сравнете стойностите на **фактическите** параметри (стойностите на променливите **apples** и **pears** в **main()**) със стойностите на **формалните** параметри (стойностите на променливите **apples** и **pears** в дефинираните **потребителски функции**) [9 – 12].

```
/* Примерна програма с предаване на аргументи – по стойност, по
адрес (референция) и чрез указател */
#include <iostream.h>
//Прототипи на функции PassByValue, Reference, Pointers
int PassByValue(int fru1, int fru2);
int Reference(int &fr1, int &fr2);
void Pointers(int *u, int *v, int *s);

void main()
{int apples = 30, pears = 15; int sum;
  cout << "\n При стартиране, в main()";
  cout << "\n\t Брой ябълки = " << apples;
  cout << "\n\t Брой круши = " << pears;
  cout << "\n Предаване на параметри по стойност=Копиране";
  sum = PassByValue(apples, pears); // Извикване на PassByValue()
  cout << "\n\t Общо плодове в main(), sum = "<<sum;
  cout << "\n След извикване на PassByValue(), в main()";
  cout << "\n\t Брой ябълки = " << apples;
```

```

cout << "\n\t Брой круши = " << pears;
cout << "\n\t Общо плодове = " << apples+pears;
    cout << "\n Предаване на параметри – по референция";
sum = Reference(apples, pears); // Извикване на Reference()
cout << "\n\t Общо плодове в main(), sum = " << sum;
cout << "\n След извикване на Reference(), в main()";
cout << "\n\t Брой ябълки = " << apples;
cout << "\n\t Брой круши = " << pears;
cout << "\n\t Общо плодове = " << apples+pears;
    cout << "\n Предаване на параметри – чрез указател";
Pointers(&apples, &pears, &sum); // Извикване на Pointers()
cout<<"\n\t Общо плодове в main, sum = " << sum;
cout << "\n След извикване на Pointers(), в main()";
cout << "\n\t Брой ябълки = " << apples;
cout << "\n\t Брой плодове = " << pears;
cout << "\n\t Общо плодове = " << apples+pears;
}

```

```

int PassByValue(int b, int g)
{
    b += 3, g += 4;
    cout << "\n В PassByValue(), има:";
    cout << "\n\t Ябълки = " << b;
    cout << "\n\t Круши = " << g;
    return b+g;
}

```

```

int Reference(int &b, int &g)
{
    b = b + 8, g = g + 5;
    cout << "\n В Reference(), има:";
    cout << "\n\t Ябълки = " << b;
    cout << "\n\t Круши = " << g;
    return b+g;
}

```

```

void Pointers(int *b, int *g, int *sum_fru)
{
    *b = 44, *g = 52; *sum_fru=0;
    cout << "\n В Pointers(), има:";
    cout << "\n\t Ябълки = " << *b;
    cout << "\n\t Круши = " << *g;
    (*sum_fru)=(*b)+(*g);
}

```

### **Примерно изпълнение:**

При стартиране, в main()

Брой ябълки = 30

Брой круши = 15

Предаване на параметри по стойност=Копиране

В PassByValue(), има:

Ябълки = 33

Круши = 19

Общо плодове в main(), sum = 52

След извикване на PassByValue(), в main()

Брой ябълки = 30

Брой круши = 15

Общо плодове = 45



Предаване на параметри – по референция

В `Reference()`, има:

Ябълки = 38

Круши = 20

Общо плодове в `main()`, `sum` = 58

След извикване на `Reference()`, в `main()`:

Брой ябълки = 38

Брой круши = 20

Общо плодове = 58

Предаване на параметри – чрез указател

В `Pointers()`, има:

Ябълки = 44

Круши = 52

Общо плодове в `main`, `sum` = 96

След извикване на `Pointers()`, в `main()`

Брой ябълки = 44

Брой плодове = 52

Общо плодове = 96

**Коментар:** Обърнете внимание на факта, че **фактическите** параметри се **съгласуват с формалните** параметри по **брой, тип и ред на следване** в съответните списъци.

Извикването на първите две функции `PassByValue()` и `Reference()` става като се включат в **израз** (като операнд). Фактическите параметри и двата случая са идентични: това са целите променливи **apples** и **pears**. Обърнете обаче внимание, на начина по който са описани съответстващите формални параметри в заглавния ред на функция `Reference()` – като адреси (**`int &b`, `int &g`**). Докато при извикване на функция `PassByValue()` става предаване на фактическите параметри **по стойност**, то при извикване на `Reference()` става предаване **по адрес**.

При извикване на функция `Pointers()` **фактически параметри са адресите на променливите** от цял тип: **apples**, **pears** и **sum**. **Формалните параметри** на функцията се декларират като **указатели**; в тялото на функцията се работи със **съдържанията на указателите** (стойностите на **променливите, сочени от тези указатели**). В общия случай, когато **формалните параметри** са указатели, **фактическите параметри са адреси** (както в случая) или **указатели**.

Обърнете внимание, че **функция `Pointers()`** се извиква **по различен начин**, от първите две. Извикването на функция по този начин е възможно **само**, ако тя **не връща с `return` резултат в извикващата**.

**Примерни задачи и програми с използване на потребителски дефинирани функции и меню техника**

**Задача 8.1.** Да се състави **C/C++** програма, която да съдържа **3** създадени от потребителя функции: **`GetName()`**, **`GetHours()`** и **`GetGrade()`**, чието предназначение е описано в заглавния коментар.

**/\* C/C++ програма с функции за:**

**- `GetName()`** – сливане (конкатениране) първото и второ име на студент, с цел – получаване на пълно име `FullName` на студента;

- GetHours() - изчисляване и връщане в главната функция отработените от студента учебни часове за текущата седмица;
- GetGrade() изчисляване и връщане в главната функция средния успех на студента.

Демонстрира се предаване на аргументи по стойност \*/

```
#include <iostream.h>
#include <string.h>
char FullName[30];          // Пълно име на студент

void GetName()
{
    char FirstName[10], LastName[10];
    cout << "\n Въведете име: ";
    cin >> FirstName;
    cout << " Въведете фамилия: ";
    cin >> LastName;
    strcpy(FullName, FirstName);
    strcat(FullName, " ");
    strcat(FullName, LastName);
}

double GetGrade()
{
    double ex1, ex2, ex3, ex4, TotalGrade;
    cout << endl << FullName << " има изпитни резултати:\n";
    cout << "Изпит 1: ";
    cin >> ex1;
    cout << "Изпит 2: ";
    cin >> ex2;
    cout << "Изпит 3: ";
    cin >> ex3;
    cout << "Изпит 4: ";
    cin >> ex4;
    TotalGrade = (ex1+ex2+ex3+ex4)/4;
    return TotalGrade;
}

void main()
{
    double Hours, grade;
    double GetHours(); // Прототип на GetHours(), описана по-долу
    GetName();         // Извикване на GetName()
    Hours = GetHours(); /* Извиква се GetHours(), с return резултатът
                        се връща тук */
    cout << "\n Студент с име " << FullName;
    cout << "\nотработил " << Hours << " часа за тази седмица.\n\n";
    grade=GetGrade(); // Извикване на GetGrade()
    cout << "\n Студент с име " << FullName;
    cout << "\nима среден успех " << grade << " за този семестър.\n\n";
}

double GetHours()
{
    double Mon, Tue, Wed, Thu, Fri, TotalHours;
    cout << endl << FullName << " има да отработва следните часове в:\n";
    cout << "Понеделник: "; cin >> Mon;
    cout << "Вторник: "; cin >> Tue;
    cout << "Сряда: "; cin >> Wed;
```

```

cout << "Четвъртък: "; cin >> Thu;
cout << "Петък: "; cin >> Fri;
TotalHours = Mon + Tue + Wed + Thu + Fri;
return TotalHours;
}

```

### Примерно изпълнение:

Въведете: Иван  
 Въведете фамилия: Иванов  
 Иван Иванов има да отработва следните часове в:  
 Понеделник: 3  
 Вторник: 5  
 Сряда: 3  
 Четвъртък: 4  
 Петък: 6  
 Студент с име Иван Иванов  
 е отработил 21 часа за тази седмица.  
 Иван Иванов има следните изпитни резултати:  
 Изпит 1: 3  
 Изпит 2: 4  
 Изпит 3: 5  
 Изпит 4: 3  
 Студент с име Иван Иванов  
 има среден успех 3.75 за този семестър.

**Задача 8.2.** Да се състави C/C++ програма, която да въвежда списък - двумерен масив float A[rows][columns], всеки ред „rows” на който съхранява „columns” оценки за даден студент. Нека за опростяване на задачата rows = 3, а columns = 4. Програмата да съдържа функция FindScore, която изчислява и съхранява в едномерен масив (float G[rows]) средния успех на всеки студент. Да се реализират и функции - ReadMatrix: за въвеждане стойностите на двумерния масив от клавиатурата, WriteMatrix: за извеждане на списъка от оценки и средния успех на всеки студент.

```

/* C/C++ програма, с функции за:
- въвеждане на двумерен масив float A[rows][columns] с оценки,
където броя на редовете rows указва броя на студентите, а броя на
колоните columns - броя на оценките за всеки студент;
- намиране средния успех на всеки студент - създаване на едномерен
масив float G[rows];
- отпечатване на масива с оценки и средния успех на всеки студент
(стойностите на двумерния и едномерния масиви). */
#include <iostream.h>
#include <iomanip.h>
#include <conio.h>
const int rows = 3;    //Брой на редовете в масива=брой студенти
const int columns = 4; //Брой на колоните на масива=брой оценки за
                        всеки студент */

void ReadMatrix (float ma[rows][columns])
// функция за въвеждане на масива от оценки
{ int r, c;
  cout << "\n Въвеждане на оценки на студенти от 2 до 6)";

```

```

for (r=0; r<rows; r++)
{cout << "\n\n-----";
  cout << "\n Данни за студент " << (r+1) << ": ";
  for (c=0; c<columns; c++)
  {do
    {cout << "\n Въведете оценка " << (c+1) << ": ";
      cin >> ma[r][c];
    }while (ma[r][c]<2 || ma[r][c]>6);
  }
}

void WriteMatrix (float ma[rows][columns],float grade[rows])
// Функция за извеждане на списъка с оценки на екрана
// За всеки студент се извежда среден успех
{ int r, c;
  cout << "\n\n===== ";
  cout << "\n Списъкът е:";
  for (r=0; r<rows; r++)
  {cout << "\n";
    for (c=0; c<columns; c++)
      cout << setw(5) << ma[r][c] << " ";
    cout << " Ср.Успех= "<<setw(5) << grade[r];
  }
}

void FindScore (float sq[rows][columns], float grade[rows])
//Функция за намиране на средния успех
{ int i, j;
  for (i=0; i<rows; i++)
  {grade[i]=0;
    for (j=0; j<columns; j++)
      grade[i] += sq[i][j];
    grade[i]=grade[i]/columns;
  }
}

void main ()
{ float A[rows][columns], G[rows]; // Дефиниране на масивите
  ReadMatrix(A); // Въвеждане на масива с оценки
// Изчисляване на успеха за всеки студент, масив G
  FindScore (A,G);
  cout << "\n\n Успехът е изчислен! ";
// Извеждане на списъка с оценки на екрана
// За всеки студент се извежда среден успех
  WriteMatrix(A,G);
  getch();
}

```

### Примерно изпълнение:

Въвеждане на оценки на студенти от 2 до 6

```

-----
Данни за студент 1:
Въведете оценка 1: 2
Въведете оценка 2: 3

```

Въведете оценка 3: 4  
Въведете оценка 4: 5

-----  
Данни за студент 2:  
Въведете оценка 1: 5  
Въведете оценка 2: 3  
Въведете оценка 3: 2  
Въведете оценка 4: 5

-----  
Данни за студент 3:  
Въведете оценка 1: 5  
Въведете оценка 2: 4  
Въведете оценка 3: 4  
Въведете оценка 4: 3

Успехът е изчислен!

=====;

Списъкът е:

|   |   |   |   |           |      |
|---|---|---|---|-----------|------|
| 2 | 3 | 4 | 5 | Ср.успех: | 3.5  |
| 5 | 3 | 2 | 5 | Ср.успех: | 3.75 |
| 5 | 4 | 4 | 3 | Ср.успех: | 4    |

**Задача 8.3.** Да се състави C/C++ програма за моделиране на играта "Камък, ножици или хартия". В тази игра, играчът и компютърът избират един от 3 предмета: камък – 's', ножици – 'x' или хартия – 'p'. Валидни са следните правила: камъкът побеждава ножиците, ножиците побеждават хартията, хартията побеждава камъка. Всяка победа носи точка. Първият, който натрупа MAX\_POINT точки побеждава. Да се реализират следните функции: Computer\_choice() - за генериране избора на компютъра: връща случаен символ 's', 'x', и 'p' и WhoWins() за сравняване на избора, направен от потребителя с този – на компютъра. Втората функция връща: 0 при равенство, 1 - ако печели потребителя, 2 – ако печели компютъра, -1 – при грешен символ [9 -12].

/\* "Камък, ножици или хартия". В тази игра, играчът и компютърът трябва да изберат един от тези 3 предмета (камък – 's', ножици – 'x' или хартия – 'p'). \*/

```
#include <iostream.h>
```

```
#include <stdlib.h>    // За rand()
```

```
#include <time.h>      // За функция time (NULL)
```

```
#define MAX_POINT 3    // Играе се до MAX_POINT точки
```

```
// * Връща случаен символ 's', 'x', и 'p'
```

```
char Computer_choice (void)
```

```
{ char option;
```

```
    srand (time (NULL) );    // (Ре)инициализация на генератора
```

```
    int value = rand()%3;    // Генерира цяло число между 0 и 2
```

```
    switch (value) {
```

```
        case 0: option='s'; break;
```

```
        case 1: option='x'; break;
```

```
        case 2: option='p'; break;
```

```
    }
```

```

    return option;
}

/* WhoWins. Проверява се, кой печели. Връща: 0=(наравно), 1=
(първия), 2=(втория), -1=(грешка) */
int WhoWins (char a, char b)
{ switch (a)
    { case 's':
        if (b=='x') return 1;
        else if (b=='p') return 2;
        else return 0;
      case 'x':
        if (b=='p') return 1;
        else if (b=='s') return 2;
        else return 0;
      case 'p':
        if (b=='s') return 1;
        else if (b=='x') return 2;
        else return 0;
      default:
        return -1;
    }
}

/* Забележете, че няма break в switch оператора, тъй като се
използва return (никога няма да бъде достигнат). По същата причина
- няма изричен return оператор в края на функцията.*/
}

void main ()
{ char you, me;      // you - потребител, me - компютър
  int mypoints=0;    // Точки на компютъра
  int yourpoints=0;  // Точки на потребителя
  int rezult;        // Резултат от сравнението на 2-та отговора
  do{                // Подканя се потребителя да направи своя избор.
    cout << "\n Изберете s, x or p ";
    cout << "(s=камък, x=ножици, p=хартия): ";
    cin >> you;      // Избор на потребителя
    // me - случаен избор на компютъра
    me = Computer_choice();
    cout << " Аз казвам: " << me << "\n";
    // Проверява се резултата:0, 1, 2
    rezult = WhoWins (you,me);
    // Извежда се съобщение:
    if (rezult==0) cout << " Равенство\n";
    else if (rezult==1)
    { cout << " Ти си победител\n"; yourpoints++; }
    else if (rezult==2)
    { cout << " Аз съм победител\n"; mypoints++; }
    else
    cout << " Съжалявам. Въвел си невалиден символ!\n";
    // Показва се текущия сбор точки.
    cout << " Точки за теб:" << yourpoints;
    cout << " Точки за мен:" << mypoints << "\n";
  }while (yourpoints<MAX_POINT && mypoints<MAX_POINT);
  if (yourpoints>mypoints) cout << " Ти печелиш състезанието!\n";
}

```

```

    else cout << " Аз печеля състезанието!\n";
}

```

**Коментар:** Функцията `time(NULL)` от файла `time.h` се използва като аргумент на `srand()`. Функцията `srand()` се използва веднъж в началото на `main()`, за да бъде генерираната последователност от числа с `rand()`, различна при всяко стартиране на програмата.

Функцията `rand()` връща случайно цяло число, в интервала от 0 до `RAND_MAX`, където `RAND_MAX` е цяла константа (`RAND_MAX=32767`), дефинирана във файл `stdlib.h`. Изразът `rand()%3` ще даде остатък от делението на `rand()` на 3, тоест цяло число, в интервала 0 до 2.

**Задача 8.4.** Условието на задачата е представено в заглавния коментар на примерната програма.

```

/* C/C++ програма, която използва функция menu(), предлагаща на
потребителя да избере: кръг, квадрат, правоъгълник или триъгълник.
Да се използват отделни функции, в които да се въвеждат входните
данни (според избраната фигура) и се изчислява и извежда лицето на
фигурата. */
// Намиране на площта на различни фигури
#include <iostream.h>
char menu();           // Извежда меню
void Circle_area();    // Въвежда радиус, отпечатва площта
void Rectangle_square_area(char ch);
// За правоъгълник - въвежда дължина, ширина, отпечатва площта
// За квадрат - въвежда страна, отпечатва площта
void Triangle_area(); // Лице =(основа*височина)/2

void main()            // Извежда меню
{char choice;
  do
  {choice=menu();      // Извежда меню
   switch (choice)
   {case 'C': case 'c': Circle_area(); break;
    case 'T': case 't': Triangle_area(); break;
    case 'R': case 'r':
    case 'S': case 's': Rectangle_square_area(choice); break;
    case 'Q': case 'q': break; // Изход
    default: cout<<" Невалиден избор, опитайте пак. \n";
   } // switch choice
  }while (choice !='Q' && choice !='q');
}

char menu()
{ char ch;
  cout<<"\n Моля, изберете фигура: \n";
  cout<<" C: Кръг \n";
  cout<<" R: Правоъгълник \n";
  cout<<" S: Квадрат \n";
  cout<<" T: Триъгълник \n";
  cout<<" Q: Изход \n\n";
  cout<<" Вашият избор е: ";
  cin>>ch;
}

```

```

return ch;
} // Край на menu()

void Circle_area() // Въвежда радиус, отпечатва площта
{ int radius;
  cout<<" Въведете радиуса на кръга: ";
  cin>>radius;
  cout<<" Площта на кръга е "<<3.14*radius*radius<<"\n";
} // Край на Circle_area()

void Rectangle_square_area(char ch)
// За правоъгълник - въвежда дължина, ширина,отпечатва площта
// За квадрат - въвежда страна,отпечатва площта
{int length, width; float side;
  switch (ch)
  {case 'R':case 'r':
    cout<<" Въведете дължина на правоъгълника: ";
    cin>>length;
    cout<<" Въведете ширина на правоъгълника: ";
    cin>>width;
    cout<<" Площта на правоъгълника е "<<length*width<<"\n";
    break;
    case 'S': case 's':
      cout<<" Въведете страна на квадрата:";
      cin>>side;
      cout<<" Площта на квадрата е "<<side*side<<"\n";
    }
} // Край на Rectangle_square_area()

void Triangle_area() // Въвежда страна, височина, изчислява лице
{float side, height;
  cout<<" Въведете страна на триъгълника:";
  cin>>side;
  cout<<" Въведете височината, към тази страна:";
  cin>>height;
  cout<<" Площта на триъгълника е "<<(side* height)/2<<"\n";
} // Край на Triangle_area()

```

**Задача 8.5.** Да се състави C/C++ програма, демонстрираща предаването на **структури** към функции, тоест използването на структури за **параметри на функция**. Декларираната примерна структура **student** описва студентски данни в полета (елементи на структурата): **name** (за името на студент), **scholarship** (за стипендия) и **grade** (за успеха на студента). Да се реализира функция **scholarship\_student()**, която да изчислява стипендията, която студента ще получи в зависимост от успеха си: ако успехът на студента е по-голям от **4.50**, стипендията му е **50** лева, в противен случай – студентът не получава стипендия. Да се реализират два варианта на предаване, а именно: чрез копиране на стойности на структурата в параметъра на функция и предаване чрез указател.



**Забележка:** За допълнителна информация относно структурите, като потребителски дефиниран тип данни, вижте **Тема 10**.

/\* С/С++ програма, демонстрираща предаване на структури към функции.

Вариант 1: Чрез копиране на на структурата в параметъра на функция. Това е неефективно, ако структурата е с голям размер. \*/

```
#include <iostream.h>
#include <string.h>
#include <conio.h>

struct student          // Декларира се структура student
{   char name[32];       // Име и фамилия
    float scholarship;   // Стипендия
    float grade;         // Успех
};

student scholarship_student (student t) // функция за стипендия
{   if (t.grade > 4.5)
    {   t.scholarship = 50.00;
        return t;
    }

void main()
{   student person;
    cout<<"\n Въведете име на студент: "; cin>>person.name;
    cout<<"\n Въведете успех на студент: "; cin>>person.grade;
    person.scholarship=0.0;           // Студентът няма стипендия
    person = scholarship_student(person);
    cout << person.name << " получава стипендия " <<
        person.scholarship<<" лв.";
    getch();
}

/* Лабораторно упражнение 8. Задача 5.*/
/* Вариант 2: Предаване чрез указател. */
.....
void scholarship_student (student* t) // функция за стипендия
{   if (t->grade > 4.5)
    {   t->scholarship = 50.00;
    }

void main()
{   student person;
    cout<<"\n Въведете име на студент: "; cin>>person.name;
    cout<<"\n Въведете успех на студент: "; cin>>person.grade;
    person.scholarship=0.0;           // Студентът няма стипендия
    .....
    scholarship_student(&person);
    cout << person.name << " получава стипендия " <<
        person.scholarship<<" лв.";
    getch();
}
```

### **Примерно изпълнение:**

Въведете име на студент: Иван Йорданов

Въведете успех на студент: 4.77

Иван Йорданов получава стипендия 50 лв.

### **Въпроси и задачи за самостоятелна работа**

1. Каква е разликата между глобални и локални променливи? Какво е характерно за локалните променливи и защо е препоръчително използването им?
2. Какво е предназначението на прототипите (декларациите) на функциите в C/C++ програмите?
3. Какво е предназначението на оператор return? Вярно ли е, че една функция трябва да има винаги оператор return като последен оператор?
4. В какви случаи функция в C/C++ програма се дефинира от тип void?
5. Кое е името на функцията, която се изпълнява първа в една C/C++ програма?
6. Какви са изискванията за съответствие между формалните и фактическите параметри?
7. Какви начини за предаване на параметри между функции могат да се използват в C/C++ програмите и какво е характерно за всеки от тях?
8. Напишете функция main() и втора функция, която се извиква от main(). Нека в main() потребителят да въвежда своя годишен доход. Стойността на дохода да се предава към втората функция, която да класифицира потребителя съобразно дохода на беден, среден и богат.
9. Съставете функция, която да получава 2 целочислени стойности. Функцията трябва да връща стойността на по-малкото число.
10. Съставете функция, която да получава целочислена стойност. Ако тази стойност е положителна, то във викащата функция да се връща сумата на числата от 1 до въведената стойност. В противен случай – да се връща самото число.
11. Съставете C/C++ програма, която да изчислява и извежда квадрата на целите числа от 0 до n ( $n \leq 100$ ). Намирането на квадрата да се реализира във функции `int square (int x)`.
12. Съставете C/C++ програма – конвертор, от квадратни сантиметри в квадратни ярдове. Конвертирането да се извърши чрез функция `double sqmeters_to_sqyards (double sqmeters)`.
13. Съставете C/C++ програма, която да генерира цяло число (от 1 до 100) – чрез функция `int random_int()`. Програмата да дава възможност на потребителя да въвежда число – във функция `int get_int()`, с цел познаване на генерираното чрез функцията `random_int(число)`. Броят на опитите за познаването на генерираното число не надхвърля 6. Да се извеждат подсказващи текстове, водещи процеса на отгатване на числото.

## Тема 9. Рекурсия. Съставяне и анализ на програми с рекурсия. Модулна организация на C/C++ програми.

**Цел:** Запознаване с особеностите и използването на рекурсивни C/C++ функции, както и с проектирането на по-сложни програми с модулна организация и "скриване на данните" (data hiding).

### Съставяне и анализ на програми с рекурсия

В C/C++ програмите е възможно, а в някои случаи и препоръчително, използването на т. нар. **рекурсивни** функции.

Една функция е **рекурсивна** когато в тялото си извиква сама себе си - така наречената **пряка** рекурсия.

Възможна е и **непряка** (косвена или взаимна) рекурсия, при която една функция извиква друга функция, а тази друга функция на свой ред извиква първата. Използва се сравнително по-рядко. Възможна е **взаимна** рекурсия и между повече от **две** функции.

#### 1. Функции с пряка рекурсия.

От предходната тема знаете, че когато една **функция** бива **извиквана**, се **заделя** съответно място за параметрите и локалните ѝ променливи в стековата памет.

По подобен начин, когато дадена **функция** се извиква **рекурсивно**, всеки път при последователните рекурсивни извиквания тя започва с **нова група от параметри и локални променливи**, съхранявани на съответни места (адреси) в **стековата памет** [1].

Следователно, всяка **рекурсивна функция** трябва да съдържа **оператор с условие**, определящо дали функцията да се извика отново или да върне резултата от изпълнението си. Без подобно условие, рекурсивната функция ще се изпълнява, докато **запълни** заделената за **стек памет**, и след това ще **блокира програмата**!

Освен това **запомнете**, че **кодът** дефиниращ функцията, **остава един и същ** (т.е. не съществуват множество **копия** на рекурсивната функция) [4].

Следва **примерна** програма с една от най-често демонстрираните рекурсивни функции за пресмятане на **факториел  $n!$** , която е **рекурсивна по определение**: за  $n = 0$ ,  $0! = 1$ , а за всяко следващо  $n$  се използва формулата  $n! = n \cdot (n-1)!$ .

Функцията е с име **facr**, за да "подсказва", че се използва рекурсивен вариант за извикване [1].

#### Примерна програма 1:

```
#include <iostream.h>
#include <conio.h>
#include <dos.h>
#define N 12

long unsigned facr(int n)    // Рекурсивна функция за n!
```

```

{ if(n==0)
    return 1;
else
    {cout << "\n n = " << n; // Извежда n - "прав" ход на рекурсията
      cout << " Адрес на n = " << &n; // Текущ адрес на n в стека
      delay(1000); // Закъснение за показване на n и адреса
      return n*facr(n-1); // Рекурсивно извикване на функцията
    }
}

void main()
{int k; // За фактически параметър на функцията
char otg; // За запитване за ново пресмятане
clrscr();
do{
    do{cout << "\n\n\t Введете стойност за n <= " << N << ": ";
       cin >> k;
       }while(k<0 || k>N);
    cout << "\n\n\t Факториел от " << k << " е " << facr(k) << endl;
    cout << "\n Желаете ли да продължите? За отказ въведете N/n. ";
    otg = getche();
    }while(!(otg=='N' || otg=='n' || otg=='H' || otg=='h'));
getch();
}

```

Както разбирате от условието за пресмятане на  $n!$  и от програмата, в общия случай се извършва "прав" и "обратен" ход за рекурсията (в случая - до  $1!$  и обратно - до  $n!$ ) със съхраняване на всички стойности на параметрите и локалните променливи на рекурсивната функция.

За да се убедите в това, можете да наберете програмата и да я изпълнете по стъпки (натискане на клавиш **F7** за всяка стъпка), като във функцията **main()** извикате **facr()** с параметър **2**. При извикването ѝ, в стека ще бъде заделена памет за параметъра **n** и ще се съхрани стойността **2**. Тъй като условието на оператора **if** не е изпълнено (**n** е **2**), ще се достигне до изпълнение на оператора **return n\*facr(n-1);** (след **else**), при което трябва да се пресметне израза **n\*facr(n-1)**. Следователно отново ще бъде извикана, но с параметър, чиято стойност е **n-1**, т.е. **1**. При това второ извикване на функцията, в стека ще бъде заделена нова памет за нейния параметър, различна от заделената при първото извикване (параметърът **n** от първото извикване на функцията запазва своята стойност, която е **2**).

За да видите това, преди всяко рекурсивно извикване на функцията **facr()**, в програмата се извежда стойността на **n** и текущия адрес (в шестнадесетичен код) в стека (по време на "правия" ход за рекурсията).

При второто извикване на функцията условието на оператора **if** ще бъде изпълнено, тъй като **n** има стойност **1**. Следователно ще бъде изпълнен оператора **return 1;**, с което ще завърши изпълнението на второто извикване на функцията **facr()**.

Тогава ще продължи изпълнението на оператора **return** от първото извикване на функцията ("обратен" ход на рекурсията). Върнатата

стойност от второто извикване на функцията **facr()**, която е **1**, ще бъде **умножена** по стойността на **n** от първото извикване, която е **2** и полученият резултат (със стойност **2**) ще бъде резултатът от извикването на функцията **facr(2)**. Изпълнението на тази функция приключва и управлението се връща на **main()**.

Използвайте рекурсивните функции много внимателно. Обикновено всяка рекурсивна функция има **итеративен** (цикличен) **вариант** без рекурсия.

Следва **примерен** вариант на програма за пресмятане стойността на факториел **n!** (**n <= 1700**) чрез **итерационна** функция (циклично пресмятане по формулата **fac = fac\*i**, за **i** от **1** до **n**). За максимално разширяване на обхвата на допустимите стойности на **n**, функцията е дефинирана от тип **long double** [1].

#### Примерна програма 2:

```
#include <iostream.h>
#include <conio.h>
#define N 1750                                // Максимална стойност за N!

long double fac(int n)                        // Итерационна функция за n!
{ int i;
  long double fac=1;
  for(i=1; i<=n; ++i)
    fac *= i;
  return fac;
}

void main()
{int k;
 char otg;
 clrscr();
 do{
   do{cout << "\n\n\t Введете стойност за n <= " << N << ": ";
     cin >> k;
   }while(k<0 || k>N);
   cout << "\t Факториел от " << k << " е " << fac(k) << endl;
   cout << "\n Желаете ли да продължите? За отказ въведете N/n. ";
   otg = getche();
 }while(!(otg=='N' || otg=='n' || otg=='H' || otg=='h'));
 getch();
}
```

Рекурсията може да бъде доста полезна в опростяването на някои типове **алгоритми** като например за **бързо сортиране** (метод **Quicksort**), за работа с **дървовидни структури** от данни, за работа със структури данни от тип **граф** и други [7].

### Модулна организация на C/C++ програми

При по-сложни програмни **проекти** една **C/C++** програма може да бъде разположена в **няколко файла**, които се компилират **независимо** един от друг, т.е. осъществява се т. нар. **разделна компилация**. В резултат на **компилацията** се получават няколко **обектни модула**

(файлове с разширение **.obj**). **Изпълнимият код** на програмата (**.exe** файл) се получава след **свързване (linking)** на обектните модули [4].

**Преимуществото на разделната компилация** се състои в това, че при промени в някои от файловете **не е необходимо** да прекомпилирате останалите. В резултат на това се съкращава времето за **компиляция** и съответно времето за **проектиране** на програмата.

Когато една програма се състои от **множество файлове**, възникват ситуации, при които **глобални** (т. нар. **външни**) променливи, дефинирани в **един файл**, трябва да бъдат достъпни за **функции**, дефинирани в **други файлове** (функциите ползват **общи данни**). За **достъп до тези данни**, съответните **глобални променливи** се дефинират **еднократно** в някой от файловете, а за осигуряване на достъп в съответните **други файлове** те се декларираат с **ключовата дума extern** (от **external**, което означава **външен**).

По подобен начин трябва да се **декларираат** и **функциите**, като за целта се използват познатите ви от предходната тема **прототипи на функциите**. Но при това деклариране на функции за използването им в други файлове ключовата дума **extern** **не е необходима**.

В случаите когато **достъпът до глобални променливи** или **функции** трябва да бъде **ограничен** само в областта на "**видимост**", от мястото им на дефиниране до края на съответен **файл**, дефиницията им трябва да се предхожда от ключовата дума **static** (**статични функции** и **статични глобални променливи**).

Чрез **статичните глобални променливи** и **функции** може да се реализира идеята за "**скриване на данните**" (**data hiding**), която е в основата на **модулното програмиране**.

Примерна програма с модулна организация е показана по-долу.

## **Примерни задачи и програми с рекурсивни функции и с модулна организация**

**Задача 9.1.** Условието на задачата [3] е представено в заглавния коментар на представената по-долу примерна програма.

```
/* Лабораторно упражнение 9. Задача 1. */
/* C/C++ програма за пресмятане чрез рекурсивна функция на
биномните коефициенти C(n,k) от n-ти ред (n<=13), които за k = 0,
1, 2, ... n, се получават по формулата: C(n,k)=1, за k=0 или k=n;
иначе C(n,k)= C(n-1,k-1) + C(n-1,k).
Коефициентите да се изведат на екрана във вид на триъгълник. */
#include <stdio.h>
#include <conio.h>

int bin_k(int n, int k);          // Прототип на рекурсивната функция

void main()
{int n;
 clrscr();
 do{printf("\n Въведете n <= 13: ");
```

```

    scanf("%d", &n);
}while(n < 0 || n > 13);
printf("\n Биномни коефициенти: \n");
for(int n1=0; n1<=n; n1++)
    {printf("\n %d %*c", n1, 3*(n+1-n1), ' '); /* Нов ред,
                                                    номериране и "изместване" */
        for(int k1=0; k1<=n1; k1++)
            printf("%5d", bin_k(n1,k1));
        }
getch();
}

int bin_k(int n, int k)          // Дефиниция на рекурсивната функция
{int result;
  if(k==0 || k==n)
    result = 1;
  else
    result = bin_k(n-1, k-1) + bin_k(n-1, k);
  return result;
}

```

Въведете и компилирайте предложената примерна програма. Изпълнете програмата и анализирайте резултатите за различни стойности на **n** (например със стойности **5**, **7** и **13**). Ще забележите, че граничната стойност за **k** се определя от **размера** на триъгълника, извеждан на екрана. Модифицирайте програмата да бъде с възможности за **запитване за продължение**.

**Задача 9.2.** C/C++ програма, в която чрез рекурсивна функция **ngodr( )** да се намира най-големия общ делител (NGOD) на две положителни цели числа **m** и **n**, които не са едновременно равни на нула. Да се използва алгоритъма на Евклид (рекурсивна функция):

$$NGOD(M,N)=\begin{cases} NGOD(N,M), & N > M \\ M & N = 0 \\ NGOD(N,M\%N), & N \leq M \end{cases}$$

```

/* Лабораторно упражнение 9. Задача 2. */
/* C/C++ програма, в която чрез рекурсивна функция да се намира
най-големия общ делител (NGOD) на две положителни цели числа m и
n, които не са едновременно равни на нула. Да се използва
алгоритъма на Евклид (рекурсивна функция):
    Ако n>m, то NGOD(m,n) = NGOD(n,m);
    иначе ако n=0, то NGOD(n,m) = m; иначе NGOD(m,n) = NGOD(n,m%n). */
#include <iostream.h>
#include <conio.h>

int ngodr(long m, long n);    // Прототип на рекурсивната функция
int br;                       // За брой рекурсии - глобална
void main()
{long n,m;                    // Целочислени числа от тип long int
  char otg;                   // За запитване за продължение

```

```

clrscr();
do{
    do{cout << "\n\n Въведете m >= 0: ";
        cin >> m;
        cout << "\n Въведете n >= 0: ";
        cin >> n;
    }while(m < 0 || n < 0 || m == 0 && n == 0);
    br=0;
    cout << "\n NGOD(" << m << "," << n << ")= " << ngodr(m,n);
    cout << "\n Брой рекурсии: " << br;
    cout << "\n\n Ще продължите ли? За край въведете N/n: ";
    otg = getche();
    }while(!(otg=='N' || otg=='n' || otg=='H' || otg=='h'));
}

int ngodr(long m, long n)          // Дефиниция на рекурсивната функция
{if(n > m)
    {br++;
        return ngodr(n,m);}      // Рекурсивно извикване на функцията
else if(n == 0)
    return m;
else
    {br++;                        // Броене на рекурсиите
        return ngodr(n,m%n);}
}

```

Тази програма е пример за случаи, при които **дълбочината** на рекурсията не може да се определи предварително [7]. За демонстриране на тази особеност, в програмата се използва променливата **br** (за преброяване **броя** на рекурсивните извиквания на функцията **ngodr()**)

Въведете и след успешна компилация изпълнете програмата с различни стойности на **m** и **n** (например **m = 234546** и **n = 234**; **m = 234556** и **n = 234**; **m = 234566** и **n = 234**) и проследете резултатите, включително и **броя на рекурсиите**. Разменете стойностите на **m** и **n**.

**Задача 9.3.** В следващата примерна програма се демонстрира отново алгоритъма на Евклид за намиране на най-големия общ делител, но чрез използване на **итеративна функция ngodi()** [7].

За проследяване на **последователните итерации** във функцията, в цикъла **while** се извеждат и **междинните** резултати.

```

/* Лабораторно упражнение 9. Задача 3. */
/* C/C++ програма, в която чрез итеративна функция да се намира
най-големия общ делител (NGOD) на две положителни цели числа m и
n, които не са едновременно равни на нула. Да се използва
алгоритъма на Евклид: Остатък от деление на двете числа (с размяна
и запомняне), до остатък = 0 и запомняне на последния делител
(като резултат). */
#include <iostream.h>
#include <conio.h>

```

```

int ngodi(long m, long n);      // Прототип на итеративната функция

```



```

void main()
{long n,m;
  char otg;
  clrscr();
  do{
    cout << "\n Въведете m >= 0: ";
    cin >> m;
    cout << "\n Въведете n >= 0: ";
    cin >> n;
  }while(m < 0 || n < 0 || m == 0 && n == 0);
  cout << "\n NGOD(" << m << ", " << n << ")= " << ngodi(m,n);
  cout << "\n\n Ще продължите ли? За край въведете N/n: ";
  otg = getche();
  }while(!(otg=='N' || otg=='n' || otg=='H' || otg=='h'));
}

int ngodi(long m, long n) // Дефиниция на итеративната функция
{long k, nn, mm; // Локални променливи от тип long int
  if(n > m)
    {nn = n; mm = m;}
  else
    {nn = m; mm = n;}
  while(nn) // Цикъл до остатък равен на 0
    {k = nn;
      cout << "\n k = " << k;
      nn = mm%nn;
      cout << " nn = " << nn;
      mm = k;
      cout << " mm = " << mm; // За проследяване на итерациите
    }
  return mm;
}

```

Въведете и след успешна компилация изпълнете програмата с различни стойности на **m** и **n** (например **същите** като използваните във варианта с рекурсивна функция). Проследете получаваните след **всяка итерация** стойности на локалните променливи **k**, **nn**, **mm**.

Сравнете и анализирайте получаваните резултати.

**Задача 9.4.** Примерна C/C++ програма с вариант за **разделна компилация** и дефиниране на **статични глобални променливи и функции**. Програмата се състои от **два модула**, условно наречени **file1** (представен като **Задача 9.4А**) и **file2** (представен като **Задача 9.4В**).

Обработват се данни, съхранявани в структура от тип "опашка" ("първи въведен елемент, първи изведен" - **FIFO**). Характеристиките на структурата "опашка" (организирана като едномерен масив **fifo[MAX]**) и извършваните с елементите ѝ (от тип **double**) обработки са описани подробно чрез използваните във варианта на програма коментари [1].

```

/* Лабораторно упражнение 9. Задача 4А. */
/* Модул от програма за моделиране на структура от данни тип
"опашка". Опашката се реализира чрез масив с име fifo (First In,

```

First Out), в който се съхраняват елементите на опашката. Достъпът до елементите се извършва чрез два индекса `first` и `last`, които са съответно номерът на първия и на последния елемент в опашката. Променливата `count` съдържа текущият брой на елементите в опашката, който не може да бъде по-голям от максималния (константа `MAX`). Чрез функцията `interface()`, която не е статична, се извиква една от функциите `put_last()` - въвеждане на елемент, `get_first()` - извеждане на елемент или `clear()` - изчистване на опашката, в зависимост от параметъра `comanda`. Този параметър има смисъл на команда и се въвежда от клавиатура във функция `main()`, дефинирана в друг файл и имаща достъп единствено до функцията `interface()`. \*/

```

#include <iostream.h>
#include <conio.h>
#include <dos.h>
#define TRUE 1
#define FALSE 0

const MAX = 6;           // Максимална дължина на опашката
// Дефиниции на статични данни
static int first = 0, last = 0, count = 0;
static double fifo[MAX]; // Масив fifo[MAX] с елементи double

// Статична функция put_last - вмъква (въвежда) елемент в опашката
static int put_last(double *element)
{ if(count < MAX)          // Ако опашката не е пълна
    {count++;
      fifo[last] = *element;
      if(++last > MAX-1) last = 0;
      return TRUE;
    }
  else
    return FALSE;          // За пълна опашка
}

// Статична функция get_first - извличане от началото на опашката
static int get_first(double *element)
{ if(count == 0)
    return FALSE;          // Празна опашка
  else
    {count--;
      *element = fifo[first];
      if(++first > MAX-1) first = 0;
      return TRUE;
    }
}

// Статична функция clear - изчистване на опашката
static void clear(void)
{ first = count = last = 0;
}

// Дефиниция на функция interface - интерфейс за работа с опашката
void interface(char comanda)
{double chislo;

```

```

switch(comanda)
{
case 'I': // Вмъкване на елемент в края на опашката
    cout << "\n \t Въведете число за елемент на опашката: ";
    cin >> chislo;
    if(put_last(&chislo))
        cout << "\n \t Вмъкнат е елемент в опашката. \n ";
    else
        cout << "\n \t Опашката е пълна! \n "; break;
case 'O': // Извличане на елемент от началото на опашката
    if(get_first(&chislo))
        cout << "\n \t Извлечен елемент от опашката: "<<chislo<< endl;
    else
        cout << "\n \t Опашката е празна! \n "; break;
case 'C': // Изчистване на опашката
    clear();
    cout << "\n \t Опашката е изчистена! \n "; break;
case 'Q': break;
}
delay(3000); // За прочитане текста на екрана
}

/* Лабораторно упражнение 9. Задача 4В. */
/* Програмен модул, съдържащ функцията main(). Функцията main()
извиква функцията с име interface(), дефинирана в друг програмен
модул (файл), съдържащ статични променливи и функции за операции
със структура от данни, тип "опашка". */
#include <iostream.h>
#include <conio.h>
#include <ctype.h>

void interface(char); /* Прототип на функцията interface(),
дефинирана в друг файл */

void main(void)
{
char comanda; // Команда за избор от меню
do{clrscr();
    cout << "\n \t Въведете латинска буква за команда: ";
    cout << "\n \t I-Въвеждане; O-Извеждане; C-Нулиране; Q-Изход: ";
    cin >> comanda;
    comanda = toupper(comanda);
    cout << "\n ";
    interface(comanda);
}while (comanda != 'Q');
}

```

Файлът **file1** се третира като един **модул**, който има определена вътрешна **структура и поведение** (статични променливи и функции) и **интерфейс** (функцията **interface()**), чрез който се осъществява взаимодействието с външната среда (в случая - функцията **main()** от **file2**). Вътрешната структура на **модула** е недостъпна за никоя друга функция, освен за функцията **interface()**. По този начин се постигат два ефекта: вътрешните данни на **опашката** са предпазени от некоректна работа с тях, а потребителят спазва определена **дисциплина** за работа с **опашката** (което намалява възможността за **грешки**).

За да **свържите двата обектни** файла в **изпълним**, активирайте команда **Open project...** (от меню **Project**) и в диалоговия прозорец задайте име, например **LU9\_Z4.PRJ**. Активирайте бутон **Open** и в отворения прозорец на **LU9\_Z4.PRJ** добавете двата файла **LU9\_Z4A** и **LU9\_Z4B** (или със зададените от вас имена) като преди това активирате команда **Add item...** от меню **Project** (или команда **Add** от реда на състоянието).

Затворете диалоговия прозорец за **добавяне** на файлове в **проектния файл** и активирайте команда **Make** или команда **Build all** от меню **Compile**. При успешно **свързване**, изпълнете с **Run .exe** файла.

Опитайте изпълнението на всички **команди** от **менюто** за работа с **опашката**: **Въвеждане (I)** на елементи; **Извеждане (O)** на елементи; **Нулиране (C)** на опашката; **Изход (Q)**. Въвежданите елементи (не повече от **6**) се извеждат в реда на тяхното въвеждане (Първи Въведен Първи Изведен). Можете да въвеждате числа **int**, **float** или **double**.

### Въпроси и задачи за самостоятелна работа

1. Какви видове рекурсивни функции могат да бъдат дефинирани и използвани в C/C++ програмите?
2. Какво е характерно за функциите с пряка рекурсия?
3. Какво и защо се съхранява в стековата памет при всяко рекурсивно извикване на рекурсивна функция?
4. Какво задължително е необходимо да бъде включено в тялото на рекурсивна функция за да не се запълни заделената стекова памет?
5. Кой е “алтернативния” вариант на рекурсивна функция?
6. За кои програмни обработки е препоръчително и ефективно използването на рекурсивни функции?
7. Кои са “преимущества” на разделната компилация?
8. Какво се реализира чрез дефинирането и използването на статични променливи и функции при прилагане на модулно програмиране?
9. Съставете C/C++ програма, в която чрез рекурсивна функция се изчислява стойността на полинома:  
$$P_n(x) = a_0x^n + a_1x^{n-1} + a_2x^{n-2} + \dots + a_n,$$
където  $x$  и  $a_i$  ( $0 \leq i \leq n$ ) са въвеждани реални числа.
10. Съставете C/C++ програма за изчисляване чрез рекурсивна функция на полинома на Ермит  $H_n(x)$ , където  $x$  е реална, а  $n$  – цяла променлива ( $n \geq 0$ ), като се използват изразите:  
 $H_0(x) = 1; \quad H_1(x) = 2x; \quad H_n(x) = 2x \cdot H_{n-1}(x) - 2(n-1) \cdot H_{n-2}(x)$  за  $n > 1$ .
11. Съставете C/C++ програма, в която чрез рекурсивна функция да се изчислява стойността на функцията (двойно рекурсива, вижте [7]) на Акерман  $A(m,n)$ , дефинирана за  $m \geq 0, n \geq 0$  ( $m$  и  $n$  са цели числа), като се използват изразите:  
 $A(0,n) = n+1$ , за  $m=0$ ;  
 $A(m,0) = A(m-1,1)$ , за  $n=0$ ;  
 $A(m,n) = A((m-1), A(m,n-1))$ , за  $m > 0$  и  $n > 0$ .

## Тема 10. Структури. Масиви от структури. Указатели към структури. Обединения. Програми със структури и обединения.

**Цел:** Запознаване с декларирането на данни от тип структура, с дефинирането и използването на променливи-структури. Съставяне, тестване и анализ на **C/C++** програми, включващи обработка на данни от тип структура, на масиви от структури както и използване на обединения.

### Структури – дефиниране и инициализация. Достъп до елементите на структура.

Използването на структури предоставя възможността да се групират данни и да се работи с тях като едно цяло. Всяка **структура** представлява колекция от **един** или **няколко типа променливи**. **Всеки елемент** в дадена структура може да бъде от **различен тип** за разлика от елементите на масива, които са от един и същи тип.

Типичен пример за структура са данните за студент: всеки студент се описва с набор от атрибути като име, специалност, адрес, факултетен номер и така нататък. Някои от тези компоненти (използва се и термина **членове**) също могат да бъдат структури, например адресът, който съдържа от своя страна също няколко компонента: град, улица и номер.

**Структурите** могат да се **присвояват една на друга**, да се **предават на функции** и да се **връщат от функции** [5].

#### 1. Дефиниране на структури.

Ключовата дума **struct** означава **декларация** на структура, която представлява списък от декларации, заграден от **фигурни скоби**. Променливите, именувани в структурата, се наричат **членове** (**елементи, компоненти**) на структурата. Декларацията **struct** определя **потребителски дефиниран тип**. Дясната фигурна скоба означава края на списъка с членове и след нея може да се постави **списък от променливи**, както при всеки друг тип. Променливата **myaddress**, масивът **addressbook[20]** и указателят **\*ptrstr** в следващия пример са **дефинирани** от тип **структура** и се заделя **памет** за тях.

**Пример:**

```
struct          /* Адрес */
{ char town[20]; /* Град */
  char street[20]; /* Улица */
  int number;     /* Номер */
} myaddress, addressbook[20], *ptrstr;
```

Всяка **структура** може да бъде **именувана**, но това не е задължително. Името се поставя след думата **struct** и може да се

използва впоследствие като кратък запис за декларациите във фигурните скоби.

**Пример:**

```
struct address      /* Адрес */
{ char town[20];    /* Град */
  char street[20];  /* Улица */
  int number;       /* Номер */
};
```

**Декларация на структура**, след която няма списък от променливи, **не заделя място в паметта** (тя описва шаблона или формата на структурата). Ако структурата е именувана, то впоследствие името може да се използва при дефиниране на обекти от тип структура:

```
struct address myaddress, addressbook[20], *ptrstr;
```

Езикът **C/C++** предоставя още една възможност за създаване на **нови типове данни**, наречена **typedef** [3]. **Дефиниране** на потребителски тип **структура** с име **student**:

```
typedef struct      /* За студент */
{ char name[30];    /* Име */
  char speciality[30]; /* Специалност */
  struct address addr; /* Вложена структура */
  unsigned fnumber;  /* Фак. номер */
  int grant;         /* Стипендия */
} student;
```

При дефиниране на променливи се използва **името** на дефинирания тип структура (в случая **student**) и се изпуска ключовата дума **struct**:

```
student stud, array[MAX], *ps[MAX];
```

## 2. Вложени структури.

Съществува възможност да се **влага** една дефиниция на структура в друга. Веднъж декларирана дадена структура, тя се превръща в нов тип данни в програмата и може да бъде използвана навсякъде, където може да присъства друг тип данни (като **int**, **char**, **float** и т.н.). В типа структура **student** по-горе, се съдържа елемент **addr** от тип **struct address**.

## 3. Инициализиране на елементите на структура.

Инициализирането на елементите на дадена структура може да се осъществи по **два** начина: могат да се инициализират, когато се **декларира структурата** като след декларацията ѝ се постави **списък от инициализатори**, съответстващи на елементите на структурата, и всеки инициализатор представлява постоянен израз или по-късно в

програмата посредством присвояване или при извикване на функция, която връща структура от даден тип:

**Пример:**

```
/* Деклариране и инициализиране на структура
едновременно */
struct address      /* Адрес */
{ char town[20];    /* Град */
  char street[20]; /* Улица */
  int number;       /* Номер */
}myaddress = {"Варна", "Свобода", 9};
.....
/* Деклариране и инициализиране на променлива от тип
структура student */
student stud={0}; /* Ако в списъка има по-малко
инициализатори, отколкото е броят на членовете в
структурата, последните членове се инициализират с 0.
*/

/* Елементи на масива array и функцията inputstr() са
от тип структура student */
array[i]= inputstr();
```

В повечето случаи обаче, потребителят **въвежда данните**, които трябва да се съхранят в **структурите**, или данните се прочитат от файл на диск. За тази цел е необходимо да има механизми за обръщане към всеки елемент на променлива от тип структурата.

#### 4. Достъп до елементите на структура.

- Използването на *оператора точка* . е един от начините за третиране на всеки елемент на структурата. Пред оператора точка винаги трябва да има **име на променлива** от тип структурата, а след него винаги трябва да се поставя **име на елемент на структурата**. Например:

|                         |                                                                                                                                         |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| <b>myaddress.town</b>   | Достъп до елемент <b>town</b> на променливата <b>myaddress</b> от тип структура <b>address</b> .                                        |
| <b>array[i].fnumber</b> | Достъп до елемент <b>fnumber</b> на i-я елемент елемент на едномерен масив <b>array</b> от променливи от тип структура <b>student</b> . |
| <b>stud.name</b>        | Достъп до елемент <b>name</b> на променливата <b>stud</b> от тип структура <b>student</b> .                                             |
| <b>stud.addr.town</b>   | Достъп до елемент <b>addr.town</b> на вложената структура <b>address</b> на променливата <b>stud</b> от тип структура <b>student</b> .  |

- Друг начин на достъп до елемент на структура е чрез **указател** към структурата и **указателна операция стрелка** ->. Например:

```
ptrstr=&stud;  
printf(" Име:%s \n", ptrstr->name);
```

Единствените позволени **операции със структура** са **копирането** и **присвояването** ѝ като едно цяло, вземането на **адрес** чрез **&** и работа с **елементите** ѝ. Копирането и присвояването включват също подаването на аргументи към функции и връщането на стойности от функции. Структурите **не могат да се сравняват**.

### Масиви от структури

Докато масивите предлагат удобен начин за съхраняване на еднотипни стойности, чрез масивите от структури може да се съхраняват заедно набори от данни от различен тип, отговарящи на определени шаблони на структури.

Дефинирането на **масив от структури** е напълно аналогично на дефинирането на всеки друг тип масив. Например:

```
student array[MAX];
```

Този оператор дефинира масив с име **array** от **MAX** – елемента като всеки елемент е структура от декларирания тип структура **student**. Така например, първият елемент на масива **array[0]** е структура от тип **student** и следователно съдържа **пет** елемента, към които може да се обърнете чрез оператора точка. Операторът точка се използва по един и същ начин както за елементите на масив от структури, така и за обикновените променливи-структури.

Както знаете, името на масива **array** представлява константен указател към първия елемент на масива – **array[0]**. Следователно може да се използва нотация за указатели, за да се извърши обръщение към данните на отделните студенти. Може да се увеличи стипендията на **третия** студент с **10%**, като се използва нотация за указатели или нотация с индекси, както следва:

```
array[2].grant=(*(array+2)).grant*1.1;
```

Външните кръгли скоби са необходими, защото операция **.** има по-висок приоритет от операция **\***.

### Задача 10.1:

Да се създаде списък от следните данни за студенти: име, специалност, адрес, факултетен номер и стипендия. Програмата да предлага: възможности за въвеждане и извеждане на данни; извеждане данните на студенти от зададена специалност, техния брой и сумата от стипендиите им; извеждане на списък на студенти с най-голяма стипендия; сортиране на студенти по факултетен номер; сортиране на студенти по имена.



```

/* C/C++ програма за създаване, извеждане, обработка и сортиране
на масив array от елементи от тип структура student */
#include <stdio.h>
#include <conio.h>
#include <string.h>
#include <dos.h>

#define MAX 10          /* Максимален брой студенти */

/* Дефиниране на структура */
struct address          /* Адрес */
{ char town[20];        /* Град */
  char street[20];      /* Улица */
  int number;           /* Номер */
};

/* Дефиниране на тип структура */
typedef struct          /* За студент */
{ char name[30];        /* Име */
  char speciality[30];  /* Специалност */
  struct address addr;  /* Вложена структура */
  unsigned fnumber;     /* Фак. номер */
  int grant;            /* Стипендия */
} student;

/* Масиви от структури и от указатели към структури */
student array[MAX], *ps[MAX];
int n;

/* Прототипи на функции */
int menu(void);          /* Избор от меню */
student inputstr(void);  /* Въвеждане на данни за един студент */
void enter(void);        /* Въвеждане на данни */
void display(int i);     /* Извеждане на данни за студент */
void search_spec(void);  /* Търсене на студенти от специалност */
void max_grant(void);    // Списък на студенти с максимална стипендия
void order(student *po,int); /* Сортировка по възходящ ред на фак.
                               номер */
void sort_name(student *[],int); /* Сортировка по азбучен ред на
                                   имена */
void liststr(student *[],int); /* Извеждане на данни с помощта на
                                   указатели */
void outputstud(void);     /* Извеждане на данни */

int main(void)
{ int choice;            /* Локална променлива за избор от меню */
  char start='N';        /* Признак за въведени данни за студенти */
  char ans;              /* За въвеждане на нови данни */
  do {
    clrscr();
    choice=menu();
    switch (choice) {
      case 1: if (start=='N') {
        enter();

```

```

start='Y';    } /* Има въведени данни за студенти */
else { printf("\n Има въведени данни за студенти!");
    printf("\n Изберете кой да е клавиш за отказ!");
    printf("\n За ново въвеждане -->> Y/y: ");
    ans=getche();
    if (ans=='Y' || ans=='y')
        enter(); }
printf("\n Натисни клавиш за продължение!");
getch();
break;
case 2: if (start=='Y')
    search_spec();
    else printf("\n Няма въведени данни за студенти!");
    printf("\n Натисни клавиш за продължение!");
    getch();
    break;
case 3: if (start=='Y')
    max_grant();
    else printf("\n Няма въведени данни за студенти!");
    printf("\n Натисни клавиш за продължение!");
    getch();
    break;
case 4: if (start=='Y') {
    order(array,n);
    printf("\n Списък на студенти по фак. номер\n");
    outputstud(); }
    else printf("\n Няма въведени данни за студенти!");
    printf("\n Натисни клавиш за продължение!");
    getch();
    break;
case 5: if (start=='Y') {
    sort_name(ps,n);
    printf("\n Списък на студенти по имена\n");
    liststr(ps,n); }
    else printf("\n Няма въведени данни за студенти!");
    printf("\n Натисни клавиш за продължение!");
    getch();
    break;
case 6: if (start=='Y') {
    printf("\n Списък на студенти \n");
    outputstud(); }
    else printf("\n Няма въведени данни за студенти!");
    printf("\n Натисни клавиш за продължение!");
    getch();
    break;
default: { printf("\n \t \t ### Край на програмата! ###");
    delay(2000); }
}
} while(choice!=7);
return 0;
} /* Край на main */

menu(void) /* Избор от меню */
{ int ch; /* Връщана стойност за избор от меню */

```

```

printf("\n #####");
printf("\n # Меню #");
printf("\n # 1.Въвеждане на данни за студент #");
printf("\n # 2.Списък, брой и сума от стипендии на #");
printf("\n # студенти от зададена специалност #");
printf("\n # 3.Списък на студенти с най-голяма стипендия #");
printf("\n # 4.Сортиране на данни за студенти по фак. номер #");
printf("\n # 5.Сортиране на данни за студенти по имена #");
printf("\n # 6.Извеждане на данни за студенти #");
printf("\n # 7.Край на програмата #");
printf("\n #####");

do { printf("\n Въведете вашия избор: ");
    scanf("%d",&ch);
    printf("\n Изборът е: %d",ch);
    delay(2000);
} while (ch<1 || ch>7);
return ch;
} /* Край на menu */

student inputstr(void) /* Въвеждане на данни за един студент */
{ student stud={0}; /* Инициализация на структура */
    printf("\n Въведи име: ");
    gets(stud.name);
    printf("\n Въведи специалност: ");
    gets(stud.speciality);
    printf("\n Въведи град: ");
    gets(stud.addr.town);
    printf("\n Въведи улица: ");
    gets(stud.addr.street);
    printf("\n Въведи номер: ");
    scanf("%d",&stud.addr.number);
    printf("\n Въведи фак.номер: ");
    scanf("%u",&stud.fnumber);
    printf("\n Въведи стипендия: ");
    scanf("%d",&stud.grant);
    fflush(stdin); /* Изчиства буфера */
    return(stud); /* Връща структура */
} /* Край на inputstr */

void enter(void) /* Въвеждане на данни */
{ int i;
    clrscr();
    fflush(stdin);
    do { /* Въвеждане на реален брой студенти <=MAX */
        printf("\n Въведи брой студенти <=%d: ",MAX);
        scanf("%d",&n);
    } while ((n<1) || (n>MAX));
    fflush(stdin);
    for (i=0;i<n;i++)
        { printf("\n Студент # %d \n",i+1);
          array[i]= inputstr(); /* Данни за i-тия студент */
          ps[i]=&array[i]; }
}

```

```

} /* Край на enter */

void display(int i) /* Извеждане на данни за i-тия студент */
{ printf("\nИме: %s",array[i].name);
  printf("\n Специалност: %s",array[i].speciality);
  printf("\n Адрес Град: %s",array[i].addr.town);
  printf("\n          Улица '%s' Номер: %d",array[i].addr.street,
          array[i].addr.number);
  printf("\n Фак. номер: %u",array[i].fnumber);
  printf("\n Стипендия: %d",array[i].grant);
  return;
} /* Край на display */

void search_spec(void)
/* Търсене на студенти от зададена специалност */
/* Списък, брой и сума от стипендии */
{ char spec[30];
  int i, sum, count;
  clrscr();
  fflush(stdin);
  printf(" Въведи специалност: ");
  gets(spec);
  delay(1500);
  sum=0; // Инициализиране на променливите, които ще отчитат сумата
  count=0; /* и броя */
  for (i=0;i<n;i++)
    if (!strcmp(spec,array[i].speciality)) {
      sum+=array[i].grant; count++; /* Намерен студент */
      display(i); } /* Извежда данни за студент */
  if (count>0) {
    printf("\n Общ брой студенти - %d от специалност %s.",
    count,spec);
    printf("\n Сумата от стипендиите е %d.",sum); }
  else
    printf("\n Няма студенти от специалност %s!",spec);
} /* Край на search_spec */

void liststr(student *ptr_str[],int n)
{ int i;
  for(i=0;i<n;i++)
    { printf("Име: %s\n",ptr_str[i]->name);
      printf("Специалност: %s\n",ptr_str[i]->speciality);
      printf("Адрес Град: %s\n",ptr_str[i]->addr.town);
      printf("          Улица '%s' Номер: %d\n",
      ptr_str[i]->addr.street, ptr_str[i]->addr.number);
      printf("Фак. номер: %u\n",ptr_str[i]->fnumber);
      printf("Стипендия: %d\n",ptr_str[i]->grant);
    }
} /* Край на liststr */

void max_grant(void)
/* Списък на студенти с максимална стипендия */
{ student *psmax[MAX]; /* Масив от указатели към структури */
  int i, maxgrant, j, imax;

```

```

clrscr();
j=0;
maxgrant=array[0].grant; imax=0;
for (i=1;i<n;i++)
    if (maxgrant<array[i].grant)
        { maxgrant=array[i].grant; imax=i; }
for (i=imax;i<n;i++)
    if (array[i].grant==maxgrant) {
        psmax[j]=&array[i]; /* Присвояване на адреса на данните
                               на i-тия студент на съответния указател от масива psmax */
        j++;
    }
printf("\n Списък на студенти с максимална стипендия - %4d\n",
        maxgrant);
liststr(psmax,j);
} /* Край на max_grant */

void order(student *po, int n)
{ /* Сортировка на данни за студенти по фак. номер */
    /* po - указател към началото на масива от структури */
    /* n - брой на елементите на масива */
    student *p, *pend, work; /* Работна променлива */
    int change, flag=1;
    change=0;
    while (flag!=0)
    { flag=0; /* Флаг за проверка наличието на размяна */
      change++;
      pend=po+n-change;
      for(p=po;p<pend;p++)
          if (p->fnumber>(p+1)->fnumber) /* Проверка на наредбата */
          { work=*p; /* Промяна съдържанието на указателите */
            *p=*(p+1);
            *(p+1)=work;
            flag=1; }
    } /* Край на while */
} /* Край на order */

void sort_name(student *po[],int n)
{ /* Сортировка на данни за студенти по азбучен ред на имена */
    student *temp;
    int top, search;
    for (top=0;top<n-1;top++)
        for (search=top+1;search<n;search++)
            if (strcmp(po[top]->name,po[search]->name)>0)
            { temp=po[top]; /* Размяна на адреси */
              po[top]=po[search];
              po[search]=temp; }
} /* Край на sort_name */

void outputstud(void)
{ /* Извеждане на данни за студенти */
    int i;
    clrscr();
    for(i=0;i<n;i++)
        display(i);
}

```

```
} /* Край на outputstud */
```

Функцията за въвеждане на данните за един студент е **inputstr()** от тип структура **student**. Тази функция се извиква в тялото на цикъл **for** във функция **enter()** и така се въвеждат всички данни за елементите на масив **array** от декларирания тип структура **student**. Освен това адресът на всеки елемент на този масив се записва в масив от указатели към структури от тип **student** - **ps**.

Функцията **search\_spec()** се използва за търсене на студенти от зададена (от клавиатурата) специалност. При наличие на такива студенти се извеждат техните данни, броят и сумата от стипендиите им. За извеждане на данни за *i*-тия студент се извиква функция **display()**. За достъп до компонентите на структурите като елементи на масива е използван оператора точка.

Функцията **max\_grant()** служи за извеждане на списък на студенти с максимална стипендия. С първия цикъл **for** се намира максималната стойност и индекса на елемента с тази стойност. С втория цикъл **for**, започващ от елемента с максималната стойност, се проверява дали има други елементи на масива със стойност на стипендията равна на максималната. Адресите на елементите на масива от структури с максимална стипендия се присвояват на съответни елементи от масив **psmax** от указатели към структури от тип **student**. За извеждането на списъка е създадена функция **liststr()** с формални параметри масив от указатели към структури от декларирания тип структура **student**, като масива е без фиксиран размер и броя на елементите на масива. При извикването на функцията като фактически параметър се използва името на масива **psmax** и **j** – броя на студентите с максимална стипендия. Името на масива е константен указател към началния елемент на масива. Достъпът до елементите на масива от указатели се осъществява по неговия индекс, определящ отместването на адреса на елемента спрямо началния адрес на масива. И тъй като стойностите на елементите на масив **psmax** са адресите на променливите-структури с максимална стипендия от масив **array**, за достъп до техните компоненти се използва операция стрелка **->**.

Представени са две функции за сортиране на данните на студентите с използване на метода на мехурчето. Функцията **order()** дава възможност да се сортират данните за студенти по факултетен номер във възходящ ред като формалните ѝ параметри са указател към тип структура **student** и брой на елементите на масива. В резултат от изпълнението на тази функция са сортирани елементите на масив **array**.

Чрез функцията **sort\_name** с формални параметри: масив без фиксиран размер от указатели към структури от тип **student** и брой на елементите на масива, данните се сортират по азбучен ред на имената. Външният цикъл **for** указва кой елемент на масива трябва да бъде определен, а вътрешният цикъл **for** намира стойността, която трябва да

бъде поставена там така че данните да бъдат сортирани. Всъщност в този случай, самите данни в масив **array** не се сортират, ***сортират се указателите, които сочат към тях.***

Функцията **outputstud()** извежда студентските данни от масив **array**. В тялото на масив **for** се извиква функцията **display()** за извеждане на текущия елемент на масива.

## Обединения

**Обединението** е променлива, която може да съдържа (в различни моменти) **обекти от различен тип и големина**, като компилаторът следи размера и изискванията за подравняване. Ако една константа може да бъде **int**, **long int** или **масив** от тип **char**, съответната стойност трябва да се записва в променлива от подходящ тип като заема едно и също количество памет и се съхранява на едно и също място, независимо от типа ѝ.

От синтактична гледна точка декларирането и достъпът до членовете на обединението се извършва точно както при структурите. **Шаблонът и променливата обединение** могат да се декларират **едновременно** или, ако се **използва име на обединението, последователно на две стъпки**:

```
union u_holder /* Деклариране на шаблон на обединение */
{
    int ival;
    long int lival;
    char strval[6];
} uvar={10};
union u_holder save[10], *p;
```

**Компилаторът** отделя **достатъчно памет** за разполагане на **най-големия** от трите типа. На **uvar** може да се присвои всеки един от тези типове и после да се използва в изрази, стига само употребата да е последователна: **извлечения тип** трябва да бъде **последният съхранен**. В действителност обединението представлява структура, в която всички елементи притежават нулево отместване от основата и структурата е достатъчно голяма, за да побере „най-широкия“ елемент, а подравняването е подходящо за всички типове в обединението.

Обединението може да се **инициализира** само със стойност от типа на **първия член**; следователно обединението **uvar** може да се инициализира само с целочислена стойност.

Можем да осъществим достъп до елемента **ival** чрез записа:

```
uvar.ival=60;
```

или като се използва операция **стрелка ->**:

```
p=&uvar; /* Указател към променлива от тип u_holder */
x=p->ival; // Също както и x=uvar.ival; x е от тип int
```

Може да се поставят обединения в структури и масиви, както и обратно. Получаването на достъп до член на обединение, което е

поставено в структура (или обратно), е идентично както при вградени структури. Същите **операции**, които са разрешени при **структурите**, са позволени и при **обединенията**: присвояване или копиране на обединението като едно цяло, вземане на адреса или получаване на достъп до членовете му.

### Задача 10.2:

Да се състави програма за създаване на масив, състоящ се от елементи с еднакъв размер, всеки от които може да съдържа различни типове данни. Да има възможност за въвеждане и извеждане на данни за масива.

```
/* C/C++ програма с използване на обединения за създаване и
извеждане на масив от елементи с еднакъв размер (тип структура
fit), всеки от които може да съдържа различни типове данни,
например int, long int или масив от тип char. */
#include <stdio.h>

#define INT 1
#define LONGINT 2
#define STRING 3
#define N 10

union u_holder    /* Деклариране на шаблон на обединение */
{
    int ival;
    long int lival;
    char strval[6];
} ;

/* Деклариране на структура и дефиниране на масив от структури */
struct fit
{
    int utype; /* За запомняне на типа, съхранен в u */
    union u_holder u;
} tab[N];

void input(struct fit table[]); /* Прототипи на функции */
void output(struct fit *p);

void main()
{
    struct fit *pu, *puend;
    printf("\n Въвеждане на данни за масива\n");
    input(tab); /* Въвеждане на данни за масива */
    pu=tab; /* Сочи началото на масива */
    puend=pu+N; /* Сочи края на масива */
    printf("\nСъдържание на масива\n\n");
    for(pu;pu<puend;++pu)
        output(pu);
}

void input(struct fit table[])
/* Функция за въвеждане на данни за масива */
{
    int i;
    for(i=0;i<N;i++)
        { do {
```



```

printf("\nВъведи тип: ");
printf("1 за int, 2 за long int, 3 за масив от тип char:\n");
scanf("%d",&table[i].utype);
} while ((table[i].utype<INT)|| (table[i].utype>STRING));
fflush(stdin);
if (table[i].utype==INT)
{ printf("\nВъведи число тип int:");
  scanf("%d",&table[i].u.ival); }
else if (table[i].utype==LONGINT)
{ printf("\nВъведи число тип long int:");
  scanf("%lx",&table[i].u.lival); }
else if (table[i].utype==STRING)
{ printf("\nВъведи низ:");
  gets(table[i].u.strval);
}
}
}

void output(struct fit *p)
/* функция за извеждане съдържанието на елемент на масива */
{ if (p->utype==INT)
  printf("%d int - %d\n",p->utype,p->u.ival);
  else if (p->utype==LONGINT)
  printf("%d long int - %lx\n",p->utype,p->u.lival);
  else if (p->utype==STRING)
  printf("%d string - %s\n",p->utype,p->u.strval);
}

```

В програмата са декларирани шаблон за обединение **u\_holder** и структура **fit**, която включва елемент **utype** от тип **int** и елемент обединение **u** от декларирания шаблон **u\_holder**. Дефиниран е масив **tab** с елементи от тип структура **fit**. Всеки елемент на масива **tab[i].u** ще бъде достатъчно голям, за да съхранява различен тип данни - **int**, **long int** или масив от тип **char** като стойностите ще заемат едно и също количество памет, независимо от типа им. Елементът **tab[i].utype** ще се използва за да се помни типа на данните, съхранени в **tab[i].u**.

За **въвеждане** на данните за масива е създадена функция **input()** с формален параметър масив от структури от декларирания тип структура **fit**, като масива е без фиксиран размер. Цикъл **for** указва кой елемент на масива трябва да бъде определен. Първо се избира тип като коректността на въведената стойност се проверява с цикъл **do-while**. В зависимост от избрания тип, се извежда подсказващо съобщение и се реализира въвеждането на данни от съответния тип. При извикването на функцията като фактически параметър се използва името на масива **tab**.

Чрез функцията **output()** с формален параметър указател към тип структура **fit** се извежда съдържанието на един елемент на масива. Достъпът до елементите на структурата се осъществява с операция стрелка **->**.

В главната функция **main()**, указателите **pu** и **puend** към тип структура **fit** са инициализирани да сочат съответно началото и края на масива **tab**. Цикълът **for** използва указателя **pu**, за да изведе съдържанието на елементите на масива чрез функция **output()**.

### Въпроси и задачи за самостоятелна работа

1. По какво се различава декларацията на структура от дефиницията на променлива-структура?
2. От какъв тип могат да бъдат елементите на деклариран тип структура?
3. Как е възможно да се извърши инициализация на елементите на дефинирана променлива-структура?
4. Как е възможно да се осъществи достъп до елементите на дефинирана променлива-структура?
5. От какво се определят допустимите операции с елемент от дефинирана променлива-структура?
6. Как се извършва въвеждането на променливи-структура от клавиатура и извеждането им на екран?
7. Какво е неправилното в шаблона на структура?

```
structure {char *subject;
           int codesub;
           int semester; }
```

8. Какво ще изведе следния фрагмент от програма?

```
struct item
{
    char *name;
    int quantity;
    float price; };

struct item computer = {"HP",20,1000.5};
struct item *ptri;
ptri=&computer;
printf("%s\n",computer.name) ;
printf("%c %c \n",ptri->name[0],computer.name[1]) ;
printf("%d %4.2f \n",computer.quantity,ptri->price) ;
```

9. Съставете вариант на програмата от Задача 10.1 като дефинирате факултетния номер за студента като низ. Въведете проверка за стипендията в определен диапазон и за уникален (неповтарящ се) факултетен номер (при въвеждане).
10. Да се състави C/C++ програма, която връща броя на дните на месец, въведен или като име или като номер.
11. Да се състави програма, която въвежда данни за дисциплини: код, име и семестър, в който се изучава дисциплината. Да се реализира извеждане на информацията по реда на въвеждане, сортирана или по код или по семестър, без да се променя входния ред.
12. Да се състави програма, която въвежда данни за специалности в университет: код и име на специалност, брой студенти. Да се

изведат данните за първите пет специалности с най-голям брой студенти както и специалността с най-малък брой студенти.

13. Да се състави C/C++ програма, която въвежда данни за студенти: факултетен номер, име, фамилия, име на дисциплина, оценка. Да се изведат данните за студенти, сортирани по среден успех както и дисциплините с най-голям брой двойки и отлични оценки.
14. Да се дефинира масив от структури, съдържащи информация за книги в библиотека: сигнатура, заглавие на книга, автор, цена и бройки. Да се състави C/C++ програма, която предлага следните възможности за избор от меню: въвеждане; извеждане наличността от книги или по зададен автор или по азбучен ред на заглавията; извеждане на общия брой книги, налични в библиотеката; и общата сума, за която има книги в библиотеката.
15. Да се състави C/C++ програма със следното меню: създаване на списък от служители на фирма със следните данни: единен граждански номер (ЕГН), име, фамилия, име на отдел, основна заплата, надбавка за прослужени години; извеждане на ЕГН, име, фамилия и брутна заплата за служителите от въведен отдел както и фонд заплати за този отдел; извеждане на информация за служителите със заплата в зададен диапазон.
16. Да се дефинира един масив от структури, съдържащи информация за клиентите на сервиз за коли: код на клиент, име, фамилия, адрес (град, улица/квартал, номер), данни за кола (марка, година на производство, номер) и друг масив от структури, съдържащи информация за извършените ремонти: указател, който сочи клиент, дата на ремонта (ден, месец, година) и цена на ремонта. Да се реализира следното меню: извършване на ремонт - въвежда се кода на клиента, в случай, че е наличен в масива с клиенти, се прави запис в масива с ремонтите, в противен случай се въвеждат данни за клиента и за ремонта; извеждане списък на клиентите, направили ремонт на зададена дата и общата сума от ремонти за тази дата. Да се използват вложени структури.
17. Разширете Задача 10.2 с функции, които да илюстрират различни обработки на данните, съхранени в масива, състоящ се от елементи с еднакъв размер, всеки от които съдържа различни типове данни.

## Тема 11. Файлов вход/изход. Функции за работа с файлове. Програми за работа с текстови и двоични файлове.

**Цел:** Запознаване с основите на файловата система в **C/C++**, четене/записване на двоични и текстови данни и реализация на произволен достъп до записите на файл. Съставяне и изпълнение на програми с използване на стандартни функции за файлов вход/изход.

### 1. Какво представляват поток и файл?

Входно-изходната система на **C/C++** предоставя постоянен интерфейс между програмиста и хардуера, който не зависи от използваното устройство. За целта се дефинира и използва **поток (stream)** от данни. Действителното устройство, чрез което извършва вход/изход се нарича **файл** (дисков, клавиатурен, екранен).

Така третиран **файлът** дава възможност **C/C++** програмата да не прави разлика между отделните устройства за вход/изход. **Потокът** “автоматично” преодолява различията.

Файловете, използвани в **C/C++** програмите имат две “имена”. Външното име на файла, което се използва и от операционната система, може да бъде: константен стринг, променлива от тип стринг или параметър на функция от тип стринг.

След като файлът се отвори, програмата използва файлова променлива, с която е асоцииран файла или входно/изходен поток.

**Потокът** се свързва с **файл** посредством операция за **отваряне**, а се освобождава чрез операция за **затваряне**.

Използват се **два** вида потоци: **текстови** и **двоични**. **Текстовият** поток съдържа **ASCII** символи. **Двоичният** поток може да се използва с **всякакъв тип данни** и предаването на информацията е едно към едно между **потока** и **файла**.

### 2. Файлов вход/изход в C.

В **C** програмите, за работа с файлове се използват **стандартни функции**, дефинирани в заглавния файл **stdio.h** [1].

Функцията **fopen()** е за **отваряне** на файла. Като аргументи на функцията се задават **името на файла** и **режимите** (входно/изходен **статус**), в които ще работи файла [1]. Затварящата функция е **fclose()**.

Ако функцията **fopen()** не успее да отвори файла, връща **нулев указател**. Функцията **fclose()** връща **нула** при успешно затваряне.

**Задача 11.1.** Съставете програма, която записва низа “Моят първи файл на C” в изходен файл “out.txt”. В програмата да се затваря файла, след това да се отваря за четене и да се извежда информацията на екрана.

**Примерен вариант на решение:**

```
#include <iostream.h>
#include <stdio.h>
```

```
#include <stdlib.h>

void main()
{ char str[80]="Моят първи файл на С \n";
  FILE *fp;
  char *p, ch;
  if((fp=fopen("out.txt","w"))==NULL) // Отваря файла за запис
  {cout << endl << "Грешка при отваряне на файла";
   exit(1);}
  p = str;
  while(*p)
    {if(fputc(*p,fp) == EOF)      // Записва стринга във файла
     {cout << endl << "Грешка при запис във файла";
      exit(1);}
     p++;
    }
  fclose(fp);                      // Затваря файла
  if((fp=fopen("out.txt","r"))==NULL) // Отваря файла за четене
  {cout << endl << " Грешка при отваряне на файла";
   exit(1);}
  for(;;)      // Четене от файла и извеждане на екрана
  {   ch = fgetc(fp);
     if(ch == EOF ) break;
     putchar (ch);
  }
  fclose(fp);
}
```

Файлът **out.txt** се създава в **текущата папка**. При разполагане на файла на друго място, **пътят** до него се изписва в **fopen()**.

**Функцията fgetc()** чете следващ байт от **файла**. Ако достигне края на файла или грешка - връща **EOF**. **Функцията fputc()** записва байт във **файла**. Връща **EOF**, ако се появи грешка.

Възможно е тази програма да бъде променена по следния начин:

```
.....
p = str;
while(*p)      // Записва стринга (по символи) във файла
  {if(fputc(*p++,fp) == EOF) // p++ се включва в fputc()
   {cout << endl << "Грешка при запис във файла";
    exit(1);}
  }
fclose(fp);

И // За четене от файла
// Операция четене и присвояване е поставена в логическия израз
// на while и цикъла съдържа само командата putchar(ch)
while ((ch = fgetc(fp)) != EOF) putchar (ch);
fclose(fp);
```

Когато файловата система на **С** работи с **двоични файлове** се използват **функциите fread()/fwrite()**. Тези функции въвеждат/извеждат всякакви типове данни, като използват техните **двоични представления**. Когато използвате тези две функции, **файлът** трябва да е отворен като **двоичен (binary)**. Забележете, че при отваряне на **файл**, той по **подразбиране е текстов**.

**Задача 11.2.** Съставете програма, която инициализира масив с реални числа, записва ги във двоичен файл, след което ги чете обратно и ги извежда на екрана на монитора.

**Примерен вариант на решение:**

```
#include <iostream.h>
#include <stdio.h>
#include <stdlib.h>

double massiv[10] = {1.34, 18.67, 1009.674, 666.88, 12.8,
                    0.0, 34.86, 22.222, 1.00001, 0.7777};

void main()
{int i;
 FILE *fp;
 if((fp = fopen("out_massiv.dat","wb")) == NULL)
 {cout << endl << "Грешка при отваряне на файла";
  exit(1);}
 for (i = 0;i < 10;i++) // Записване във файла
  if(fwrite(&massiv[i], sizeof(double),1,fp) != 1)
  {cout << endl << "Грешка при запис" << endl;
   exit(1);}
 fclose(fp);
 if((fp = fopen("out_massiv.dat", "rb")) == NULL)
 {cout << endl << "Файлът не се отваря";
  exit(1);}
 for(i = 0;i < 10; i++) // Нулиране на масива
  massiv[i] = 0.0;
 for(i = 0; i< 10; i++) // Четене от файла
  if(fread(&massiv[i], sizeof(double), 1, fp)!=1)
  {cout << endl << "Грешка при четене" << endl;
   exit(1);}
 fclose(fp);
 for(i = 0; i < 10; i++) // Извеждане на масива
  cout << endl << massiv[i];
}
```

Файлът се отваря за **запис** с **"wb"** - като **двоичен**, а с **"rb"** – за **четене, двоичен**.

Посредством **fwrite()**, в тялото на цикъл **for**, стойностите на **елементите** на масива **massiv[i]** се записват във файла **out\_massiv.dat**.

Посредством **fread()** се четат обратно, като масива предварително е попълнен със стойности **0**.

Функцията **fwrite()/fread()** съдържа **4** параметъра: **адреса** на променливата, който ще получи записаната/прочетената в/от файла стойност, **брой** байтове, които се записват/четат в/от файла, **брой** записани/прочетени „единици - обекти" и **файловия указател (fp)**. **Броят байтове** се формира чрез оператора **sizeof**, който определя **размера** на променливата или на нейния тип (в **брой байтове**).

Това решение може да се подобри като се **премахнат циклите** за **запис** и **четене** на **масива** и тези действия се извършат чрез **еднократно** извикване на **fwrite()** и **fread()**. В този случай:

```
for(i = 0; i < 10; i++)
    if(fwrite(&massiv[i], sizeof(double), 1, fp) != 1)
        { cout << endl << "Грешка при запис" << endl;
          exit(1); }
```

Ще се замести от:

```
if(fwrite(massiv, sizeof massiv, 1, fp) != 1)
    { cout << endl << "Грешка при запис" << endl;
      exit(1); }
```

Заместването за **fread()** е подобно.

Обърнете внимание, че **вместо адрес на променлива** във функциите се използва само **името на масива** (то е указател към първия елемент на масива).

Операторът **sizeof massiv** определя **размера на масива** в брой байтове.

Възможно е данните на файла да не се четат последователно. Ако се използва **функцията fseek()**, винаги може да имате **достъп до всяка позиция във файла** (позицията във файла, в която ще се осъществи следващият достъп). Функцията оперира с **3** параметъра: файлов **указател**, **брой байтове**, които ще прескочи (стойност на **отместване**) и позицията от която ще се извърши **отместването** (вижте таблицата). Забележете, че ако търсенето се прави от **SEEK\_END**, **отместването** трябва да бъде с **отрицателен знак**.

| Начало   | Значение                   |
|----------|----------------------------|
| SEEK_SET | Търси от началото на файла |
| SEEK_CUR | Търси от текущата позиция  |
| SEEK_END | Търси от края на файла     |

**Функцията fseek()** връща **0** при **успешно завършване** и **ненулева стойност** при **грешка**.

**Текущата позиция на даден файл** се определя с помощта на **функцията ftell()**. Ако се появи **грешка** се връща стойност **-1**.

Допълнете **Задача 11.2** като “изискате” от програмата да върне елемента, който се намира на въведена от потребителя позиция (номер на записа) във файла.

Допълнителният фрагмент към програмата изглежда така:

```
// Търсене на елемент
cout << endl << "Кой номер елемент търсите?" << endl;
cin >> pos;
// pos е променлива, дефинирана от тип long int, показваща
// номера на търсения елемент
if (fseek(fp, pos*sizeof(double), SEEK_SET))
    { cout << endl << "Грешка при търсене" << endl;
      exit(1); }
fread(&value, sizeof(double), 1, fp);
// value е променлива, дефинирана от тип double, в която се
// записва търсената стойност
cout << endl << "Търсеният елемент е: " << value;
fclose(fp);
```

Получените **результати**, примерно ще изглеждат така:

Кой номер елемент търсите?

7

Търсеният елемент е: 22.222

В поредицата елементи във файла, това е **осмият** елемент. Ако за стойност на променливата **pos** се въведе 7 (номер на елемента), файловият указател прескача 7 елемента тип **double** и застава в началото на **осмия** елемент. Функцията **fread()** прочита този елемент и той се извежда на екрана. Така че ако се търси **pos** елемент, целесъобразно е функцията **fseek()** да прескочи **pos - 1** позиции.

Добавете в програмата **обратното** действие: търсене на стойността **22.222**. На коя **позиция във файла** е записана тя?

**Вариант** на добавения код изглежда така:

```
cout << endl<< "Въведи число:";
cin >> value;
for (i=0; i<10; i++)
{if(fread(&massiv[i], sizeof(double), 1, fp) !=1 )
    {cout << endl << "Грешка при четене" << endl;
    exit(1);}
else
    if(massiv[i] == value) break;
}
pos=ftell(fp);
fclose(fp);
cout << endl << "Позицията е:"<<pos/sizeof(double) << endl;
```

**Резултатът** се получава в следния вид:

Въведи число: 22.222

Позицията е 8

Забележете, че **променливата pos** трябва да бъде от тип **long**, тъй като **функцията ftell()** връща стойност от тип **long**.

При използване на **функцията fseek()**, **параметърът на отместване** също трябва да бъде от тип **long**.

Понякога се налага да се изчисти **буфера на файла**, за да не се въведе "излишна" информация във файла. За тази цел се използва функцията **fflush()** в вида: **fflush(fp)** ;

Функцията връща **0**, ако е завършила успешно и **EOF**, при грешка.

### 3. Файлов вход/изход в C++.

За да работите с този **файлов вход/изход** трябва да включите заглавния файл **fstream.h** във вашата програма. Той дефинира класовете **ifstream**, **ofstream** и **fstream**. Потоците (**входен**, **изходен**, **входно/изходен**) се свързват с файлове посредством файловия указател **fp**, който се дефинира от тип **ifstream** или **ofstream**.

В следващите програмни фрагменти накратко са представени варианти на **входно/изходни оператори**, като за имена на файлове се използват: за четене – **infile.txt**, а за запис – **outfile.txt**.

- включване на заглавни файлове:



```
#include <ifstream.h> // За входно/изходни файлове
#include <iostream.h> // За cin и cout
#include <stdlib.h> // За функция exit()
```

- избор на входен (**inStream**) и изходен (**ofStream**) поток и дефинирането им:

```
ifstream inStream;
ofstream outStream;
```

При използване на файлов указател се дефинира:

```
ifstream fp;
ofstream fp;
```

- свързване на всеки поток към файл с външно файлово име и използване на функцията **fail()** за проверка коректността на отварянето на файла:

```
inStream.open("infile.txt");
if (inStream.fail())
{cout << "Файлът не може да се отвори" <<endl;
 exit(1);}
outStream.open("outfile.txt");
if (outStream.fail())
{cout << "Файлът не може да се отвори" << endl;
 exit(1);}
```

- използване на потока **inStream** за **въвеждане** на данни от файла **infile.txt**, по същия начин както се въвеждат данни от клавиатурата чрез **cin**:

```
inStream >> variable1 >> variable2;
```

- използване на потока **outStream** за **извеждане** на данни във файла **outfile.txt**, по същия начин както се извеждат данни на екрана чрез **cout**:

```
outStream << variable1 << variable2 << endl;
```

- затваряне на файловете и асоциираните с тях потоци чрез функцията **close()**:

```
inStream.close();
outStream.close();
```

**Задача 11.3.** Съставете програма, която създава файла **infile.txt** съдържащ **3** цели числа. След това въвежда трите числа от файла, сумира ги и записва сумата във втори файл **outfile.txt**.

**Примерен вариант на решение:**

```
#include <fstream.h>
#include <iostream.h>
#include <stdlib.h>
```

```
void main()
{int chislo1(10),chislo2(20),chislo3(30);
 ofstream fp; // Създаване на файл infile.txt
 fp.open("infile.txt");
 if (fp.fail())
 {cout << "Файлът не може да се отвори" << endl;
 exit(1);}
 fp << chislo1 << ' ' <<chislo2 << '\n' << chislo3; << '\n';
 fp.close();
```

```

chislo1 = chislo2 = chislo3=0; // Нулиране на променливите
ifstream fp1;                // Отваряне на двата файла
ofstream fp2;
fp1.open("infile.txt");
if(fp1.fail())
{cout << "Файлът не може да се отвори" << endl;
 exit(1);}
fp2.open("outfile.txt");
if(fp2.fail())
{cout << "Файлът не може да се отвори" << endl;
 exit(1);}
fp1 >> chislo1 >> chislo2 >> chislo3;
// Контролно извеждане на стойностите на трите променливи след
// прочитането им от файла infile.txt
cout << endl << chislo1 << endl << chislo2 << endl<<chislo3;
fp2 << "Сумата на 3 числа"<< endl << " е равна на: " <<
      chislo1+chislo2+chislo3 << endl;
fp1.close();
fp2.close();
}

```

Ако отворите файла **outfile.txt**, резултатът е следния:

**Сумата на 3 числа  
е равна на: 60**

Обърнете внимание на фрагмента за записване на трите числа във файла **infile.txt**. Ако се използва директно пренасочване:

```
fp << chislo1 << chislo2 << chislo3;
```

числата се записват във файла **едно до друго** и при четене от файла за първо число се взема стойността **102030**. За останалите **2** числа остава **0**. Затова е необходимо поставянето на **празен интервал** или символ за **нов ред**.

**Добавете** в програмата на **Задача 11.3.** фрагмент за четене на данните от файла **outfile.txt** и извеждането им на екрана.

**Примерния фрагмент на решение може да бъде:**

```

// Прочитане на резултата от файла outfile.txt и извеждането му
// на екрана
ifstream fp3;
fp3.open("outfile.txt");
if(fp3.fail())
{cout << "Файлът не може да се отвори" << endl;
 exit(1);}
char result1[80], result2[80];
fp3.getline(result1,80); /* При пренасочване се игнорират
                          празните интервали и символа за нов ред,
                          затова се използва getline() */
fp3.getline(result2,80);
cout << endl << result1 << result2 << endl;
fp3.close();

```

В този случай четенето от файла се извършва с **f3.getline()**, защото резултата е разположен на **2** реда в текстовия файл **outfile.txt** и двата реда - стрингове се записват в **result1** и **result2**.

Функцията **open()**, освен **име на файла**, съдържа стойности на **ios** статуса (режима) за работа с файла, представени в следната таблица:

| Режим         | Значение                                              |
|---------------|-------------------------------------------------------|
| <b>app</b>    | Отваря файла за добавяне на данни към края му         |
| <b>ate</b>    | Премества указателя в края на файла при отваряне      |
| <b>binary</b> | Отваря файла в двоичен режим на достъп                |
| <b>in</b>     | Отваря файла за четене                                |
| <b>out</b>    | Отваря файла за запис                                 |
| <b>trunc</b>  | Изтрива съдържанието на съществуващ файл при отваряне |

**Задача 11.4.** Съставете **C++** програма за създаване на текстов файл **symbol.txt**, съдържащ символни низове. Програмата да спира при въвеждане на **\$**. Името на файла да се укаже като параметър на функцията **main()**. Да се прочита съдържанието на файла докато се стигне края на файла и да се изведе на екрана. Да се добавят към файла 2 нови низа. Разгледайте съдържанието на текстовия файл.

**Примерен вариант на решение:**

```
#include <fstream.h>
#include <iostream.h>
#include <stdlib.h>

int main(int argc, char *ime[]) // Името на файла се подава
    // като аргумент в командния ред при стартиране на програмата
{ if(argc!=2)
    {cout << "Задайте име на файл" << endl;
    return 1;}
    ofstream fp; // Създаване на файла
    fp.open(ime[1]);
    if(!fp)
    {cout << endl << "Файлът не може да се отвори" << endl;
    exit(1);}
    char str[80];
    cout<< endl<< "Запиши стринга във файла и $ за стоп" << endl;
    do
    {cout << ": ";
    cin.getline(str,80);
    fp << str <<endl;
    }while((*str) != '$'); // Записа продължава докато се въведе
    // $ за край

    fp.close();
    // Извеждане съдържанието на файла
    char ch;
    ifstream fp1;
    fp1.open(ime[1]);
    fp1.unsetf(ios::skipws); // Да не пропуска интервали
    while(!fp1.eof())
    {fp1 >> ch;
    cout << ch;}
    fp1.close();
    // Добавяне на 2 нови низа към файла
    fstream fp2;
```

```

    fp2.open(ime[1], ios::app);
    cout << endl << "Въведете стринг" << endl;
    cin.getline(str,80);
    fp2 << str <<endl;
    cout << endl << "Въведете стринг" << endl;
    cin.getline(str,80);
    fp2 << str << endl;
    fp2.close();
    return 0;
}

```

В този **вариант на решение** на задачата, обърнете внимание на няколко програмни реда:

- външното име на файла се предава като аргумент към функцията **main()** по време на стартиране на програмата. Ако липсва име, се извежда съобщението “Задайте име на файл”;
- проверката за коректно отваряне на файла се извършва **с просто** логическо условие **if(!fp)** ;
- при четене съдържанието на файла **symbol.txt**, се използва режима **ios::skipws** за да **не се пропускат** празните интервали и четенето продължава докато се стигне края на файла **while(!fp1.eof())** ;
- при добавяне на **2** нови низа в края на файла **symbol.txt**, се използва статуса за работа на файла **app** (за **добавяне** на данни в края на съществуващ файл): **fp2.open(ime[1], ios::app)**.

Съдържанието на файла **symbol.txt**, можете да разгледате с текстов редактор (например **Notepad**).

**Задача 11.5** Съставете програма, която записва в двоичен файл низ и две числа от тип **double**. Програмата да прочита данните от файла и да ги извежда на екрана.

#### Примерен вариант на решение:

```

#include <fstream.h>
#include <iostream.h>
#include <stdlib.h>
#include <string.h>

void main()
{
    char niz[80];
    double chislo1, chislo2;
    fstream fp1;
    fp1.open("chisla.dat",ios::binary| ios::out);
    if(fp1.fail())
    {
        cout << endl << "Файлът не може да се отвори"<< endl;
        exit(1);
    }
    cout << endl << "Въведете низ:";
    cin.getline(niz,80);
    int razmer = strlen(niz);
    fp1.write(niz,razmer);
    cout << endl << "Въведете число1:";
    cin >> chislo1;

```

```

fp1.write((char*)&chislo1,sizeof(double));
cout << endl << "Въведете число2:";
cin >> chislo2;
fp1.write((char*)&chislo2,sizeof(double));
fp1.close();
strcpy(niz,"");
chislo1=chislo2=0.0;
fp1.open("chisla.dat",ios::binary|ios::in);
fp1.read(niz,razmer);
fp1.read((char*)&chislo1, sizeof(double));
fp1.read((char*)&chislo2, sizeof(double));
fp1.close();
cout << endl << niz;
cout << endl << chislo1 << " " << chislo2 << endl;
}

```

В този **вариант на решение** се използва заглавния файл **string.h**. Това е необходимо при използване на функцията **strcpy()**, чрез която низа **niz** се нулира (празен низ). Преобразуването на тип към **(char\*)** е необходимо, когато се извежда **в буфер**, който не е дефиниран като **символен масив**. Указателят от даден тип не може автоматично да се преобразува в указател от друг тип.

**Задача 11.6.** Преработете програмата на **Задача 11.2.**, така че да използвате **C++** файлови функции.

**Примерен вариант на решение:**

```

#include <iostream.h>
#include <fstream.h>
#include <stdlib.h>
double massiv[10] = {1.34, 18.67, 1009.674, 666.88, 12.8,
                    0.0, 34.86, 22.222, 1.00001, 0.7777};

void main()
{
    int i;
    // Записване на елементите на масива във файла
    fstream fp;
    fp.open("out_massiv.dat", ios::out|ios::binary);
    if(fp.fail())
        {cout << endl << "Файлът не може да бъде отворен";
        exit(1);}
    fp.write((char*) massiv, sizeof(massiv));
    fp.close();
    for(i = 0; i < 10; i++)    // Нулиране на масива
        massiv[i] = 0.0;
    // Четене на данните от файла
    fp.open("out_massiv.dat", ios::in|ios::out);
    if(fp.fail())
        {cout << endl << "Файлът не може да се отвори";
        exit(1);}
    fp.read((char*) massiv, sizeof(massiv));
    fp.close();
    // Извеждане елементите на масива на екрана
    for(i = 0; i < 10; i++)
        cout << endl << massiv[i];
}

```

Когато се използва един **указател** на файла за **четене** и **запис** във файла (**fstream**), задължително в **open()** се посочва режима, в който ще работи файла.

**Задача 11.7.** Съставете **C++** програма за създаване на файл от 10 символа. Да се реализира възможност за замяна на конкретен номер символ с друг и да се изведе съдържанието на файла на екрана.

**Примерен вариант на решение:**

```
#include <fstream.h>
#include <iostream.h>
#include <stdlib.h>
#include <string.h>

void main()
{char simvoli[] = "Varna e mojat roden grad";
  char symbol;
  long pos;
  // Записва стринга във файла
  fstream fp;
  fp.open("sentence.dat",ios::out|ios::binary);
  if(fp.fail())
    {cout << endl << "Файлът не се отваря";
     exit(1);}
  fp.write(simvoli,strlen(simvoli));
  fp.close();
  // Смяна на символа
  cout << endl << "Въведете, коя позиция ще промените ";
  cin >> pos;
  fp.open("sentence.dat",ios::in|ios::binary|ios::out);
  if(fp.fail())
    {cout << endl << "Файлът не се отваря";
     exit(1);}
  fp.seekg(pos,ios::beg); // Позиционира файла върху позиция pos
  fp.get(symbol);
  cout << endl << "Символът е "<< symbol; // Визуализира
                                           // символа на позиция pos
  cout << endl<< "Въведете новия символ";
  cin >> symbol;
  fp.seekg(-1L,ios::cur); // Връщане една позиция назад
  fp.put(symbol);        // Заместване с новия символ
  fp.close();
  fp.open("sentence.dat",ios::in|ios::binary);
  if(fp.fail())
    {cout << endl << "Файлът не се отваря";
     exit(1);}
  fp.read(simvoli,strlen(simvoli)); //Четене на променения стринг
  fp.close();
  cout << endl << "Променен стринг: " << simvoli;
}
```

**Резултатът** се получава в следния вид:

Въведете коя позиция ще сменяте 9

Символът е o

Въведете новия символ \$

**Променен стринг : Varna e m\$jat roden grad**

**Символът 'о'** се явява **десети** символ в низа, но позициите на указателя на файла започват да се броят от **0** и в този случай тя е **9**.

Променливата **pos** е от **тип long**, тъй като функцията **seekg()** изисква като **параметър отместването** в байтове да е стойност от **тип long**. Тази функция поставя указателя на файла в началото на указаната позиция.

При прочитане на **символа 'о'**, указателят на файла се премества в началото на **символа 'j'**. Затова чрез:

```
fp.seekg(-1L, ios::cur)
```

**указателят на файла се връща една позиция назад** (отместването е **-1L** – назад спрямо текущата позиция на указателя на файла – **cur**), т. е. отново застава върху **'о'**.

Прекият (произволен) достъп до записите на файла се осъществява с помощта на функцията **seekg()**, като позициите спрямо които се извършва отместването са: **ios::beg** – спрямо **началото** на файла; **ios::cur** – спрямо **текущата** позиция на указателя на файла; **ios::end** – спрямо **края** на файла. Ако записите на файла са по-големи от **1** байт, то тогава може да се използва оператора **sizeof**.

```
fp.seekg(pos*sizeof(тип на данните), ios::beg);
```

Указателят на файла ще се **отмести** толкова **байта**, колкото е стойността на **произведението на pos и типа на данните**.

За осъществяване на **произволен достъп** до записите на двоичния файл, може да се използва и функцията: **tellg()**, която връща резултат **тип long** - **текущата позиция на указателя на файла**.

## **Примерен вариант на информационна система, използваща външен файл с данни.**

Следващата задача представлява една малка информационна система, включваща създаване на файл със записи тип структура.

**Задача 11.8.** Да се състави **C/C++** програма, която:

- създава двоичен файл със записи, представляващи данни за сътрудник във фирма: идентификационен номер на сътрудника, почасово плащане, брой отработени часове за една седмица, седмична заплата;
- включва възможност за допълване на нови записи във файла;
- включва възможност за извеждане на изчислената седмична заплата на екрана. При изчисляване на работната заплата е необходимо да се знае, че всеки извънреден час (над 40 часа седмично) се заплаща 150%, а стойността на данъците е 3,625% от общата заработка.
- включва меню в главната функция **main()**.

**Примерен вариант на решение:**

```
#include <iostream.h>
```

```

#include <fstream.h>
#include <conio.h>
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

const N=30; // Най-голям брой сътрудници
const char NAME[]="my.dat"; // Име на файл в текущата папка
struct person
{
    char idn[6]; // Идентификационен номер
    float rate; // Почасово заплащане
    int hours; // Отработени часове
    double zaplata; // Седмична заплата
};

int n; // Брой сътрудници
fstream fp; // Указател към файла

void syzdavane(struct person per[]); // Прототипи на функции
void izvejdane(struct person per[]);
void dopylvane();
void menu();

void main(void)
{
    menu();
}

void menu()
{
    struct person per[N]; // Масив от сътрудници
    int izbor; // За избор от менюто
    do{
        cout << endl<<"*****Меню*****" << endl;
        cout << "1.Създаване на файл" << endl;
        cout << "2.Извеждане на данните" << endl;
        cout << "3.Допълване" << endl;
        cout << "4.Край" << endl;
        cout << "Въведете своя избор от 1 до 4!" << endl;
        cin >> izbor;
    }while(izbor<1||izbor>4);
    switch(izbor){
        case 1: syzdavane(per); break;
        case 2: izvejdane(per); getch(); system("cls"); break;
        case 3: dopylvane(); break;
        default: cout<<endl<<"*****Край!*****"<<endl;
    }
    }while(izbor!=4);
}

/* Функция за въвеждане на данните за сътрудниците и
записването им във файл */
void syzdavane(struct person per[])
{
    cout << endl << "Въведете броя на сътрудниците: " << endl;
    cin >> n;
    fp.open(NAME,ios::binary|ios::out); // Двоичен файл за запис

```



```

    if(!fp)
    {cout << endl << "Грешка при създаване на файла:" << endl;
     exit(1);}
    for(int i=0;i<n;i++)
    {fflush(stdin); // Изчистване на входния буфер
     cout << endl << "Въведете данните за сътрудник номер "
      << i+1 << endl;
     cout << endl << "Идентификационен номер: ";
     cin >> per[i].idn;
     cout << endl << "Почасовото заплащане: ";
     cin >> per[i].rate;
     cout << endl << "Отработени часове: ";
     cin >> per[i].hours;
     per[i].zaplata=per[i].rate*per[i].hours; // Седмична заплата
    }
    fp.write((char*)per,n*sizeof(person)); /* Запис на масива във
                                           файл */
    fp.close();
}

/* Функция за четене на данните от файл, попълване на масива и
   извеждането им на екрана на монитора */
void izvejdane(struct person per[])
{int i=0; long pos;
 person p; // Локална променлива за един сътрудник
 cout << endl << "Пълен списък на сътрудниците: " << endl;
 fp.open(NAME, ios::binary|ios::in); // Файлът се отваря за четене
 if(!fp)
 {cout<< endl << "Грешка при прочитане на файла!" << endl;
  exit(1);}
 fp.seekg(0l,ios::end); // Премества указателя на файла в края
 pos=fp.tellg(); // Определя дължината на файла в брой байтове
 fp.close();
 fp.open(NAME, ios::binary|ios::in); // Указателят на файла се
 // позиционира в началото
 for(i=0; i<pos/(sizeof(person)); i++)
 {fp.read((char*)&p,sizeof(person)); // От файла се четат данните
 // за един сътрудник
 per[i]=p; // Попълване на масива с данни
 cout << endl << "idn: "<< per[i].idn;
 cout << endl << "Отработено време: " << per[i].hours;
 cout << endl <<"Коефициент на заплащане :"<<per[i].rate;
 if(per[i].hours>40)
 cout<<endl<<"Отработената заплата е: "<< per[i].zaplata +
 (0.5*per[i].zaplata)-(0.3625*per[i].zaplata)<<endl;
 else
 cout << endl << "Отработената заплата е: " << per[i].zaplata
 << endl;
 }
 fp.close();
}

// Функция за допълване на файла с данни за един сътрудник
void dopylvane()

```

```

{person pp; // Локална променлива с данните за един сътрудник
long pos;
fp.open(NAME, ios::binary|ios::in); // Файлът се отваря за четене
if(!fp)
{cout<< endl << "Грешка при прочитане на файла!" << endl;
exit(1);}
fp.seekg(0l,ios::end); // Премества указателя на файла в края
pos=fp.tellg(); // Определя дължината на файла в брой байтове
fp.close();
fp.open(NAME,ios::binary|ios::app); // Файлът се отваря за
// допълване

if(!fp)
{cout<< endl << "Грешка при отваряне на файла:" << endl;
exit(1);}
fflush(stdin); // Изчистване на буфера
cout << endl<<"Въведете данните за сътрудник номер "
<< pos/(sizeof(person))+1<<endl;
cout << endl << "Идентификационен номер: ";
cin >> pp.idn;
cout << endl << "Почасово заплащане: ";
cin >> pp.rate;
cout << endl << "Отработените часове: ";
cin >> pp.hours;
pp.zaplata=pp.rate*pp.hours;
fp.write((char*)&pp,sizeof(person));
fp.close();
}

```

Променливата **n** е дефинирана глобална, тъй като се използва от всички функции в програмата. Възможно е тя да се предава като параметър.

Главната функция **main()** извиква само функция **menu()**. Този подход може да се замени с премахване на **menu()** и поставяне на тялото на функция **menu()** изцяло в **main()**.

Обърнете внимание, че в **case 2** се използва командата **system("cls")**, чрез която се изпълнява DOS команда за изчистване на екрана.

Във функцията **syzdawane()** за запис на данните се използва **fp.write** във вида:

```
fp.write((char*)per,n*sizeof(person))
```

като елементите на масива **per** се записват наведнъж във файла, а **n\*sizeof(person)** е **общият размер** на данните в байтове, които те заемат във файла. Обърнете внимание във функцията **izvejdane()** за четене на данните за един сътрудник се използва **&p**:

```
fp.read((char*)&p,sizeof(person))
```

Функцията **izvejdane()** едновременно с извеждане на данните от файла, **попълва** и елементите на масива **per**. Използването на функциите **seekg()** и **tellg()** е необходимо, за да се определи броя записите във файла (**pos/sizeof(person)**).

Функцията **dopylwane()** не използва масива **per**, а само една локална променлива **pp**. Файлът се отваря за допълване посредством

**ios::app**, което поставя указателя на файла в края на файла и новите данни се записват чрез `fp.write( (char*) &pp, sizeof(person) );`.

### Въпроси и задачи за самостоятелна работа

1. Каква е разликата между текстов и двоичен файл?
2. Каква е разликата между запис и структура?
3. Какво става ако отворите за четене несъществуващ файл?
4. Къде сочи файловият указател, когато отворите файла в режим `ios::ate`?
5. Избройте всички начини, които преместват указателя на файла?
6. Какво ще се запише във файла `stuff.txt`, когато се изпълни следния програмен код?

```
ofstream fout;
fout.open("stuff.txt");
fout << "*" << setw(5) << 123 << "*" << 123 << "*" << endl;
fout.setf(ios::showpos);
fout << "*" << setw(5) << 123 << "*" << 123 << "*" << endl;
fout.unsetf(ios::showpos);
fout.setf(ios::left);
fout << "*" << setw(5) << 123 << "*" << setw(5) << 123 << "*"
    << endl;
```

7. Напишете C/C++ програма, която чете текстов файл и брои колко пъти се съдържа всяка от буквите от 'A' до 'Z'. Не правете разлика между малки и големи букви.
8. Напишете програма, копираща текстов файл. Използвайте `fputs()` и `fgets()` за да копирате файла. Включете пълна проверка за грешки.
9. Напишете програма, която позволява на потребителя да въведе толкова `double` стойности във файл, колкото желае. Озаглавете файла `VALUES`. Пребройте колко стойности са въведени и запишете броя им във файл, наречен `COUNT`.
10. Напишете програма, използваща `fwrite()` за записване във файл `RAND` на 100 случайни числа. Използвайки файла `RAND`, добавете програмен фрагмент, който чрез `fseek()` позволява да се изведе всяка стойност от файла.
11. Напишете програма, която размества всяка двойка символи в един текстов файл. Например, ако файлът съдържа "1234", то след изпълнение на програмата, файлът ще съдържа "2143" (файлът да съдържа четен брой символи).
12. Банка съхранява всичките си сметки във файл като данните са в следния вид: `nomer_na_smetkata, suma`. Напишете програма, която симулира банкомат. Потребителят може да внася пари, като зададе номера на сметката и сумата, да тегли пари, да се осведомява за сумата в сметката и да прехвърля от една сметка в друга.
13. Напишете програма, която конкатенира съдържанието на няколко файла с данни в един.

## Тема 12. Програми и функции с побитови операции. Управление на паметта.

**Цел:** Съставяне, изпълнение и анализ на програми с използване на побитови операции и динамично заемане на памет.

### Програми и функции с побитови операции

В случаи когато е необходим "достъп" до **отделни битове на целочислени/символни данни**, в C/C++ програмите се използват т. нар. **побитови операции** [4].

Поддържат се **шест** оператора за извършване на операции на **побитово ниво**:

| <u>Оператор</u> | <u>Действие</u>                                |
|-----------------|------------------------------------------------|
| &               | побитов AND (И)                                |
|                 | побитов OR (ИЛИ)                               |
| ^               | побитов XOR (изключващо ИЛИ - неравнозначност) |
| ~               | побитов NOT (НЕ - отрицание)                   |
| <<              | побитово изместване вляво (left shift)         |
| >>              | побитово изместване вдясно (right shift)       |

**Побитовите оператори** са валидни и могат да се използват **само** с данни от тип **char** и **int** (**целочислени**).

Подобно на логическите оператори, **действието** на **побитовите оператори** (без операторите за изместване) и **резултатите** от съответните **операции** могат да се представят с **таблицы на истинност**.

В Таблица 12.1. с **x** и **y** е означен отделен **бит** от **целочислен** операнд.

Таблица 12.1.

| x | y | x&y | x y | x^y | ~x | ~y |
|---|---|-----|-----|-----|----|----|
| 0 | 0 | 0   | 0   | 0   | 1  | 1  |
| 0 | 1 | 0   | 1   | 1   | 1  | 0  |
| 1 | 0 | 0   | 1   | 1   | 0  | 1  |
| 1 | 1 | 1   | 1   | 0   | 0  | 0  |

Забележете, че операторите **&** (AND), **|** (OR) и **^** (XOR) са **бинарни (двуоперандни)**, а операторът **~** (NOT) е **унарен (еднооперанден)**.

Операторите **побитов**: **AND** (&), **XOR** (^), **OR** (|) генерират **резултат**, базиран на сравнението на **всеки бит** от **първия** операнд със съответния **бит** от **втория** операнд.

Както виждате от Таблица 12.1., операторът **побитов AND** установява в **1** съответен **бит в резултата**, ако **и двата** сравнявани бита са **1**. Операторът **побитов OR** установява в **1** съответен **бит в резултата**, ако **поне един** от сравняваните битове е **1**. Операторът **побитов XOR** установява в **1** съответен **бит в резултата**, ако **един** от сравняваните **два бита** е **1**, но **не** и когато и **двата** са **1** или **0**.

Операторът **побитов NOT (~)** инвертира (от 0 в 1 и от 1 в 0) стойността на **всеки бит** на операнд от тип **цяло число** или **символ**.

Подобно на логическите оператори, **реда на изпълнение** на тези **побитови оператори** за реализация на съответни операции е както следва: ~ (**най-висок** приоритет), &, ^, | (**най-нисък** приоритет).

Операторите за **побитово изместване вляво (<<)** и **вдясно (>>)** са **бинарни**.

**Общата форма** на представяне на операциите с тези оператори е:

**операнд1 << операнд2**

**операнд1 >> операнд2**

Както вече разбрахте, **операнд1** и **операнд2** могат да бъдат **изрази** от **целочислен** или **символен** тип.

**Операнд2**, задаващ **броя битове** за **изместване**, определя на колко **позиции наляво** или **надясно** трябва да се **изместят съдържанията на битовете** на **операнд1**.

**Пример.** Фрагмент от програма с операции за **изместване**:

```
int a=66;    // Двоичният код на 66 е 01000010
int b=166;   // Двоичният код на 166 е 10100110
cout<<"\n Стойността на a след изместване вляво е "<<(a<<=1);
cout<<"\n Стойността на b след изместване вдясно е "<<(b>>=1);
```

**Резултати:**

Стойността на a след изместване вляво е 132 // 132(10)=10000100(2)

Стойността на b след изместване вдясно е 83 // 83(10)=01010011(2)

В изразите за извеждане на резултатите (**задължително** са в скоби) е използван съставен оператор за присвояване (<<= и >>=). Виждате, че **изместването вляво** е еквивалентно на **умножение по 2**, а **изместването надясно** е еквивалентно на **деление на 2**.

Операторите за **изместване** са с **по-нисък** приоритет от **аритметичните** оператори и с **по-висок** приоритет спрямо операторите за **сравнение** и **бинарните побитови** оператори.

**Побитовите** операции се **използват** основно за **формиране**, **преобразуване** и **анализ** на **цифрови кодове**, на **отделни битове** или **групи от битове** в данни от **целочислен** тип.

**Целите числа със знак** в паметта на компютъра обикновено се представят **двоично** в т. нар. **допълнителен код**. За **знака** на числото се използва **най-старшия** (най-левия) бит, който за **отрицателните цели числа** е със стойност **1**, а за **положителните** - със стойност **0**.

**Пример 1.** Използване на оператор **побитов OR** за превръщане на цяло положително число в отрицателно.

/\* Програма за превръщане на цяло положително число в отрицателно.

Използва се операция | (побитово OR), чрез която най-старшия бит на числото (определящ знака: 0 за положително или 1 за отрицателно число) се формира със стойност 1. Двоичният код на целочислената константа 32768 е 1000000000000000. \*/

```
#include <iostream.h>
#include <conio.h>
```

```

void main()
{int i;
  cout << "\n Въведете положително цяло число <=32767: ";
  cin >> i;  i = i|32768;      // Сменя знака на положително число
  cout << "\n Допълващото отрицателно число е " << i;
  getch();
}

```

При изпълнението на примерната програмата ще забележете, че при смяната на **знака** на въведеното положително число се **променя** и стойността на **числовата част**. Получава се отрицателно число, чиято стойност е "допълнението" до **-32768** (най-малката стойност на отрицателно **int** число за **16** битова програмна среда) на положителното (с обратен знак).

#### Примерно изпълнение:

Въведете положително цяло число <=32767: 16384  
 Допълващото отрицателно число е -16384

**Пример 2.** В следващия пример е представено още едно интересно свойство на операцията **XOR** (**побитово изключващо ИЛИ**, известно и като **сума по модул 2**) при двукратното ѝ последователно изпълнение. Използван е и съставен оператор за присвояване **^=**.

```

/* Програма за показване на едно от приложенията на операцията
побитово изключващо ИЛИ (оператор ^ – побитов XOR). След
двукратното ѝ изпълнение с две цели числа, стойността на резултата
е равна на стойността на първото число. */
#include <stdio.h>
#include <conio.h>

```

```

void main()
{ int i;
  printf("\n Въведете цяло число i < 32768: ");
  scanf("%d", &i);
  printf("\n Стойността на i след първия XOR е %d", i^66);
  printf("\n Стойността на i след втория XOR е %d", i^66);
  getch();
}

```

#### Примерно изпълнение:

Въведете цяло число i < 32768: 128  
 Стойността на i след първия XOR е 194  
 Стойността на i след втория XOR е 128

Проверете резултатите “ на ръка” като “оперирате” с числата в двоичен код: 128 => 10000000<sub>(2)</sub>, а 66 => 01100110<sub>(2)</sub>.

В представените по-долу примерни задачи и програми с побитови операции се демонстрират и други техни характеристики и приложения.

### Управление на паметта. Динамично заделяне на памет.

В практиката се срещат задачи и приложения, изискващи обработка на променливи с предварително неопределен брой елементи, както и

на типични **динамични структури данни** от типа на **свързани списъци, стекове, опашки, дървета** или **графи** [7]. При тези случаи, в **C/C++** програмите се използва **динамично заделяне на блокове** от паметта, наричана **динамична (heap) памет**. Съответните динамични променливи (структури), заемащи тази памет, се **създават и унищожават** не от компилатора, а по **време на изпълнение на програмата**, като при това могат да се "разширяват" или "свиват".

**Динамичната памет** се управлява чрез **указатели**, които указват съответните области от тази памет за **временно съхраняване на данни**.

За управлението на динамичната памет в **C** се използват библиотечни функции, дефинирани в хедърния файл **alloc.h**. Основните, използвани от тях функции са: **malloc()** - за динамично заделяне на памет и **free()** – за освобождаване на заделената памет.

**Общият вид** на деклариране на функцията **malloc()** е:

**void \*malloc(size\_t size);**

Функцията заделя блок с указан размер в байтове (толкова, колкото сте указали) от динамичната памет (**memory heap**).

При **успешно** заделяне, **malloc()** връща **указател** към началото на новозаделения блок от паметта. При **недостатъчно** налична свободна памет за новия блок, функцията връща **NULL**. При това съдържанието на блока остава непроменено. Функцията връща **NULL** и ако при извикването ѝ размера за **size == 0**.

За да освободите памет, извикайте функцията **free()** с указател към началото на блока от паметта (предварително заделен с **malloc()**), който искате да освободите.

Функцията не връща стойност и освобождава указания блок от паметта, заделен от предходно извикване на функции: **malloc()**, **calloc()** или **realloc()** (за тези и др. функции, вижте **Help** информацията за **alloc.h**).

Когато **завърши** изпълнението на една програма, цялата заделена за нея **памет се освобождава**.

Библиотечните функции **malloc()** и **free()** са декларирани и в хедърния файл **stdlib.h**. Въпреки че тези функции са достъпни и в **C++** програми, **C++** предлага по-удобен и безопасен метод за **заделяне и освобождаване** на динамична памет.

В **C++** можете да **заделяте** памет с помощта на оператора **new** и да я **освобождавате** с оператор **delete**.

**Пример 3.** Следващата **C** програма заделя **50** байта памет и асоциира **char** указател към тях. Така се създава **динамичен символен масив**, с който се извършват съответни обработки в програмата. Освобождаването на заетата за масива памет се извършва с извикването на функцията **free()**, която в случая не е задължителна.

**/\* C програма с динамично заделяне на памет за символен низ.**

**Низът се въвежда в заделената памет, извежда се на екрана, след което паметта се освобождава. \*/**

```

#include <iostream.h>
#include <stdlib.h>

void main()
{char *niz;
 niz = (char*)malloc(50); // Указателят сочи низа в динам. памет
 if(!niz) {
     cout << "\n Грешка при заделянето на памет!";
     exit(1);}
 cout << "\n Въведете Вашето име, презиме и фамилия: ";
 cin.getline(niz,50);
 cout << "\t Вие въведохте: " << niz;
 free(niz); // Освобождаване на паметта
}

```

Обърнете внимание на необходимостта от програмна проверка за успешно извикване на функцията **malloc()**, преди да използвате връщания указател.

**Пример 4.** С програма за приблизително определяне обема на свободната динамична памет.

/\* Програма за приблизително определяне обема на свободната динамична памет. Програмата заделя циклично 4096-байтови нови "блокове" памет докато malloc() върне null - динамичната памет е напълно заета. Адресът на всеки блок се извежда на екрана. \*/

```

#include <iostream.h>
#include <conio.h>
#include <stdlib.h>
#include <dos.h>

void main()
{unsigned *mem;
 long obem = 0; // За обема на свободната динамична памет
 clrscr();
 do{
     mem =(unsigned*)malloc(4096);
     cout << "\n Адресът в динамичната памет е " << (unsigned)mem;
     delay(500);
     if(mem) obem += 4096;
 }while(mem);
 cout << "\n\n Има " << 65536-obem << " байта свободна памет." ;
 getch();
}

```

Забелязвате, че в тази програма не е използвана функцията **free()** (не е задължителна, тъй като при излизане от програмата заделената динамична памет се освобождава). За обема на свободната динамична памет, която е от порядъка на **64** kB, е дефинирана целочислена променлива от тип **long**.

Въведете и изпълнете програмата като проследите процеса на последователно заделяне на блокове динамична памет. Ако не използвате временното представяне на адресите в десетичен код (чрез явното преобразуване - **(unsigned)mem**), извеждането им ще бъде в шестнадесетичен код.



## Примерни задачи и програми с побитови операции и динамично заделяне на памет

**Задача 12.1.** C/C++ програма с побитови операции, реализирани с 6-те побитови оператора [1].

```
/* C/C++ програма за демонстриране побитови операции, реализирани  
със съответни побитови оператори: ~ (NOT), << (изместване вляво),  
>> (изместване вдясно), & (AND), ^ (XOR), | (OR). Дефинират се две  
символни константи без знак и се извеждат на екран резултатите  
от съответните операции. Данните и резултатите се представят в  
шестнадесетичен код, като се показва и съответния двоичен код */  
#include <stdio.h>
```

```
void main()  
{unsigned char i = '\xFF';  
 unsigned char j = '\x59';  
 printf("\n Шестнадесетичната \n стойност на:");  
 printf("\n\t i е %x = 11111111", i);  
 printf("\n\t j е %x = 01011001", j);  
 printf("\n\t ~j е %x = 10100110", (unsigned char)~j);  
 printf("\n\t j<<1 е %x = 10110010", j<<1);  
 printf("\n\t j>>1 е %x = 00101100", j>>1);  
 printf("\n\t i&j е %x = 01011001", (unsigned char)i&j);  
 printf("\n\t i^j е %x = 10100110", (unsigned char)i^j);  
 printf("\n\t i|j е %x = 11111111", i|j);  
 getchar(); // Очаква натискане на Enter  
}
```

Използвани са символни променливи (**еднобайтови** - със стойности в **шестнадесетичен** код) за по-лесно проследяване на резултатите, представени и в еквивалентен **двоичен код**.

Забележете, че при **изместване наляво** с операция **j << 1**, всички битове на **j** се изместват с една позиция наляво, а **отдясно** се вкарва **0** (за **най-младшия** бит). При **изместване надясно** - **j >> 1**, стойностите на битовете се изместват надясно, а в **най-левия (най-старшия)** бит се вкарва **0**.

Обърнете внимание, че между изместването **наляво** и **надясно** има и още едно **различие**. Изместването **наляво** е **логическо**, а изместването **надясно** за **целите числа със знак** е **аритметично**, което означава, че стойността на **най-старшия бит** (използван за **знак - 1** за отрицателна стойност) **се запазва**. За **целите числа без знак (unsigned)** изместването **надясно** е **логическо**, тъй като **знаковия (най-старшия) бит** приема стойност **0**. Това различие е характерно и за съответните **асемблерни инструкции** (на ниво **процесор**) и е показано в **Задача 12.3**.

Резултатът от изпълнението на операцията **i&j** в програмата е **равен** на стойността на **j**, тъй като стойността на тази променлива "**маскира**" стойността на **i** (**11111111** в двоичен код) чрез **побитовия AND**. И обратно, резултатът от изпълнението на операцията **i|j** е **равен**

на стойността на *i*, тъй като стойността на тази променлива **формира** стойността на *j* (**01011001** в двоичен код) чрез **побитовия OR**.

**Задача 12.2.** Условието на задачата е представено в заглавния коментар на програмата.

```
/* C/C++ програма за извеждане двоичния код на символ. Използват се побитови оператори: & (AND) и >> ( изместване вдясно). Символът се сканира чрез побитово AND с изместваша се вдясно 1 (променлива i със стойност 128 = 10000000), от най-старшия до най-младшия бит. */
```

```
#include <iostream.h>
#include <conio.h>
```

```
void main()
{unsigned char ch;
 int i;
 clrscr();
 cout << "\n Въведете символ от клавиатурата: ";
 ch = getche(); // Въвеждане и показване на символ
 cout << "\n Десетичният код на символа "<<ch << " е: " << int(ch);
 cout << "\n Двоичният код на символа " << ch << " е: ";
 for(i=128; i>0; i>>=1) // Цикъл за извеждане на ch в двоичен код
     if(i & ch) cout << 1;
     else cout << 0;
 getch();
}
```

Проследете изпълнението на предложения вариант на програма и анализирайте резултатите за различни въведени символи.

**Задача 12.3.** Програма за демонстриране “поведението” на операторите за изместване. Използва се функция за двоично показване на стойностите на **16-** битови числа на екрана.

```
/* C/C++ програма за показване на резултати (в двоичен код) от побитово изместване вляво (оператор << ) и вдясно (оператор >> ) на цяло десетично число без знак. */
```

```
#include <iostream.h>
#include <conio.h>
```

```
void show_binary(unsigned n); //Прототип на функцията show_binary()
```

```
void main()
{ unsigned n;
 clrscr();
 cout << "\n Въведете цяло число n от 45000 до 65000: ";
 cin >> n;
 cout << "     Двоичен код на въведеното число:";
 show_binary(n);
 n <<=1;
 cout << "\n Двоичен код след изместване вляво:";
 show_binary(n);
 n >>=1;
 cout << "\n Двоичен код след изместване вдясно:";
```

```

    show_binary(n);
    getch();
}
// функция за извеждане двоичния код на цяло число без знак
void show_binary(unsigned n)
{ unsigned i;
  for(i=32768; i>0; i>>=1)
    if(i & n) cout << 1;
    else cout << 0;
}

```

Въведете и изпълнете предложената програма. Можете да модифицирате програмата и да анализирате резултатите при изместване на повече от една позиция, включително и на повече от "допустимия" брой измествания.

**Задача 12.4.** C/C++ програма за реализация чрез **функции** на операции за **ротация** (циклично побитово изместване) **наляво** и **надясно** (аналогични на съответни асемблерни инструкции на процесорно ниво).

```

/* C/C++ програма с функции за ротация наляво и надясно.
   Ротацията (циклично изместване) се извършва с еднобайтово
   цяло число без знак. В цикъл със запитване за продължение
   се избира лява или дясна ротация. Резултатите се показват
   в двоичен код. */
#include <iostream.h>
#include <conio.h>

// функция за извеждане двоичния код на цяло число без знак
void show_binary(unsigned n)
{ unsigned i;
  for(i=128; i>0; i>>=1)
    if(i & n) cout << 1;
    else cout << 0;
}

void rol(unsigned *ch)          // функция за лява ротация
{ int hbit;                    // За най-старшия бит
  if(*ch&0x80)                  // 0x80 = 10000000 двоичен код
    hbit = 1;                   // Проверка на най-старшия бит = 1?
  else
    hbit = 0;
  *ch <<= 1;                    // Ляво изместване - най-младшият бит става 0
  *ch |= hbit;                  // Най-старшия бит "се завъртва" в най-младшия
  cout << "    Двоичен код след лява ротация:";
  show_binary(*ch);
}

void ror(unsigned *ch)          // функция за дясна ротация
{ int lbit;                    // За най-младшия бит
  if(*ch&1)                     // Проверка на най-младшия бит=1?
    lbit = 1;
  else

```

```

    lbit = 0;
    *ch >>= 1;          // Дясно изместване - най-старшият бит става 0
    // Най-младшият бит "се завъртва" в най-старшия
    *ch |= (lbit << 7);    // Изместване със 7 позиции наляво
    cout << "    Двоичен код след дясна ротация:";
    show_binary(*ch);
}

```

```

void main()
{ unsigned n;                // Еднобайтово число без знак
  int r;                    // За посока на ротация
  char otg;                 // За отговор на запитване
  clrscr();
  cout << "\n Въведете цяло число n от 0 до 255: ";
  cin >> n;
  cout << "\n Двоичен код на въведеното число:";
  show_binary(n);
  do{
    cout << "\n За лява ротация въведете 0, за дясна 1: ";
    cin >> r;
    if(r) ror(&n);
    else rol(&n);
    cout << "\n Желаете ли да продължите? Въведете Y/N: ";
    otg=getche();
  } while(!(otg=='N' || otg=='n' || otg=='Y' || otg=='y'));
  getch();
}

```

За проследяване и анализ на резултатите при ротация на повече от една позиция е използван цикъл със запитване за продължение.

Както разбирате, във функциите **rol()** и **ror()** параметъра се предава **по адрес**. Възможно ли е предаване на **параметъра по стойност** и връщане на **резултат** от съответната функция?

**Задача 12.5.** Условието на задачата е представено в заглавния коментар. Във варианта на **C++** програма се използват операторите: **new** за заделяне и **delete** за освобождаване на динамична памет за масив от цели числа.

/\* C++ програма за въвеждане в масив os[n] (n<=30, брой студенти в групата - въвежда се от клавиатура) оценки по ПИК 1 (с проверка - 2 до 6), определяне и извеждане на екран броя на оценките: 2,3,4,5,6 и средния успех - със съответен текст, както и на въведените в масива оценки.

Вариант с използване на динамична памет за масива с оценки. \*/  
#include <iostream.h>  
#include <conio.h>

```

void main()
{int br[5]={0};            // Масив за броячи на оценките 2,3,4,5,6
  int i=0, n;              // n е за броя на студентите - оценките
  float sr=0.0;           // sr е за среден успех
  clrscr();
  do{ cout << "\n Въведете брой оценки <= 30: ";

```

```

    cin >> n;
    if(n<2 || n>30) {cout << "\n Грешка!"; getch();}
    else break;          // За излизане от цикъла
} while(1);
int *oc = new int[n]; //Заделяне на динамична памет за масив oc[n]
do{
    // Цикъл за въвеждане на оценки, сумиране и броене
    cout << " Оценка [" << i+1 << "]: ";
    cin >> oc[i];
    if(oc[i]>=2 && oc[i]<=6)
        {sr += oc[i];      // Добавяне на оценката
         br[oc[i]-2]++;    // Броячът на оценка 2/3/4/5/6 нараства с 1
          i++;
        }
    } while(i<n);
for(i=0; i<5; i++) // Цикъл за извеждане на броячите за оценките
    cout << "\n Брой оценки (" << i+2 << "): " << br[i];
cout << "\n\n Средният успех е: " << sr/n;
cout << "\n\n Оценките в масива са: \n";
for(i=0; i<n; i++)
    cout << " " << oc[i];
delete oc;          // Освобождаване на динамичната памет за oc[n]
getch();
}

```

Виждале, че не се дефинира масив (в “явен” вид), а чрез оператора **new** се заделя динамична памет за **масив**, чиито **размер** се определя по време на **изпълнение на програмата** (чрез въведения брой оценки).

Въведете и изпълнете програмата като анализирате резултатите за “прогнозирани” от вас оценки по ПИК 1 във вашата група.

**Задача 12.6.** С програма с временно създаване на буфери при входно/изходни файлови операции чрез динамично заделяне на памет. /\* Програма с динамично заделяне за временно създаване на буфери при входно/изходни операции с **fwrite()** и/или **fread()**. Програмата заделя динамична памет за 10 цели стойности, присвоява на тази памет 10 случайни числа, индексирайки указателят ѝ като масив. Следва презаписване на масива от числа на диска и освобождаване на паметта. След ново заделяне на памет, числата се прочитат от файла и се извеждат на екрана. \*/

```

#include <stdio.h>
#include <conio.h>
#include <stdlib.h>

void main()
{int i;
 int *mem;
 FILE *fp;
 mem = (int*)malloc(10*sizeof(int)); // Заделяне на памет
 if(!mem){
     printf("\n Грешка при заделяне на динамична памет");
     exit(1);}

```

```

randomize(); // Инициализация на генератора
for(i=0; i<10; i++) // Генериране на 10 случайни числа
    mem[i] = rand();
if((fp = fopen("masfile.dat", "wb")) == NULL) {
    printf("\n Файлът не може да бъде отворен!");
    exit(1);}
if(fwrite(mem, 10*sizeof(int), 1, fp) != 1) {
    printf("\n Грешка при запис във файла!");
    exit(1);}
fclose(fp);
free(mem); // Освобождаване на паметта
/* Извършване на други обработки */
mem = (int*)malloc(10*sizeof(int)); // Ново заделяне на памет
if(!mem) {
    printf("\n Грешка при заделяне на динамична памет");
    exit(1);}
if((fp = fopen("masfile.dat", "rb")) == NULL) {
    printf("\n Файлът не може да бъде отворен!");
    exit(1); }
/* Прочитане на масива от файла */
if(fread(mem, 10*sizeof(int), 1, fp) != 1) {
    printf("\n Грешка при четене от файла!");
    exit(1);}
fclose(fp);
clrscr();
printf("\n\t Съдържанието на генерираният масив е:\n");
for(i=0; i<10; i++) printf(" %d ", mem[i]);
free(mem);
getch();
}

```

За разлика от преходната задача, в този вариант на програма [1] се използват функции за динамично заделяне на памет, дефинирани в заглавния файл **stdlib.h** (или в **alloc.h**).

## Въпроси и задачи за самостоятелна работа

1. Каква е разликата между побитовите и другите логически операции?
2. От какво се определя стойността във всеки бит на резултата при изпълнение на побитова операция?
3. Каква е разликата между побитовите операции | (OR) и ^ (XOR)?
4. Какво става със стойността на най-старшия/най-младшия бит след изпълнение на побитова операция за изместване, съответно вляво/вдясно?
5. С какво се характеризира динамичното заделяне на памет и кога се прилага?
6. Съставете C/C++ програма за 16 битови цели числа, подобна на предложената за Задача 12.2 (със запитване за продължение).
7. Съставете ваш вариант на функции за ротация на 16 битови цели числа, като използвате тези функции в C/C++ програма.
8. Съставете вариант на C++ програма за Задача 12.6 като използвате дефинираните в **iostream.h** входни/изходни функции и операции.

## Приложение А. Програмна среда Borland C++.

**Цел:** Запознаване с интегрираната програмна среда **Borland C++**, основните режими на работа и използването ѝ при проектиране и изпълнение на линейни, разклонени и циклични програми.

### Основни сведения

Интегрираната програмна среда **Borland C++** е предназначена да осигури и улесни **съставянето, редактирането, проверката и настройките, транслирането, изпълнението, съхраняването и документирането на програми.**

Характеризира се с удобен **потребителски интерфейс**, включващ съответни **прозорци, менюта и команди, стандартизирани обработки, помощна информация**, както и възможности за работа с посочващо устройство (**манипулатор мишка**) и за **потребителски настройки.**

Средата за програмиране **Borland C++** се състои от следните **компоненти:**

1. Управляваща програма
2. Файлова система (меню **File**)
3. Текстов редактор (менюта **Edit** и **Search**)
4. Компилятор и Свързващ редактор (меню **Compile**)
5. Изпълнителна система (меню **Run**)
6. Система за проверка на програми (меню **Debug**)
7. Система за работа с проекти (меню **Project**)
8. Опции за настройки (меню **Options**)
9. Система за работа с прозорци (меню **Window**)
10. Система за помощна информация (меню **Help**)

**Управляващата програма** осигурява автоматичната, функционална връзка между **редактора и файловата система**, между **компилятора и редактора**, между **системата за проверка и изпълнителната система.**

**Файловата система** осигурява работата на средата за програмиране с **файлове (създаване, отваряне, съхраняване** – чрез съответните **команди** от меню **File**), организирани в **директории** (команда **Change Dir** – смяна на директория), превключване в среда на операционната система **DOS** (команда **DOS Shell**) и **излизане** (команда **Quit**) от средата **Borland C++.**

**Текстовият редактор** е с възможности за работа с мишка, управление на курсора, **премахване (Cut), копиране (Copy), вмъкване (Paste),** показване съдържанието на **“буферната” памет (Show Clipboard), изчистване на маркиран текст (Clear** – команди от меню **Edit**), **търсене и заместване на текст** (команди **Find...** и **Replace...** от меню **Search**) и други.

**Компиляторът и свързващият редактор** се третират като **обща компонента**. Транслираната програма, която се получава от работата на компилатора и свързващия редактор (примерно след изпълнение на команда **Compile**) се съхранява обикновено в оперативната **памет**, но може да бъде записана и в определена директория на диск/дискета или на Flash памет.

**Изпълнителната система** осигурява изпълнението на транслираната програма **изцяло** (команда **Run**) или **стъпково** (по **един** – команда **Trace Into** или **повече оператори** - команда **Step Over**) при проверка на програми.

**Системата за проверка** на програми осигурява възможности за **наблюдение на стъпково (по оператори) изпълнение на програми** и евентуална промяна на някои стойности на величини.

**Опциите за настройка** осигуряват задаване на **стойности на параметри**, при които работи **компилятора (Compiler)**, **свързващия редактор (Linker)**, **системата за проверка на програми (Debugger)**, **редактора (Editor)** и цялата **среда за програмиране (Environment)**.

Действията с различните видове **прозорци** (един или повече едновременно отворени) се осигуряват от **системата за работа с прозорци** (меню **Window**). **Прозорец на C/C++ програма** можете да **затворите** като щракнете зеленото “бутонче” (горе вляво на рамката).

## Работа в програмна среда Borland C++

При зададен **път** към изпълнимия файл **BC.EXE** (обикновено по подразбиране е **C:\BorlandC\BIN\BC.EXE** – за среда **DOS**), програмната среда се активира с командата:

**BC [име на файл] <Enter>**

В среда **Windows** по-удобно е активирането чрез щракване на съответна икона-препратка/икона на изпълнимия файл **BC.EXE**.

На екрана се появява **основният работен прозорец** с: **Главно меню**; **Заглавен ред** (с името на файла); **Основна работна област** (с ленти за превъртане – хоризонтална и вертикална); **Лента за състоянието (Status Bar)**.

**Главното меню** има вида:

**File Edit Search Run Compile Debug Project Options Window Help**

Първите букви на всяко от менютата на главното меню **са оцветени** в червено, което “подказва”, че чрез тях може да бъде **активирано** (отворено) съответното меню.

В **лентата за състоянието** се представя информация за изпълняваните действия, функции на съответните клавиши и други в зависимост от избраното меню и/или изпълнявана команда. При набиране и редактиране на програми лентата за състоянието има вида:

**F1 Help F2 Save F3 Open Alt-F9 Compile F9 Make F10 Menu**



**Меню от главното меню** се избира чрез натискане на клавиш **F10** и **позициониране** чрез **клавиш-стрелка** или чрез натискане на клавиш за “оцветената” буква от съответното меню. По-удобно е активиране на меню чрез едновременно натискане на клавиш **Alt** и клавиш за съответната **оцветена буква**.

**Най-бързо и удобно** е активиране на **желано меню чрез манипулатора мишка** – **позициониране и щракване**.

Подобни са начините за активиране на **команда** от отворено меню.

Поддържаните типове прозорци и работата с тях са подобни на използваната в **Windows** прозоречна техника.

Едни от най-характерните **диалогови съобщения** са за **грешки: синтактични** (Compile-time error или Syntax) и **грешки при изпълнение на програмата** (Run-time error).

При откриване на грешка, курсора **се позиционира в реда с грешка** като се извежда **съответно съобщение**.

## **Създаване и изпълнение на програми**

В зависимост от характера на основните обработки с данни, използваните **алгоритми и програми** (или техните съставни части) се класифицират като: **линейни** (последователно изпълнявани действия без избор на алтернативи), **разклонени** (с избор на една от две или от множество възможности за изпълнявани действия) и **циклични** (многократно повтарящи се – в зависимост от **условие** за край/продължение на цикъла, изпълнявани действия с данни).

Следващите елементарни примерни програми илюстрират основни елементи от **етапите** на тяхното **проектиране, анализ и изпълнение** в среда Borland C++.

**Първа програма** – файл с име **ZD1PA\_RC.cpp**: За **въвеждане** на **две цели числа А и В** от клавиатурата, **изчисляване** на **резултата С** (**реално число** - с **цяла и дробна част**, разделени с десетична точка) от делението им и **извеждането** му на екран.

**Примерна последователност:**

1. **Активиране** на главното меню и отваряне на файл **ZD1PA\_RC.cpp**:  
Натиснете клавиш **F10** и след него – **F3** или активирайте команда **File→Open** – за отваряне на файл;  
В полето **Name** за име на файл на **диалоговия прозорец** въведете **ZD1PA\_RC** (пред подразбиращото се разширение **.cpp**) и потвърдете с бутон **Open** или чрез клавиш **Enter**.

2. **Набиране** на програмата:

```
/*Примерна програма 1 за въвеждане и извеждане на две цели числа А и В и пресмятане и извеждане на частното им С, като реално число*/  
#include <stdio.h> // Директиви на предпроцесора за включване на  
#include <conio.h> // стандартни функции за въвеждане и извеждане
```

```
int main(void) // функция main()  
{ int a, b; // Дефиниране на А и В като цели променливи-тип integer  
  float c; // Дефиниране на С като реална променлива - тип float
```

```

clrscr();/* функция за изчистване на екрана, декларирана в
        conio.h */
printf("\n Въведете две цели числа: "); /* Подсказващ текст на
        екрана */
scanf("%d %d", &a, &b);/* функция за въвеждане на променливи от
        клавиатурата */
printf("\n Въведените две числа са: A = %d, B = %d", a, b);
c = float(a)/b; /* На променливата C се присвоява частното от
        делението на A и B. C float(a) числото A се
        преобразува в реално */
printf("\n Резултатът е C = %f", c); /* функция за извеждане */
printf("\n Натиснете клавиш... ");
getch(); /* За показване на екрана с данните - очаква натискане
        на клавиш */
return 0;
} // Край на функцията main()

```

### 3. Анализ на програмата:

Както се вижда, като структура програмата се състои от директиви на предпроцесора и функция **main()**. Функцията съдържа **описателна част** за данните и **изпълнима част** с оператори (т. нар. **тяло на функцията**, заградено с фигурни скоби: { - за начало и } – за край). **Всеки оператор** завършва с разделителя ; (точка и запетая).

Използвани са коментари с пояснителен текст, представяни във вида: **/\* <коментар> \*/** (т. нар. **C** коментари), или **// <коментар>** (т. нар. **C++** коментари, които се разполагат само на един ред!). Коментарите **не се изпълняват**, а са предназначени за потребителя.

За показване на екрана с данните (т. нар. **потребителски екран - User Screen**), в края на програмата е използвана стандартна функция **getch()** (дефинирана във файла с библиотечни функции **conio.h**), преди която със стандартна функция **printf()** се извежда съответен “подсказващ” текст. С натискането (“прочитането”) на произволен клавиш се осъществява връщане към екрана с програмата.

### 4. Съхраняване на диск/flash памет или дискета:

Активира се команда **Save** (от менюто **File**) или чрез клавиш **F2**. Файлът се съхранява с име **ZD1PA\_RC.cpp** на съответното устройство в текущата директория. Ако е необходимо да се съхрани под друго име се активира командата **Save As**, а за смяна на директорията е необходимо да се активира командата **Change dir....**

### 5. Компилиране на програмата:

Активира се менюто **Compile** (чрез **мишката** или **F10→C** или **Alt+C**) и се изпълнява командата **Compile**. Същата може да бъде изпълнена и чрез **комбинацията от клавиши Alt+F9**.

При наличие на **синтактични** грешки, спира процесът на **компиляция** и се извеждат съответните **диалогови съобщения**. Необходимо е коригиране на **синтактичните** грешки и прекомпилиране на редактираната програма (и съхраняване с **F2**).

За **търсене** на дума (част от текст в програмата), позициониране на избран ред от програмата, откриване на грешки (команда **Find Error...**)

и други подобни действия за търсене (и замяна) могат да се използват съответни команди от менюто **Search**.

#### **6. Изпълнение** на програмата:

Активира се командата **Run** от менюто **Run** или чрез комбинацията от клавиши **Ctrl+F9**. Появява се *потребителски екран (User Screen)* с подсказващия текст от програмата: **Въведете две цели числа:**. Числата се въвеждат от **един ред**, разделени **поне с един интервал**, или от **два отделни реда** (чрез натискане на клавиш **Enter**).

Появява се резултат във вида:  $\pm\text{XXXXX.XXXXXX}$  (където **X** е коя да е цифра от **0** до **9**, в цялата или дробната част) и се изчаква до натискане на клавиш.

За връщане на програмния изход (показване отново на *потребителския екран с резултатите*), активирайте командата **User Screen** от менюто **Debug** или клавишната комбинация **Alt+F5**.

**Изпълнете програмата няколко пъти с различни стойности на въвежданите целочислени стойности за променливите **A** и **B** и “проверете” получаваната като резултат стойност за **C**.**

Тъй като променливите **A** и **B** са дефинирани от типа **integer** (**int** - целочислени), стойностите които въвеждате могат да бъдат само в интервала от - **32768** до **32767** (за **16 битова** среда)! Опитайте варианти със стойности извън указания интервал и следете каква е стойността на резултата.

При съобщения за *грешки от изпълнение*, потърсете помощ чрез **F1** или чрез командата **Find Error...** от менюто **Search**. В такъв случай се налага *прередактиране* за отстраняване на грешките и повторно изпълнение на програмата.

Модифицирайте програмата като изтриете ключовата дума **float** в израза за изчисляване на резултата **C**. Анализирайте получавания резултат при въвеждане на **A** или на **B** с нечетна стойност.

**Втора програма** – файл с име **ZD2PA\_RC.cpp**. Програмата е със същото условие, изпълнявано **циклично** за **поредица** от две цели числа до въвеждане на **B=0**.

Ако сте излезли от **Borland C++**, използвайки командата **DOS Shell** от менюто **File**, можете да се върнете в **Borland C++** средата с набиране и изпълнение на командата **Exit** в командния ред на **DOS**.

**Модифицирайте** програмата **ZD1PA\_RC.cpp** както е показано по-долу (с "удебелен" шрифт са показани редовете за **промяна**).

Добавени са два програмни реда с програмната конструкция – **оператор за цикъл do...while**. Операторите в обхвата на цикъла (заградени с операторни скоби { и }, т. нар. **блок от оператори**) се изпълняват многократно докато **B** е различно от нула.

```
/* Примерна програма 2 за многократно (циклично) въвеждане и  
извеждане на две цели числа A и B (докато числото B е различно от  
нула) и пресмятане и извеждане на частното им C, като реално  
число. */  
#include <stdio.h>
```

```
#include <conio.h>

int main(void)
{ int a, b;
  float c;
  clrscr();
  do{          // Цикъл с постусловие. Изпълнява се докато В не е = 0
    printf("\n Въведете две цели числа: ");
    scanf("%d %d", &a, &b);
    printf("\n Въведените две числа са: A = %d, B = %d", a, b);
    c = float(a)/b;
    printf("\n Резултатът е C = %f", c);
    printf("\n Натиснете клавиш... \n");
    getch();
  }while(b != 0); // Край на цикъла
  return 0;
}
```

**Важно:** В предложения вариант на програма "умишлено" е включен **програмен ред с грешка!**

От менюто **File** активирайте командата **Save As**. В полето **Save File As** за име на файл на **диалоговия прозорец** въведете в същата директория **ZD2PA\_RC** (пред подразбиращото се разширение **.cpp**) и потвърдете с бутон **OK** или натиснете клавиш **Enter**.

Можете да компилирате и изпълните програмата чрез **Ctrl+F9** - **Borland C++** автоматично **компилира** програмата преди да я изпълни.

От **цикъла** се излиза при **B=0**. Но в такъв случай ще се получи **грешка** при изпълнение на програмата от **деление на нула!** Следва автоматично връщане в редактора. За да видите съобщението за грешка на потребителския екран, наберете **клавишната комбинация Alt+F5**. След текста за стойностите на въведените **две числа (B** е със стойност **0)** следва **съобщение за грешка по време на изпълнение на програмата:**

**Floating point error: Divide by 0. Abnormal program termination**

(Грешка – плаваща точка: Делене на 0. Ненормално приключване)

За да се **върнете в програмата**, наберете отново **Alt+F5** или натиснете клавиш.

За **отстраняване на грешки** и настройване на програмата можете да използвате **програмата Debugger**, позволяваща изпълнението на програмата по редове (**оператори**), т.нар. **"Трасиране"**, както и показване на текущите стойности на променливите.

За целта активирайте команда **Trace Into** от менюто **Run** или чрез клавиш **F7**. Започва **изпълнението на програмата по стъпки (оператори)** – при всяко натискане на **F7**, с превключване между **редакторския и потребителския екран** (където се налага).

В конкретния случай за да се "отстрани" **програмно възможността за поява на грешка при изпълнение от делене на нула**, програмата може да се **модифицира** във вида (с "удебелен" шрифт са показани редовете в програмата за **промяна**):

```

/* Примерна програма 3 за многократно (циклично) въвеждане и
извеждане на две цели числа A и B (докато числото B е различно от
нула) и пресмятане и извеждане на частното им C, като реално
число. Извършва се програмна проверка за B = 0 и се извежда
предупреждение. */
#include <stdio.h>
#include <conio.h>

int main(void)
{ int a, b;
  float c;
  clrscr();
  do{
    printf("\n Въведете две цели числа: ");
    scanf("%d %d", &a, &b);
    printf("\n Въведените две числа са: A = %d, B = %d", a, b);
    if(b == 0) // Проверка за B = 0?
      printf("\n\! Деление на 0! C е с неопределена стойност!!!");
    else{      // Начало на блок от оператори
      c = float(a)/b;
      printf("\n Резултатът е C = %f", c);
    }        // Край на блока от оператори
    printf("\n Натиснете клавиш... \n");
    getch();
  }while(b != 0);
  return 0;
}

```

Добавена е програмната конструкция за **избор** – в случая оператор **if...else** (т. нар. условен оператор). Ако **B = 0**, с функцията **printf( )** на екрана се извежда текст “Деление на 0! C е с неопределена стойност!!!” (с предупредителен – **alert**, звуков сигнал), иначе се изпълнява **блока от оператори** в обхвата на **else**.

Както в предходната програма, направете промените в **ZD2PA\_RC.cpp** и съхранете модифицираната програма във файл с име **ZD3PA\_RC.cpp**.

Можете да **изпълните** програмата или да я **трасирате**, включително и с използване на **Watch** прозорци, до въвеждане на **B** със стойност **нула**. **Анализирайте получаваните резултати**.

Следва вариант на примерна програма с допълнителна програмна проверка за **некоректни входни данни**. Цикълът се прекратява при **брой цифри** в **A > 4** или в **B > 4** или стойност на **B=0**. Дефинират се и се използват **символни низове** (едномерни масиви от символи), които се обработват със **стандартни функции**, дефинирани в заглавния файл **string.h**. Използвани са и стандартни функции за преобразуване, дефинирани във файла **stdlib.h** – в случая за преобразуване на **цяло число** – **integer** в **(to)** символен низ – **alphabet** (функция **itoa()**, съответно **A** в **ast** и **B** в **bst**) и за абсолютна стойност (функция **abs()**, съответно когато **A** или/и **B** е отрицателно).

Както в предходната програма, направете промените и съхранете модифицираната програма във файл с име **ZD4PA\_RC.cpp**.

```

/* Примерна програма 4 за многократно (циклично) въвеждане и
извеждане на две цели числа А и В (докато А или В са с по-малко от
5 цифри или В е различно от нула) и пресмятане и извеждане на
частното им С, като реално число. Цикълът се прекратява след
проверка за некоректни входни данни. */
#include <stdio.h>
#include <conio.h>
#include <string.h>      /* За функции със символни низове */
#include <stdlib.h>      /* За функции с преобразуване */
void main(void)
{ int a, b;
  float c;
  char ast[10], bst[10]; /* Дефиниране на символни низове с брой
                           символи до 10 */

  clrscr();
  while(1) {              /* Цикъл с предусловие. Изпълнява се при
                           коректни входни данни */
    printf("\n Въведете две цели числа с брой цифри до 4: ");
    scanf("%d %d", &a, &b);
    itoa(abs(a),ast,10); itoa(abs(b),bst,10); /* Преобразуване на
        абсолютните стойности на А и В (за отрицателни числа) в
        символни низове */
    if(strlen(ast)>4 || strlen(bst)>4 || b==0) /*Проверка за
        некоректни данни – брой цифри в ast или в bst > 4 или b=0 */
    {printf("\n\a Некоректни стойности на входните данни!!!");
      printf("\n Край на цикъла! Натиснете клавиш... \n");
      getch();
      break; // Прекратяване изпълнението на цикъла
    }
    printf("\n Въведените две числа са: А = %d, В = %d", a, b);
    c = float(a)/b;
    printf("\n Резултатът е С = %f", c);
    printf("\n Натиснете клавиш... \n");
    getch();
  } // Край на цикъла
}

```

Забележете, че е използвана програмната конструкция **while** – т. нар. **цикъл с предусловие** (изпълнява се блока от оператори в обхвата на цикъла докато изразът в скобите след **while** е **различен от нула**). В случая е използван т. нар. **безкраен** цикъл, изпълнението на който се прекратява с оператор **break** (**прекъсване**) при некоректни стойности на входните данни. Тъй като функцията **main()** е декларирана от тип **void** (**неопределен**), използването на оператор **return** (за връщане на резултат) не е задължително и той е “пропуснат”.

Компилирайте програмата и я изпълнете.

Опитайте “циклично” въвеждане на различни **целочислени** стойности за **входните променливи**, включително и извън допустимия интервал от **–9999 до 9999**. **Анализирайте получаваните резултати**.

Описаните елементи, менюта, команди и режими на работа, както и етапите на изпълнение на програми са подобни и за други програмни среди, например **Microsoft Visual C++ 6.0**.

## ИЗПОЛЗВАНА И ПРЕПОРЪЧВАНА ЛИТЕРАТУРА

1. Смърков В., Програмиране и използване на компютри – Borland C/C++, Учебник, ТУ – Варна, В., 2001/2006.
2. Смърков В., Божикова В., Програмиране и използване на компютри 2 – Borland C++/MS Visual studio 6.0, Ръководство за лабораторни и семинарни упражнения, ТУ – Варна, В., 2008.
3. Георгиева Ю. и други, Ръководство по програмиране I (C), Ciela, С., 1998/2004.
4. Шилдт Хърбърт, С - Практически самоучител, Софтпрес, С., 2001.
5. Пери Грег, С++. Програмиране в 101 примери, Paraflow, С., 1994/2005.
6. Хорстман Кай, Принципи на програмирането със С++, Софтех, С., 2000.
7. Рачева Е., Николов Н., Синтез и анализ на алгоритми, Ръководство за лабораторни упражнения, ТУ – Варна, В., 2008.
8. Microsoft Visual C++ Version 6.0. User's Manuel. C/C++ Language Reference.
9. Programming in C, <http://ei.cs.vt.edu/~cs1344/Examples/>
10. C++ Examples <http://www.cis.temple.edu/~hughes/code.html>
11. C++ examples, <http://www.functionx.com/cpp/examples/>
12. Hello C++, <http://www.hello-world.com/cpp/index.php>

**ISBN – ???**

## **ПРОГРАМИРАНЕ И ИЗПОЛЗВАНЕ НА КОМПЮТРИ 1**

**Ръководство за лабораторни упражнения  
(Borland C++/Microsoft Visual C++ 6.0)**

**Автори: Васил Йорданов Смърков, доц. д-р инж.  
Милена Николова Карова, гл. ас. д-р инж.  
Ганка Петкова Ковачева, гл. ас. д-р инж.  
Виолета Тодорова Божикова, гл. ас. д-р инж.  
Валентина Радославова Антонова, гл. ас. инж.**

**Рецензенти: Елена Викторовна Рачева, доц. д-р инж.  
Недялко Николаев Николов, доц. д-р инж.**

**Народност – българска  
Първо издание**